Programación de bases de datos.

Caso práctico

Juan recuerda, de cuando estudió el Ciclo de Desarrollo de Aplicaciones Informáticos, que había muchas tareas que se podían automatizar dentro de la base de datos mediante el uso de un lenguaje de programación, e incluso que se podían programar algunas restricciones a la hora de manipular los datos. **Juan** se lo comenta a **María** y ésta se muestra ilusionada con dicha idea ya que muchas veces repiten el trabajo con la base de datos de juegos on-line que tienen entre manos (consultas, inserciones, etc. que son muy parecidas y que se podrían automatizar).

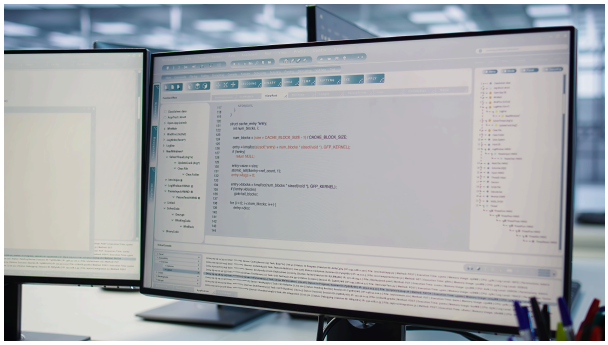
María (CC BY-NC-SA)

Para ello hablan con **Ada** y ésta les comenta que claro que se puede hacer y que precisamente eso es lo que les toca hacer ahora. **Ada** les dice que para ese propósito existe un lenguaje de programación llamado PL/SQL que permite hacer lo que ellos quieren, así que les pasa un manual para que se lo vayan leyendo y se vayan poniendo manos a la obra con la base de datos de juegos on-line.

Juan (CC BY-NC-SA)

En esta unidad aprenderás a **programar dentro de una base de datos Oracle utilizando PL/SQL**, combinando sentencias SQL con estructuras propias de un lenguaje de programación.

A lo largo del tema se trabajan desde los conceptos básicos —como **variables, constantes, bloques, estructuras de control y tratamiento de errores**— hasta contenidos más avanzados, como **colecciones, cursores, procedimientos, funciones, paquetes, objetos y disparadores**.



DC Studio. *Bases de datos* (CC BY-NC-SA)

También se estudia cómo reutilizar la lógica mediante **subprogramas**, cómo organizarla en **paquetes**, cómo reaccionar automáticamente ante cambios mediante **triggers** y cómo conectar Oracle con **aplicaciones desarrolladas en otros lenguajes**.

El objetivo es comprender cómo pasar de simples consultas SQL a programas capaces de **automatizar tareas, controlar errores, recorrer datos, aplicar reglas de negocio y**

mantener la información de forma más segura y organizada.

? Preguntas introductorias

1. ¿Qué ventajas crees que aporta **PL/SQL** frente a utilizar únicamente sentencias SQL?
2. ¿Para qué puede ser útil almacenar **procedimientos y funciones** directamente en la base de datos?
3. ¿Qué diferencia puede existir entre una **variable simple**, una **colección** y un **registro**?
4. ¿Por qué puede ser necesario utilizar un **cursor** cuando una consulta devuelve varias filas?
5. ¿Qué utilidad tienen las **excepciones** cuando se produce un error durante la ejecución de un programa?
6. ¿Qué ventajas puede aportar organizar varios procedimientos y funciones dentro de un **paquete**?
7. ¿En qué situaciones podría ser útil que una acción se ejecutara automáticamente mediante un **disparador o trigger**?
8. ¿Cómo podría una aplicación desarrollada en **Java, Python o .NET** utilizar procedimientos PL/SQL almacenados en Oracle?

👀 Debes conocer

La mayor parte de los ejemplos de esta unidad están basados en el modelo de datos extraído del siguiente caso de estudio:

Caso de estudio.

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN, FORMACIÓN PROFESIONAL Y DEPORTES

Ministerio de Educación, Formación Profesional y Deportes (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.

Aviso Legal

1.- Introducción.

Caso práctico

Miguel Martínez Monasterio (INTEF). *Taza de café* (CC BY-NC-SA)

Juan y **María** se han puesto a repasar el manual de PL/SQL que les ha pasado Ada. Aunque no han avanzado mucho con el mismo, ya saben a qué se van a enfrentar y los beneficios que pueden obtener del uso del mismo para su aplicación de juegos on-line. Cuando hacen la primera parada de la mañana para tomarse un café, ambos se ponen a comentar las primeras conclusiones que han sacado después de su primer acercamiento a este lenguaje. Ambos están deseosos de seguir avanzando en su aprendizaje y saben que para ello cuentan con la inestimable ayuda de **Ada**.

PL/SQL permite llevar la lógica de programación directamente a la base de datos. Partiendo de los conocimientos de SQL, añade variables, condiciones, bucles, procedimientos y control de errores, lo que facilita automatizar tareas y construir aplicaciones de base de datos más completas.

Puede parecer que PL/SQL supone aprender un lenguaje completamente nuevo, pero gran parte de su funcionamiento se apoya en **SQL**, por lo que el aprendizaje suele resultar más sencillo de lo que parece.

¿Qué es PL/SQL?

PL/SQL (Procedural Language/SQL) es el lenguaje procedimental de Oracle que amplía SQL mediante estructuras propias de programación.

Con PL/SQL podemos combinar:

- **Sentencias SQL** para consultar y modificar datos.
- **Variables y constantes**.

- **Estructuras condicionales y bucles**.
- **Procedimientos y funciones**.
- **Control y tratamiento de excepciones**.
- **Bloques anidados**.
- **Triggers o disparadores**.

De esta forma, SQL se encarga principalmente de trabajar con los datos y PL/SQL permite añadir la **lógica necesaria para procesarlos**.

PL/SQL se organiza en **bloques** que se ejecutan dentro del servidor Oracle. El motor de PL/SQL procesa las instrucciones procedimentales y envía al motor SQL las sentencias que necesitan acceder a la base de datos.

Ministerio de Educación. *Ejecución de PL/SQL.* (CC BY-NC-SA)

Aunque PL/SQL es específico de **Oracle Database**, otros gestores disponen de lenguajes procedimentales similares, como **PL/pgSQL en PostgreSQL** o **Transact-SQL en Microsoft SQL Server**.

Una ventaja importante es que gran parte del procesamiento puede realizarse **dentro del servidor de base de datos**. En determinados escenarios cliente-servidor, esto reduce los intercambios innecesarios entre aplicación y servidor y permite agrupar varias operaciones dentro de un mismo bloque.

Por tanto, PL/SQL no sustituye a SQL: lo **complementa**, permitiendo convertir consultas y operaciones aisladas en procesos completos que pueden reutilizarse y automatizarse.

Para saber más

En el siguiente enlace podrás encontrar una breve historia de PL/SQL.

Breve historia de PL/SQL.

En estos enlaces podrás comprobar como los gestores de bases de datos incluyen hoy día un lenguaje procedimental para programar la base de datos muy parecido a PL/SQL.

Procedimientos almacenados en MySQL.

Lenguaje procedimental en PostgreSQL.

Autoevaluación

Una empresa utiliza Oracle Database y quiere automatizar el cierre mensual. El proceso debe consultar varias tablas, calcular importes, recorrer determinados registros, tomar decisiones según el resultado y registrar cualquier error que se

produzca. Actualmente la aplicación cliente ejecuta muchas sentencias SQL por separado contra el servidor.

¿Cuál sería la solución más adecuada?

- ○ Mantener todas las sentencias SQL independientes, porque PL/SQL no permite utilizar estructuras condicionales ni controlar errores.
- ○ Crear un bloque o procedimiento PL/SQL que combine sentencias SQL con variables, bucles, condiciones y tratamiento de excepciones, ejecutando la lógica principalmente en el servidor.
- ○ Sustituir Oracle por PostgreSQL, ya que PL/SQL solo sirve para ejecutar consultas simples y no permite procedimientos ni triggers.
- ○ Crear únicamente una vista SQL, porque una vista puede recorrer registros, controlar excepciones y ejecutar lógica procedimental compleja.

Incorrecta. PL/SQL incorpora precisamente condiciones, bucles, variables y tratamiento de excepciones, además de sentencias SQL.

Correcta. Un bloque o procedimiento PL/SQL permite agrupar la lógica y las operaciones SQL en el servidor, facilitando la automatización y reduciendo intercambios innecesarios con el cliente.

Incorrecta. PL/SQL es el lenguaje procedimental de Oracle y permite crear procedimientos, funciones, triggers y otras estructuras avanzadas.

Incorrecta. Una vista permite representar una consulta, pero no sustituye a un lenguaje procedimental con bucles, decisiones y control de errores.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

2.- Conceptos básicos.

Caso práctico

Juan ha avanzado muy rápido en la lectura del manual que le pasó **Ada**. **Juan** ya sabe programar en otros lenguajes de programación y por tanto la lectura de los primeros capítulos que se centran en cómo se estructura el lenguaje, los tipos de datos, las estructuras de control, etc. le han resultado muy fáciles de comprender. Sabe que lo único que tendrá que tener en cuenta son algunos aspectos en cuanto a las reglas de escritura y demás, pero la lógica es la de cualquier otro lenguaje de programación.

INTEF. *Vocales.* (CC BY-NC-SA)

Como **María** está un poco más verde en el tema de la programación, **Juan** se ha ofrecido a darle un repaso rápido a todo lo que él ha visto y a explicarle todo aquello en lo que tenga dudas y a ver si pronto se pueden poner manos a la obra con la base de datos de juegos on-line.

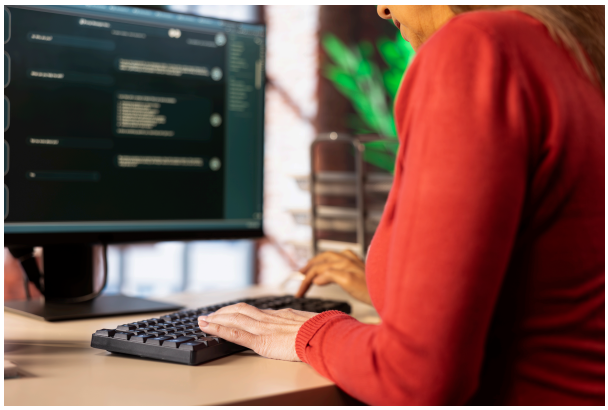
Antes de empezar a programar en PL/SQL es importante conocer sus elementos básicos. Dominar la sintaxis, los tipos de datos, las estructuras de control y el tratamiento de errores permitirá escribir bloques claros, correctos y fáciles de mantener.

En este apartado nos vamos

En este apartado se introducen de forma progresiva los conceptos necesarios para comenzar a programar en **PL/SQL**.

Como en cualquier lenguaje de programación, será necesario conocer:

- Las **reglas de sintaxis**.
- Los principales **elementos del lenguaje**.
- Los **tipos de datos** disponibles.
- Las estructuras **condicionales**.
- Las estructuras **iterativas o bucles**.
- El **tratamiento de errores y excepciones**.



DC Studio. *PL/SQL* (CC BY-NC-SA)

A partir de estos conceptos será posible crear pequeños bloques de código y avanzar posteriormente hacia **procedimientos, funciones y programas más completos** dentro de Oracle Database.

a ir introduciendo poco a poco en los diferentes conceptos que debemos tener claros para programar en PL/SQL. Como para cualquier otro lenguaje de programación, debemos conocer las reglas de sintaxis que podemos utilizar, los diferentes elementos de que consta, los tipos de datos de los que disponemos, las estructuras de control que nos ofrece (tanto iterativas como condicionales) y cómo se realiza el manejo de los errores.

Como podrás comprobar, es todo muy sencillo y pronto estaremos escribiendo fragmentos de código que realizan alguna tarea particular. ¡Vamos a ello!

Autoevaluación

¿Es verdadero o falso?

Un alumno empieza a escribir un bloque PL/SQL utilizando variables, pero no respeta la sintaxis del lenguaje y asigna valores incompatibles con el tipo de dato declarado. Afirma que, mientras la lógica del programa sea correcta, Oracle podrá ejecutar igualmente el bloque.

○ Verdadero ○ Falso

Falso

En PL/SQL es necesario respetar tanto las reglas de sintaxis como los tipos de datos. Un error en cualquiera de estos aspectos puede impedir que el bloque se compile o se ejecute correctamente.

Un programa debe comprobar el saldo de varias cuentas y aplicar una acción distinta según el valor de cada una. El programador afirma que para resolverlo será necesario utilizar únicamente sentencias SQL, porque PL/SQL no dispone de estructuras condicionales ni bucles.

○ Verdadero ○ Falso

Falso

PL/SQL incorpora estructuras condicionales y estructuras iterativas, por lo que permite tomar decisiones y repetir instrucciones dentro de un bloque.

Un procedimiento realiza varias operaciones sobre una base de datos y puede producirse un error durante la ejecución. El desarrollador decide incluir un bloque de tratamiento de excepciones para controlar ese fallo y responder de forma adecuada.

○ Verdadero ○ Falso

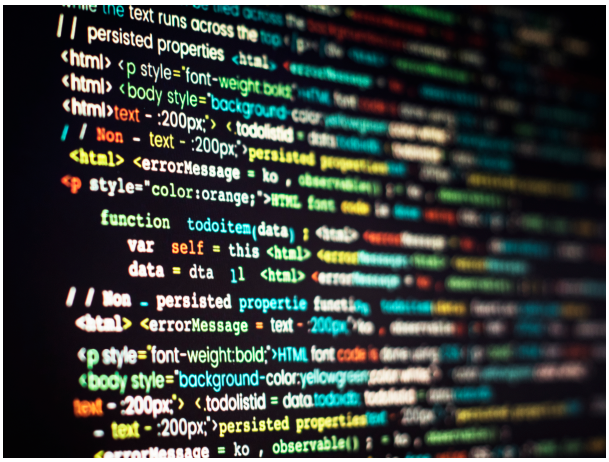
Verdadero

El tratamiento de errores y excepciones es uno de los elementos fundamentales de PL/SQL. Permite detectar situaciones anómalas y definir qué debe hacer el programa cuando se producen.

2.1.- Unidades léxicas (I).

Las unidades léxicas son las piezas básicas con las que se construye cualquier programa PL/SQL. Conocer identificadores, literales, delimitadores y comentarios ayuda a escribir código correcto, legible y fácil de mantener.

En PL/SQL, las **unidades léxicas** son los elementos más pequeños con significado propio. Al combinarse siguiendo las reglas de sintaxis forman sentencias válidas.



rawpixel.com. *Unidades léxicas* (CC BY-NC-SA)

PL/SQL **no distingue entre mayúsculas y minúsculas** en palabras clave e identificadores no entrecomillados. Por ejemplo, `IF`, `if` e `If` se interpretan igual. La excepción son los **literales de texto**, donde sí se conserva el contenido escrito.

Estos fragmentos son equivalentes:

```
IF A=CLAVE THEN ENCONTRADO:=TRUE;ELSE ENCON
```

Sería equivalente a escribir la siguiente línea:

```
if a=clave then encontrado:=true;else encontrado:=false;end if;
```

Sin embargo, una escritura estructurada mejora la legibilidad:

```
IF a = clave THEN
     encontrado := TRUE;
ELSE
     encontrado := FALSE;
END IF;
```

Las unidades léxicas principales son:

- **Delimitadores**
- **Identificadores**
- **Literales**
- **Comentarios**

Delimitadores

Los **delimitadores** son símbolos que permiten realizar operaciones, separar elementos, asignar valores o marcar comentarios.

Símbolo	Significado
+	Suma
-	Resta o negación
*	Multiplicación
/	División
**	Exponenciación
=	Igualdad
<> o !=	Distinto
< > <= >=	Comparaciones
:=	**Asignación**
`	
..	Rango
,	Separador
;	Fin de sentencia
.	Selector
%	Indicador de atributo
--	Comentario de una línea
/* ... */	Comentario de varias líneas
=>	Asociación de parámetros

Conviene prestar especial atención a **:=**, porque en PL/SQL se utiliza para **asignar valores**, mientras que **=** se emplea para realizar **comparaciones**.

Autoevaluación

Un alumno escribe el siguiente fragmento PL/SQL:

```
IF usuario = 'ADMIN' THEN
    acceso = TRUE;
    -- Acceso autorizado
END IF;
```

El código no funciona como esperaba y revisa las unidades léxicas utilizadas. ¿Qué modificación es la más adecuada?

- ○ Cambiar IF por if, porque PL/SQL distingue entre mayúsculas y minúsculas en las palabras clave.
- ○ Cambiar 'ADMIN' por 'admin', porque los literales de texto no distinguen entre mayúsculas y minúsculas.
- ○ Cambiar acceso = TRUE; por acceso := TRUE;, porque := realiza asignaciones y = se utiliza para comparaciones.
- ○ Eliminar -- Acceso autorizado, porque PL/SQL no permite comentarios dentro de un bloque.

Incorrecta. PL/SQL no distingue entre mayúsculas y minúsculas en palabras clave e identificadores no entrecomillados. IF, if e If son equivalentes.

Incorrecta. Los literales de texto sí conservan su contenido, por lo que 'ADMIN' y 'admin' no tienen por qué representar el mismo valor.

Correcta. En PL/SQL, := es el operador de asignación, mientras que = se utiliza para comprobar igualdad.

Incorrecta. -- introduce un comentario de una línea y puede utilizarse dentro del código PL/SQL.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta
4. Incorrecto

2.1.1.- Unidades léxicas (II).

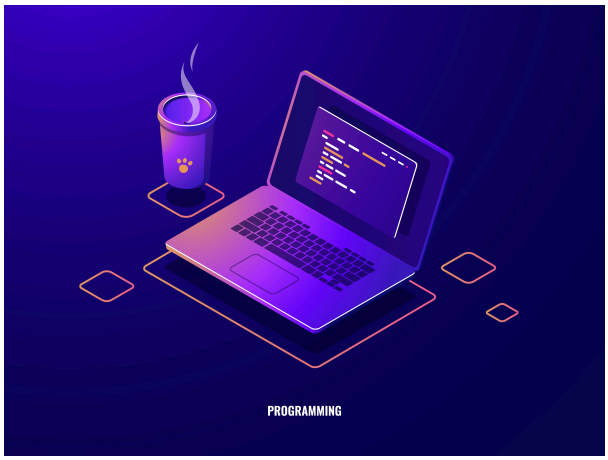
Identificadores, literales y comentarios son elementos básicos de cualquier bloque PL/SQL. Saber cómo nombrar variables, representar valores y documentar el código ayuda a evitar errores y hace los programas más claros y fáciles de mantener.

Identificadores

Los **identificadores** se utilizan para dar nombre a variables, constantes, procedimientos, funciones y otros elementos del programa.

Al utilizarlos hay que tener en cuenta:

- Un identificador normal comienza por una **letra** y puede continuar con letras, números, _, $ o #.
- No puede coincidir con una **palabra reservada** de PL/SQL.
- Ejemplos válidos: `x`, `a1`, `codigo_postal`.
- Ejemplos no válidos: `rock&roll`, `on/off`.



fullvector. *Identificadores* (CC BY-NC-SA)

También pueden utilizarse **identificadores entre comillas dobles**, que permiten nombres con caracteres especiales o coincidentes con palabras reservadas:

```
1 | "X*Y"
```

Este tipo de identificadores debe utilizarse con cuidado, porque obliga a respetar exactamente su escritura.

Algunas palabras pueden ser válidas como nombres de columnas en SQL, pero causar conflictos en PL/SQL. En esos casos pueden delimitarse con comillas dobles:

```
1 | SELECT "TYPE" ...
```

Literales

Los **literales** representan valores concretos utilizados en asignaciones y comparaciones.

Los principales son:

- **Numéricos:** `234`, `7.45`, `2E3`, `-2E-3`.
- **Texto:** se escriben entre **comillas simples**, por ejemplo `'Madrid'`.
- **Lógicos:** `TRUE` y `FALSE`.
- **NULL:** representa la **ausencia de un valor conocido**.

Por ejemplo:

```
nombre := 'Ana';
activo := TRUE;
```

Comentarios

Los **comentarios** permiten explicar el código sin afectar a su ejecución. Son especialmente útiles cuando un programa debe mantenerse o modificarse posteriormente.

PL/SQL permite dos formas principales:

- **Una línea:**

```
a := b;  -- Asignación
```

- **Varias líneas:**

```
/* Cálculo del importe
   antes de aplicar descuentos */
```

Un código bien comentado debe explicar **por qué se hace algo**, evitando repetir simplemente lo que ya indica la propia instrucción.

Ejercicio resuelto

Dada la siguiente línea de código, haz su descomposición en las diferentes unidades léxicas que contenga.

```
IF A <> B
THEN iguales := FALSE; --No son iguales
```

Mostrar retroalimentación

La descomposición en unidades léxicas sería la siguiente:

- Identificadores: **A**, **B**, **iguales**.
- Identificadores (palabras reservadas): **IF**, **THEN**.
- Delimitadores: **<>**, **:=**, **;**.

- Comentarios: **--No son iguales.**

Autoevaluación

¿Es verdadero o falso?

Un desarrollador necesita utilizar en PL/SQL una columna llamada TYPE, creada anteriormente en una tabla. Al escribir el código obtiene un conflicto y decide referirse a ella como "TYPE".

○ Verdadero ○ Falso

Verdadero

Algunos nombres pueden entrar en conflicto con palabras reservadas o identificadores especiales de PL/SQL. En esos casos, utilizar comillas dobles permite tratar el nombre como un identificador delimitado.

Un programador escribe:

nombre := "Ana";
activo := 'TRUE';
y afirma que ambas asignaciones son correctas porque tanto los textos como los valores lógicos pueden escribirse entre comillas.

○ Verdadero ○ Falso

Falso

Los literales de texto deben escribirse entre comillas simples, por ejemplo 'Ana'. En cambio, TRUE y FALSE son literales lógicos y no deben escribirse como cadenas de texto.

En un procedimiento complejo, un programador añade comentarios como:

/* Se comprueba primero el límite
para evitar aplicar el descuento dos veces */
Otro desarrollador afirma que este comentario puede cambiar el resultado del programa porque ocupa varias líneas dentro del bloque PL/SQL.

○ Verdadero ○ Falso

Falso

Los comentarios delimitados por /* ... */ pueden ocupar varias líneas y no afectan a la ejecución del programa. Su función es mejorar la comprensión y facilitar el mantenimiento del código.

2.2.- Tipos de datos simples, variables y constantes.

Elegir correctamente los tipos de datos en PL/SQL es fundamental para almacenar información con precisión, evitar errores y facilitar el mantenimiento del código. También es importante conocer %TYPE, ya que permite declarar variables compatibles con columnas de la base de datos sin repetir manualmente su definición.

En PL/SQL, cada variable o constante tiene asociado un **tipo de dato**, que determina qué valores puede almacenar, cuánto espacio necesita y qué operaciones se pueden realizar con ella.

macrovector. *Tipos de datos* (CC BY-NC-SA)

Tipos numéricos

- NUMBER: es el tipo numérico más utilizado. Permite indicar **precisión y escala**. Por ejemplo, NUMBER(6,2) admite hasta 6 dígitos, 2 de ellos decimales.
- PLS_INTEGER: está pensado para trabajar con **enteros** de forma eficiente en cálculos PL/SQL.
- BINARY_INTEGER: tipo entero tradicional de PL/SQL. Actualmente suele preferirse PLS_INTEGER.

Tipos de texto y binarios

- CHAR(n): almacena cadenas de **longitud fija**.
- VARCHAR2(n): almacena cadenas de **longitud variable** y es uno de los tipos más utilizados para texto.
- RAW: permite almacenar datos binarios.
- LONG y LONG RAW: tipos antiguos que se mantienen principalmente por compatibilidad. En nuevas aplicaciones se prefieren **CLOB y BLOB**.

Objetos de gran tamaño

- BLOB: almacena grandes cantidades de datos binarios, como imágenes o documentos.
- CLOB: almacena grandes cantidades de texto.
- BFILE: mantiene una referencia a un fichero externo almacenado en el sistema de archivos.

Otros tipos

- BOOLEAN: admite los valores TRUE, FALSE y también NULL.
- DATE: almacena **fecha y hora** hasta segundos.
- Oracle también dispone de tipos como TIMESTAMP cuando se necesita mayor precisión temporal.

Atributo %TYPE

El atributo %TYPE permite declarar una variable utilizando automáticamente el mismo tipo de dato que una **columna de una tabla** o que otra variable.

Por ejemplo:

```
DECLARE
    nombre_oficina oficinas.nombre%TYPE;
    vx NUMBER(5,2);
    vy vx%TYPE;
```

En este caso:

- **nombre_oficina** tendrá el mismo tipo que **oficinas.nombre**.
- **vx** puede almacenar 5 dígitos, 2 de ellos decimales.
- **vy** tendrá exactamente el mismo tipo que **vx**.

La principal ventaja de %TYPE es que mejora el **mantenimiento del código**: si cambia el tipo de la columna de la base de datos, la variable asociada puede adaptarse sin tener que modificar manualmente su declaración.

Autoevaluación

Una aplicación PL/SQL debe leer el valor de la columna salario de la tabla empleados. Actualmente esa columna está definida como NUMBER(8,2), pero el equipo de desarrollo sabe que su definición podría cambiar en el futuro. Además, necesitan almacenar un contador entero para recorrer registros y una descripción de longitud variable.

¿Cuál sería la declaración más adecuada?

○

```
DECLARE
    v_salario NUMBER(8,2);
    contador NUMBER;
    descripcion CHAR(200);
```

○

```
DECLARE
    v_salario empleados.salario%TYPE;
    contador PLS_INTEGER;
    descripcion VARCHAR2(200);
```

```
DECLARE
    v_salario BLOB;
    contador VARCHAR2(10);
    descripcion RAW(200);
```

```
DECLARE
    v_salario empleados.salario;
    contador BOOLEAN;
    descripcion BFILE;
```

Incorrecta. Aunque podría funcionar inicialmente, declarar v_salario manualmente como NUMBER(8,2) obliga a modificar el código si cambia la definición de la columna. Además, CHAR(200) reserva una longitud fija aunque no siempre sea necesaria.

Correcta. %TYPE mantiene la variable compatible con empleados.salario, PLS_INTEGER es adecuado para cálculos con enteros y VARCHAR2 permite almacenar texto de longitud variable.

Incorrecta. BLOB y RAW están orientados a datos binarios, mientras que VARCHAR2 no es el tipo más apropiado para un contador numérico.

Incorrecta. Para heredar el tipo de una columna debe utilizarse %TYPE. Además, BOOLEAN almacena valores lógicos y BFILE referencia ficheros externos, por lo que no son adecuados para el contador y la descripción.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

2.2.1.- Subtipos.

Los subtipos permiten dar nombres más claros y significativos a tipos de datos ya existentes en PL/SQL. Utilizarlos mejora la legibilidad del código y ayuda a representar mejor el significado de los datos sin cambiar su comportamiento.

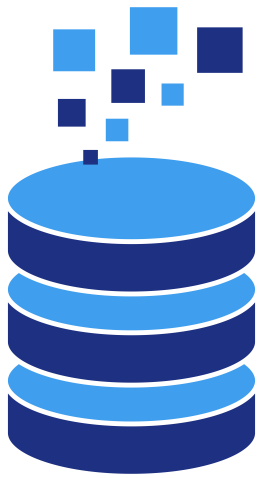
PL/SQL permite definir **subtipos** a partir de tipos de datos existentes. El subtipo mantiene las mismas operaciones y características básicas que su tipo de origen, pero puede utilizar un nombre más descriptivo.

La sintaxis general es:

```
SUBTYPE nombre_subtipo IS tipo_base;
```

Por ejemplo:

```
SUBTYPE importe IS NUMBER;
SUBTYPE activo IS BOOLEAN;
```



juicy_fish. *Subtipos* (CC BY-NC-SA)

También puede utilizarse **%TYPE** para basar el subtipo en el tipo de una variable o de una columna:

```
SUBTYPE id_familia IS familias.identificador%TYPE;
```

De esta forma, **id_familia** tendrá el mismo tipo que la columna **identificador** de la tabla **familias**.

Con **%ROWTYPE** puede definirse un subtipo basado en la estructura completa de una fila:

```
SUBTYPE agente IS agentes%ROWTYPE;
```

Así, una variable de tipo **agente** podrá contener una fila completa de la tabla **agentes**.

En PL/SQL, un subtipo definido directamente con **SUBTYPE** **no puede añadir restricciones de tamaño o precisión** al tipo base. Si se necesita conservar una restricción concreta, puede utilizarse una variable previamente declarada junto con **%TYPE**:

```
aux VARCHAR2(20);
SUBTYPE apodo IS aux%TYPE;
```

Los subtipos siguen siendo **compatibles con su tipo base**, por lo que pueden utilizarse en asignaciones y operaciones siempre que los tipos sean compatibles.

Por ejemplo:

```
SUBTYPE encontrado IS BOOLEAN;
SUBTYPE resultado IS BOOLEAN;

lo_he_encontrado encontrado;
resultado_busqueda resultado;
```

Ambas variables son compatibles porque los dos subtipos proceden de **BOOLEAN**.

Autoevaluación

¿Es verdadero o falso?

En una aplicación PL/SQL se quiere representar el identificador de una familia con un nombre más claro. El desarrollador crea:

SUBTYPE id_familia IS familias.identificador%TYPE;
y afirma que, si cambia el tipo de la columna identificador, el subtipo seguirá tomando automáticamente ese mismo tipo.

○ Verdadero ○ Falso

Verdadero

Al utilizar %TYPE, el subtipo queda vinculado al tipo de la columna indicada. Esto facilita el mantenimiento y evita repetir manualmente su definición.

Un programador necesita un subtipo llamado apodo limitado a 20 caracteres y escribe directamente:

SUBTYPE apodo IS VARCHAR2(20);
Afirma que esta es la forma correcta de añadir una restricción de longitud al subtipo.

○ Verdadero ○ Falso

Falso

Según el contenido, un subtipo definido directamente con SUBTYPE no puede añadir restricciones de tamaño o precisión. Para conservar una

restricción concreta puede utilizarse una variable auxiliar y después %TYPE.

En un bloque PL/SQL se definen:

SUBTYPE encontrado IS BOOLEAN;
SUBTYPE resultado IS BOOLEAN;
Después se asigna una variable de tipo resultado a otra de tipo encontrado. El programador afirma que la asignación es válida porque ambos subtipos proceden del mismo tipo base.

○ Verdadero ○ Falso

Verdadero

Los subtipos siguen siendo compatibles con su tipo base. Como encontrado y resultado derivan de BOOLEAN, pueden utilizarse en asignaciones compatibles entre sí.

2.2.2.- Variables y constantes.

Declarar correctamente variables y constantes en PL/SQL evita errores y hace el código más claro y predecible. También es importante controlar las conversiones de tipos y la precedencia de operadores para evitar resultados inesperados durante la ejecución.

Para declarar una **variable** en PL/SQL se indica su nombre, el tipo de dato y, opcionalmente, un valor inicial mediante `:=` o `DEFAULT`.

```
id SMALLINT;
hoy DATE := SYSDATE;
num INTEGER DEFAULT 4;
```

DC Studio. *Variables y constates* (CC BY-NC-SA)

Para declarar una **constante** se utiliza la palabra `CONSTANT`. Su valor no puede modificarse después:

```
pi CONSTANT NUMBER := 3.1415;
```

También puede utilizarse `NOT NULL` para impedir que una variable tenga valor nulo. En ese caso debe inicializarse en la propia declaración:

```
id SMALLINT NOT NULL := 9999;
```

Esta declaración sería incorrecta porque no asigna ningún valor:

```
id SMALLINT NOT NULL;
```

El ámbito de una variable depende del **bloque PL/SQL** donde haya sido declarada, aspecto que se estudiará posteriormente.

Conversión de tipos

PL/SQL puede realizar algunas conversiones implícitas entre tipos compatibles, pero es más seguro utilizar **conversiones explícitas**.

Las funciones más habituales son:

- **TO_CHAR**: convierte a texto.
- **TO_DATE**: convierte a fecha.
- **TO_NUMBER**: convierte a número.

Por ejemplo:

```
fecha := TO_DATE('28/08/2026', 'DD/MM/YYYY');
```

Utilizar conversiones explícitas evita depender de configuraciones regionales o conversiones automáticas que podrían producir resultados inesperados.

Precedencia de operadores

PL/SQL aplica un orden de prioridad al evaluar expresiones. De mayor a menor, de forma simplificada:

1. **, **NOT**
2. Signos + y -
3. *, /
4. +, -, ||
5. Comparaciones: =, <>, <, >, <=, >=, **IS NULL**, **LIKE**, **BETWEEN**, **IN**
6. **AND**
7. **OR**

Cuando una expresión puede resultar ambigua, es recomendable utilizar **paréntesis** para indicar claramente el orden de evaluación.

```
resultado := (a + b) * c;
```

Así se mejora tanto la **legibilidad** como la seguridad del código.

Autoevaluación

Un desarrollador escribe este bloque PL/SQL:

```
DECLARE
    limite CONSTANT NUMBER := 100;
    contador NUMBER NOT NULL := 10;
    fecha DATE;
    resultado NUMBER;
BEGIN
    fecha := TO_DATE('28/08/2026', 'DD/MM/YYYY');
    resultado := 5 + 3 * 2;

    contador := contador + 1;

```

```
    limite := 200;
END;
```

Al revisar el código, ¿qué afirmación es la más correcta?

- ○ El bloque es correcto porque una constante puede cambiar de valor dentro de BEGIN si se declaró previamente.
- ○ El error está en contador NUMBER NOT NULL := 10;, porque una variable NOT NULL no puede inicializarse al declararla.
- ○ La conversión de fecha es incorrecta porque TO_DATE solo admite números y no cadenas de texto.
- ○ La instrucción limite := 200; es incorrecta porque limite es una constante; además, la expresión 5 + 3 * 2 se evaluará respetando la precedencia, realizando primero la multiplicación.

Incorrecta. Una variable declarada con CONSTANT no puede modificar su valor después de la declaración.

Incorrecta. Una variable NOT NULL debe inicializarse, por lo que contador NUMBER NOT NULL := 10; es una declaración correcta.

Incorrecta. TO_DATE se utiliza precisamente para convertir una cadena de texto en un valor de tipo fecha utilizando un formato determinado.

Correcta. La constante limite no puede reasignarse. Además, por la precedencia de operadores, primero se calcula 3 * 2 y después se suma 5, obteniendo 11.

Solución

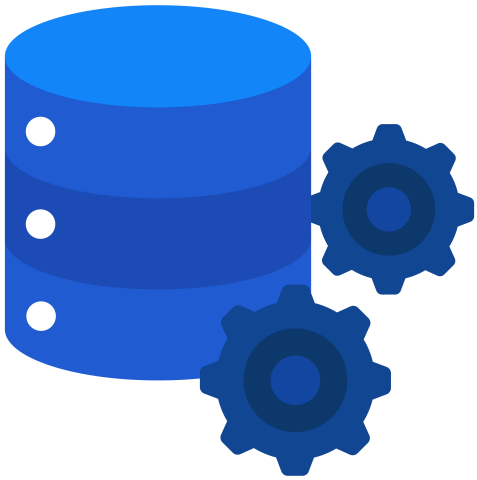
1. Incorrecto
2. Incorrecto
3. Incorrecto
4. Opción correcta

2.3.- El bloque PL/SQL.

El bloque es la unidad básica de programación en PL/SQL. Comprender sus partes y el ámbito de las variables permite organizar el código, controlar errores y evitar confusiones cuando existen bloques anidados.

Un bloque PL/SQL se organiza normalmente en **tres zonas**:

- **Declaraciones:** definición de variables, constantes, cursores y excepciones.
- **Proceso:** contiene las **sentencias ejecutables** del programa.
- **Excepciones:** permite tratar los **errores producidos durante la ejecución**.



juicy_fish. *Bloqueo PL/SQL* (CC BY-NC-SA)

La estructura general es:

```
[DECLARE
    declaraciones]
BEGIN
    sentencias ejecutables
[EXCEPTION
    manejadores de excepciones]
END;
```

Las secciones **DECLARE** y **EXCEPTION** son **opcionales**, mientras que **BEGIN** y **END** forman la parte imprescindible del bloque ejecutable.

Los bloques PL/SQL pueden **anidarse**, es decir, puede existir un bloque dentro de otro. En estos casos, las variables tienen un **ámbito** determinado por el bloque donde se declaran.

Por ejemplo:

```
DECLARE
    aux NUMBER := 10;
BEGIN
    DECLARE
        aux NUMBER := 5;
    BEGIN
        IF aux = 10 THEN
            NULL;
        END IF;
    END;
END;
```

Dentro del bloque interno, la variable **aux** con valor **5** oculta temporalmente a la variable **aux** del bloque exterior. Por tanto, la condición **aux = 10** se evalúa como **FALSE**.

Esta organización permite crear programas más estructurados, reutilizar nombres en distintos ámbitos y controlar mejor la **visibilidad de las variables**.

Para saber más

En el siguiente enlace podrás ampliar información sobre el ámbito y la visibilidad de las variables en PL/SQL.

Ámbito y visibilidad en PL/SQL.

Autoevaluación

¿Es verdadero o falso?

Un programador necesita ejecutar únicamente una instrucción sencilla y escribe:

```
BEGIN
NULL;
END;
```

Otro compañero afirma que el bloque es incorrecto porque todo bloque PL/SQL debe incluir obligatoriamente las secciones DECLARE y EXCEPTION.

○ Verdadero ○ Falso

Falso

Las secciones DECLARE y EXCEPTION son opcionales. La parte esencial del bloque ejecutable está formada por BEGIN y END.

Se ejecuta el siguiente código:

```
 DECLARE
valor NUMBER := 20;
BEGIN
DECLARE
valor NUMBER := 8;
BEGIN
IF valor = 20 THEN
NULL;
END IF;
END;
```

END;

Un alumno afirma que la condición se evaluará como verdadera porque valor fue declarado primero con 20 en el bloque exterior.

○ Verdadero ○ Falso

Falso

Dentro del bloque interno, la variable local valor := 8 oculta temporalmente a la variable exterior. Por tanto, la comparación se realiza con 8 y valor = 20 resulta FALSE.

Un bloque realiza varias operaciones que pueden producir un error durante la ejecución. El desarrollador coloca el código de tratamiento dentro de la sección EXCEPTION y afirma que esa zona se utiliza precisamente para gestionar esas situaciones.

○ Verdadero ○ Falso

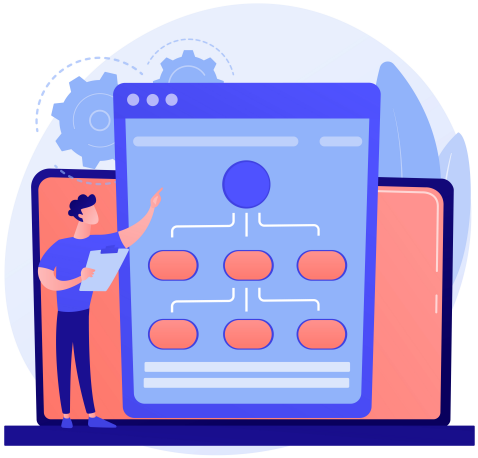
Verdadero

La sección EXCEPTION está destinada al manejo de errores en tiempo de ejecución. Permite definir qué debe hacer el programa cuando se produce una excepción, separando ese tratamiento de las sentencias ejecutables normales.

2.4.- Estructuras de control (I).

Las estructuras de control permiten que un programa PL/SQL tome decisiones y ejecute acciones diferentes según cada situación. Dominar IF y CASE es fundamental para crear programas que respondan de forma lógica a los datos y no se limiten a ejecutar siempre las mismas instrucciones.

En PL/SQL, el **flujo de control** permite decidir qué instrucciones deben ejecutarse según una condición. Dentro del control condicional destacan principalmente IF y CASE.



vectorjuice. *Estructuras de control* (CC BY-NC-SA)

Sentencia IF

IF-THEN

Se utiliza cuando una acción debe ejecutarse solo si una condición es verdadera.

```
IF a > b THEN
    DBMS_OUTPUT.PUT_LINE(a || ' es may
END IF;
```

IF-THEN-ELSE

Permite elegir entre dos alternativas:

```
IF a > b THEN
    DBMS_OUTPUT.PUT_LINE(a || ' es mayor');
ELSE
    DBMS_OUTPUT.PUT_LINE(b || ' es mayor o igual');
END IF;
```

IF-THEN-ELSIF

Se utiliza cuando existen **varias condiciones posibles**. Se evalúan en orden y se ejecuta la primera que resulte verdadera.

```
IF operacion = 'SUMA' THEN
    resultado := arg1 + arg2;
ELSIF operacion = 'RESTA' THEN
    resultado := arg1 - arg2;
ELSIF operacion = 'PRODUCTO' THEN
    resultado := arg1 * arg2;
ELSIF operacion = 'DIVISION' AND arg2 <> 0 THEN
    resultado := arg1 / arg2;
ELSE
    RAISE operacion_no_permitida;
END IF;
```

Sentencia CASE

CASE resulta especialmente útil cuando existen varias alternativas y permite escribir código más claro que una sucesión extensa de IF.

Puede utilizarse con un **selector**:

```
CASE operacion
    WHEN 'SUMA' THEN
        resultado := arg1 + arg2;
    WHEN 'RESTA' THEN
        resultado := arg1 - arg2;
    ELSE
        DBMS_OUTPUT.PUT_LINE('Operación no válida');
END CASE;
```

O mediante **condiciones de búsqueda**:

```
CASE
    WHEN nota IN (1,2) THEN
        DBMS_OUTPUT.PUT_LINE('Muy deficiente');
    WHEN nota IN (3,4) THEN
        DBMS_OUTPUT.PUT_LINE('Insuficiente');
    WHEN nota = 5 THEN
        DBMS_OUTPUT.PUT_LINE('Suficiente');
    WHEN nota = 6 THEN
        DBMS_OUTPUT.PUT_LINE('Bien');
    WHEN nota IN (7,8) THEN
        DBMS_OUTPUT.PUT_LINE('Notable');
    WHEN nota IN (9,10) THEN
        DBMS_OUTPUT.PUT_LINE('Sobresaliente');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Nota no válida');
END CASE;
```

Las condiciones de CASE se evalúan **en el orden en que aparecen** y, cuando una se cumple, no se siguen evaluando las posteriores.

Entrada mediante variables de sustitución

En herramientas como **SQL*Plus o SQLcl**, puede utilizarse `&` para solicitar un valor antes de ejecutar el bloque:

```
DECLARE
    cadena VARCHAR2(25) := '&cad';
BEGIN
    DBMS_OUTPUT.PUT_LINE(cadena);
END;
/
```

Las variables de sustitución no forman parte del lenguaje PL/SQL en sí, sino del **cliente que prepara el código antes de enviarlo a Oracle**.

Autoevaluación

Una aplicación PL/SQL recibe una nota mediante una variable de sustitución y debe mostrar una clasificación. El programador escribe varias condiciones en este orden:

```
CASE
    WHEN nota >= 5 THEN
        DBMS_OUTPUT.PUT_LINE('Aprobado');
    WHEN nota >= 7 THEN
        DBMS_OUTPUT.PUT_LINE('Notable');
    WHEN nota >= 9 THEN
        DBMS_OUTPUT.PUT_LINE('Sobresaliente');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Suspenso');
END CASE;
```

Si el usuario introduce el valor 9, ¿qué ocurrirá y cuál sería la explicación correcta?

- ○ Mostrará Sobresaliente, porque CASE busca siempre la condición más específica antes de ejecutar una acción.
- ○ Mostrará Aprobado, porque las condiciones se evalúan en orden y se ejecuta la primera que resulte verdadera.
- ○ Mostrará Notable, porque PL/SQL evalúa todas las condiciones y selecciona la intermedia cuando varias son verdaderas.
- ○ Se producirá un error, porque una sentencia CASE no puede utilizar comparaciones como >=.

Incorrecta. CASE no busca automáticamente la condición más específica. Evalúa las condiciones en el orden en que aparecen.

Correcta. Para nota = 9, la primera condición nota >= 5 ya es verdadera, por lo que se muestra Aprobado y no se evalúan las siguientes. Para evitar este problema convendría ordenar primero las condiciones más restrictivas.

Incorrecta. Cuando una condición de CASE se cumple, se dejan de evaluar las posteriores.

Incorrecta. Un CASE de búsqueda puede utilizar expresiones como >=, IN, BETWEEN u otras condiciones válidas.

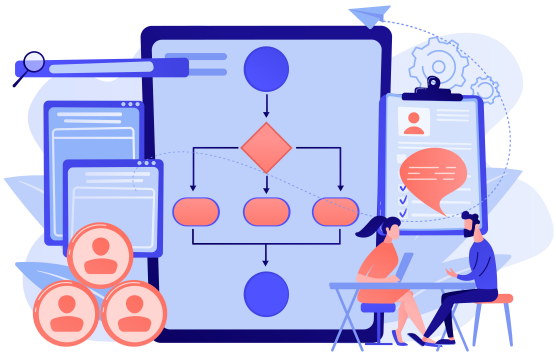
Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

2.4.1.- Estructuras de control (II). Bucles

Las estructuras iterativas permiten repetir instrucciones de forma controlada en PL/SQL. Saber elegir entre LOOP, WHILE y FOR ayuda a escribir programas más claros, evitar bucles infinitos y resolver tareas repetitivas de forma sencilla.

Después de estudiar las estructuras condicionales, el siguiente paso es conocer las estructuras de **control iterativo**, que permiten ejecutar una secuencia de instrucciones varias veces.



vectorjuice. *Bucles* (CC BY-NC-SA)

LOOP

Es la forma más básica de bucle. Si no se incluye una condición de salida, se ejecutará indefinidamente.

```
LOOP
    secuencia_de_sentencias;
END LOOP;
```

Para evitar un bucle infinito puede utilizarse EXIT:

```
LOOP
    ...
    IF encontrado THEN
        EXIT;
    END IF;
END LOOP;
```

EXIT finaliza únicamente el **bucle actual** y continúa con la siguiente instrucción después de END LOOP.

También puede utilizarse EXIT WHEN, que resulta más compacto:

```
LOOP
    ...
    EXIT WHEN encontrado;
END LOOP;
```

Por ejemplo:

```
DECLARE
    a INTEGER := 1;
BEGIN
    LOOP
        DBMS_OUTPUT.PUT_LINE(a);
        EXIT WHEN a >= 10;
        a := a + 1;
    END LOOP;
END;
/
```

WHILE LOOP

Ejecuta el bloque mientras una condición sea **TRUE**. La condición se comprueba antes de cada iteración.

```
WHILE condicion LOOP
    secuencia_de_sentencias;
END LOOP;
```

Ejemplo:

```
DECLARE
    a INTEGER := 1;
BEGIN
    WHILE a <= 10 LOOP
        DBMS_OUTPUT.PUT_LINE(a);
        a := a + 1;
    END LOOP;
END;
/
```

FOR LOOP

Se utiliza cuando conocemos de antemano el **rango de iteraciones**. PL/SQL gestiona automáticamente el contador.

```
FOR contador IN limite_inferior..limite_superior LOOP
    secuencia_de_sentencias;
END LOOP;
```

Ejemplo ascendente:

```
BEGIN
    FOR a IN 1..10 LOOP
        DBMS_OUTPUT.PUT_LINE(a);
    END LOOP;
END;
/
```

También puede recorrerse el rango en sentido inverso mediante **REVERSE**:

```
BEGIN
    FOR a IN REVERSE 1..10 LOOP
        DBMS_OUTPUT.PUT_LINE(a);
    END LOOP;
END;
/
```

En resumen, **LOOP** ofrece mayor libertad, **WHILE** repite mientras se cumpla una condición y **FOR** resulta especialmente cómodo cuando se conoce el número de iteraciones.

Autoevaluación

¿Es verdadero o falso?

Un programador quiere mostrar los números del 1 al 10 y escribe:

DECLARE
a INTEGER := 1;
BEGIN
LOOP
DBMS_OUTPUT.PUT_LINE(a);
a := a + 1;
END LOOP;
END;
/

Afirma que el bucle finalizará automáticamente cuando a llegue a 10.

○ Verdadero ○ Falso

Falso

Un **LOOP** básico no finaliza por sí solo. Si no se incluye una instrucción como **EXIT** o **EXIT WHEN**, el bucle continuará ejecutándose indefinidamente.

Un LOOP básico no finaliza por sí solo. Si no se incluye una instrucción como EXIT o EXIT WHEN, el bucle continuará ejecutándose indefinidamente.

WHILE saldo > 0 LOOP

DBMS_OUTPUT.PUT_LINE(saldo);
END LOOP;
El programador afirma que el contenido del bucle se ejecutará al menos una vez, aunque la condición sea falsa desde el principio.

○ Verdadero ○ Falso

Falso

En un WHILE LOOP la condición se comprueba antes de cada iteración. Si inicialmente es falsa, el cuerpo del bucle no se ejecuta ninguna vez.

Se necesita mostrar los valores del 10 al 1. El desarrollador escribe:

```
BEGIN
FOR i IN REVERSE 1..10 LOOP
DBMS_OUTPUT.PUT_LINE(i);
END LOOP;
END;
/
```

y afirma que no necesita modificar manualmente el valor de i dentro del bucle.

○ Verdadero ○ Falso

Verdadero

En un FOR LOOP, PL/SQL gestiona automáticamente el contador. Con REVERSE, el rango 1..10 se recorre de forma descendente, desde 10 hasta 1.

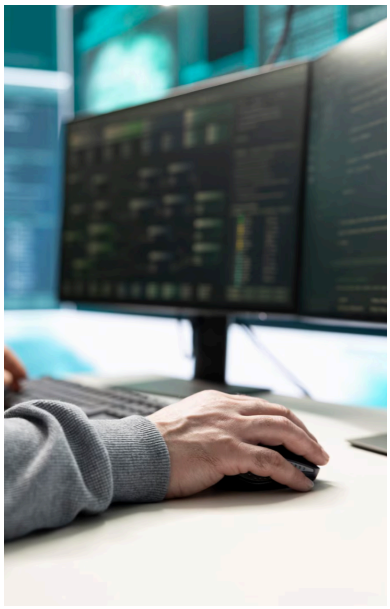
2.5.- Manejo de errores (I). Excepciones.

El tratamiento de excepciones permite que un programa PL/SQL responda de forma controlada ante errores inesperados. Saber capturar excepciones predefinidas y crear excepciones propias ayuda a evitar fallos bruscos y facilita localizar y gestionar problemas durante la ejecución.

En PL/SQL, cualquier situación de error se denomina **excepción**. Cuando se produce una excepción, la ejecución normal del bloque se interrumpe y el control pasa a la sección EXCEPTION.

Los errores se gestionan mediante **manejadores de excepciones**:

```
EXCEPTION
    WHEN nombre_excepcion THEN
        sentencias;
    WHEN OTHERS THEN
        sentencias;
END;
```



DC Studio. *Manejo de errores* (CC BY-NC-SA)

Por ejemplo:

```
DECLARE
    supervisor agentes%ROWTYPE;
BEGIN
    SELECT * INTO supervisor
    FROM agentes
    WHERE categoria = 2
      AND oficina = 3;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No se encontraron datos');

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Se produjo un error inesperado');
END;
/
```

NO_DATA_FOUND es una excepción predefinida de Oracle que puede producirse, por ejemplo, cuando un SELECT INTO no devuelve ninguna fila.

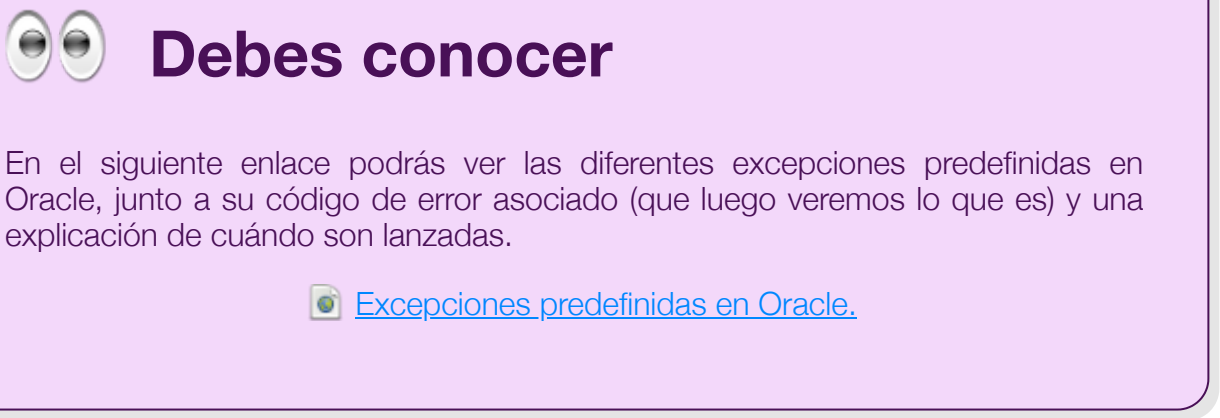
La cláusula WHEN OTHERS permite capturar cualquier excepción que no haya sido tratada anteriormente. Normalmente se coloca **al final** de los manejadores.

PL/SQL también permite crear **excepciones definidas por el usuario**. Se declaran en la sección **DECLARE** y se lanzan explícitamente mediante **RAISE**:

```
DECLARE
    categoria_erronea EXCEPTION;
BEGIN
    IF categoria < 0 OR categoria > 3 THEN
        RAISE categoria_erronea;
    END IF;

EXCEPTION
    WHEN categoria_erronea THEN
        DBMS_OUTPUT.PUT_LINE('Categoría no válida');
END;
/
```

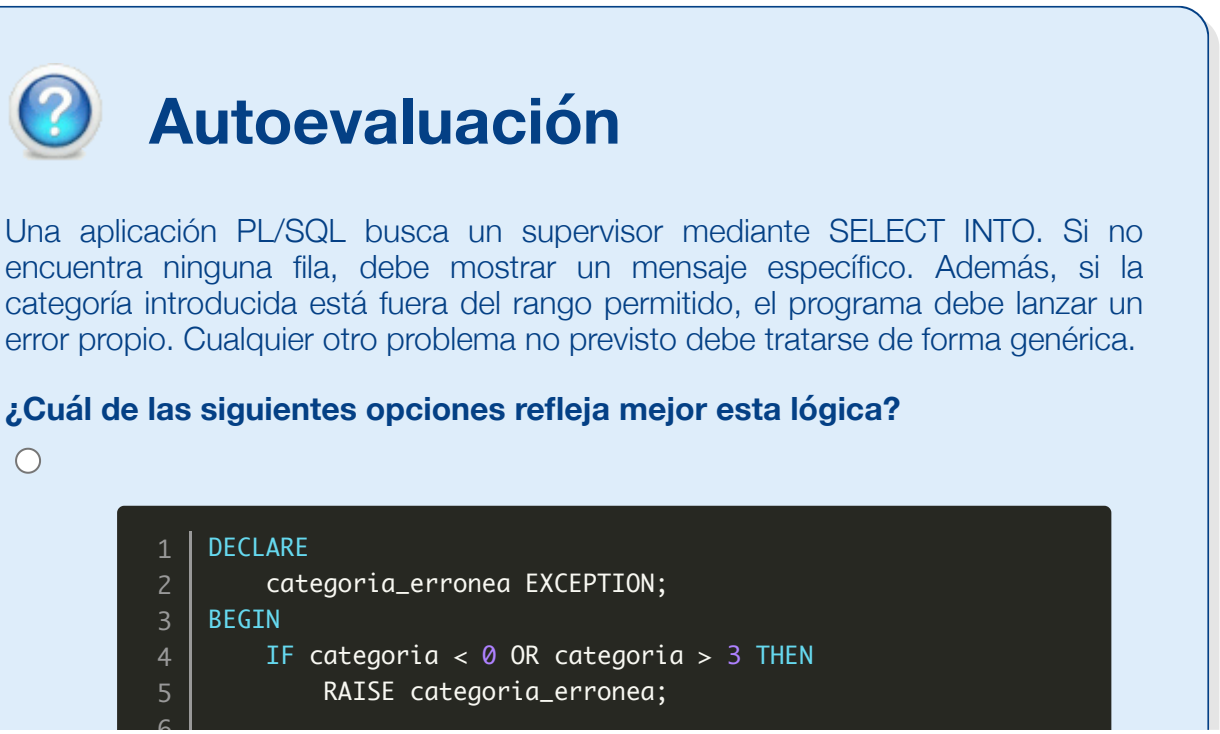
Por tanto, las excepciones **predefinidas** están asociadas a errores detectados por Oracle, mientras que las excepciones **definidas por el usuario** permiten controlar situaciones particulares de la lógica de la aplicación.



Debes conocer

En el siguiente enlace podrás ver las diferentes excepciones predefinidas en Oracle, junto a su código de error asociado (que luego veremos lo que es) y una explicación de cuándo son lanzadas.

Excepciones predefinidas en Oracle.



Autoevaluación

Una aplicación PL/SQL busca un supervisor mediante SELECT INTO. Si no encuentra ninguna fila, debe mostrar un mensaje específico. Además, si la categoría introducida está fuera del rango permitido, el programa debe lanzar un error propio. Cualquier otro problema no previsto debe tratarse de forma genérica.

¿Cuál de las siguientes opciones refleja mejor esta lógica?

○

```
DECLARE
    categoria_erronea EXCEPTION;
BEGIN
    IF categoria < 0 OR categoria > 3 THEN
        RAISE categoria_erronea;
```

```
    END IF;

    SELECT * INTO supervisor
    FROM agentes
    WHERE categoria = 2;

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error general');

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No hay supervisor');
END;
```

○

```
DECLARE
    categoria_erronea EXCEPTION;
BEGIN
    IF categoria < 0 OR categoria > 3 THEN
        RAISE categoria_erronea;
    END IF;

    SELECT * INTO supervisor
    FROM agentes
    WHERE categoria = 2;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No hay supervisor');

    WHEN categoria_erronea THEN
        DBMS_OUTPUT.PUT_LINE('Categoría no válida');

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error general');
END;
```

○

```
BEGIN
    IF categoria < 0 OR categoria > 3 THEN
        NO_DATA_FOUND;
    END IF;

EXCEPTION
    WHEN categoria_erronea THEN
        DBMS_OUTPUT.PUT_LINE('Categoría no válida');
END;
```

○

```
DECLARE
    categoria_erronea EXCEPTION;
BEGIN
    SELECT * INTO supervisor
    FROM agentes
    WHERE categoria = 2;

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No hay supervisor');

    RAISE categoria_erronea;
END;
```

Incorrecta. WHEN OTHERS debe colocarse al final, ya que captura cualquier excepción no tratada anteriormente.

Correcta. Se declara una excepción propia, se lanza mediante RAISE, se captura NO_DATA_FOUND de forma específica y se deja WHEN OTHERS como manejador genérico final.

Incorrecta. Una excepción definida por el usuario debe declararse previamente y lanzarse mediante RAISE. Además, NO_DATA_FOUND no se utiliza como una instrucción normal.

Incorrecta. Los manejadores WHEN deben estar dentro de la sección EXCEPTION, no directamente dentro de BEGIN.

Solución

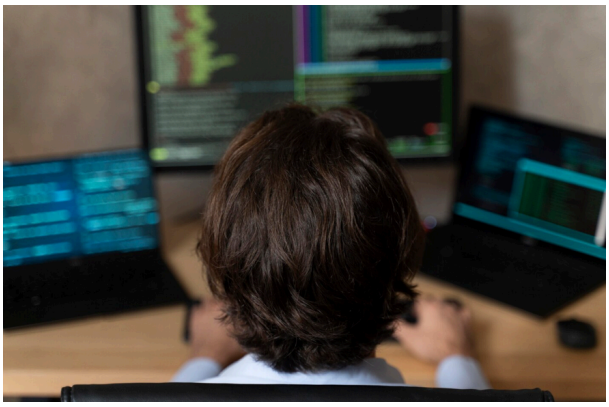
1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

2.5.1.- Manejo de errores (II).

El manejo de excepciones en PL/SQL no solo consiste en capturar errores, sino también en entender su ámbito y cómo afecta al flujo del programa. Saber dónde declarar una excepción y cuándo utilizar bloques anidados permite controlar mejor los fallos y decidir si el programa debe detenerse o continuar con otras operaciones.

El **alcance de una excepción** sigue reglas similares a las de las variables. Si una excepción se declara de nuevo dentro de un bloque interno, la definición local **oculta** a la del bloque exterior.

Por eso no conviene redefinir excepciones predefinidas como **NO_DATA_FOUND**, ya que Oracle ya las proporciona globalmente:



magnific. *Manejo de errores* (CC BY-NC-SA)

```
DECLARE
    no_data_found EXCEPTION;  -- No re
BEGIN
    ...
END;
```

Si se necesita acceder a una excepción declarada en un bloque exterior con nombre, puede hacerse mediante la notación:

```
nombre_bloque.nombre_excepcion
```

Cuando se produce una excepción y el control pasa a la sección **EXCEPTION**, **no se continúa automáticamente desde la sentencia que falló**. Tras ejecutar el manejador, finaliza ese bloque y el control pasa al bloque exterior correspondiente.

Por ejemplo, si falla este primer **INSERT**:

```
INSERT INTO familias VALUES (...);
INSERT INTO agentes VALUES (...);
```

y la excepción se captura en el **EXCEPTION** del bloque principal, el segundo **INSERT** **no se ejecutará automáticamente**.

Si queremos controlar el error de una operación concreta y después continuar con el resto, podemos utilizar un **bloque anidado**:

```
BEGIN
    BEGIN
        INSERT INTO familias VALUES (...);
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            SELECT identificador
            INTO id_fam
            FROM familias
            WHERE nombre = nom_fam;
    END;

    INSERT INTO agentes VALUES (...);
END;
```

Así, el error del primer **INSERT** se trata dentro de su propio bloque y, una vez gestionado, el programa puede **continuar con las siguientes instrucciones**.

Ejercicio resuelto

Supongamos que queremos reintentar una transacción hasta que no nos dé ningún error. Para ello deberemos encapsular la transacción en un bloque y capturar en éste las posibles excepciones. El bloque lo metemos en un bucle y así se reintentará la transacción hasta que sea posible llevarla a cabo.

Mostrar retroalimentación

```
DECLARE
     id_fam NUMBER;
     nombre VARCHAR2(40);
     oficina NUMBER;
BEGIN
     ...
     LOOP
          BEGIN
               SAVEPOINT inicio;
               INSERT INTO familias VALUES
(id_fam, nombre, NULL, oficina);
               ...
               COMMIT;
               EXIT;
          EXCEPTION
               WHEN DUP_VAL_ON_INDEX THEN
                    ROLLBACK TO inicio;
```

```
                    id_fam := id_fam + 1;
            END;
        END LOOP;
        ...
END;
```

Autoevaluación

¿Es verdadero o falso?

Un programador declara una excepción llamada NO_DATA_FOUND dentro de un bloque y después ejecuta un SELECT INTO que no devuelve ninguna fila. Afirma que el manejador local WHEN NO_DATA_FOUND capturará automáticamente la excepción predefinida lanzada por Oracle.

○ Verdadero ○ Falso

Falso

Al declarar otra excepción con el mismo nombre, la excepción local oculta a la predefinida global. Por eso no es recomendable redefinir excepciones como NO_DATA_FOUND, ya que puede provocar confusión y hacer que el manejador no capture la excepción esperada.

En un bloque principal se ejecutan dos INSERT consecutivos. El primero genera DUP_VAL_ON_INDEX y la excepción se captura en la sección EXCEPTION de ese mismo bloque. El desarrollador afirma que, una vez tratado el error, PL/SQL continuará automáticamente ejecutando el segundo INSERT.

○ Verdadero ○ Falso

Falso

Cuando una excepción transfiere el control a la sección EXCEPTION, el programa no vuelve a la instrucción siguiente a la que falló. Tras ejecutar el manejador, finaliza ese bloque, por lo que el segundo INSERT no se ejecutará automáticamente.

Una aplicación debe insertar primero una familia y después un agente. Si la familia ya existe, se quiere recuperar su identificador y continuar igualmente con la inserción del agente. El programador coloca el primer INSERT dentro de un bloque anidado que captura DUP_VAL_ON_INDEX y recupera el identificador existente.

○ Verdadero ○ Falso

Verdadero

Al aislar la operación problemática dentro de un bloque interno, la excepción puede tratarse localmente. Cuando ese bloque termina correctamente tras manejar el error, el bloque exterior puede continuar con las instrucciones siguientes, como el INSERT del agente.

2.5.2.- Manejo de errores (III).

Cuando una excepción puede producirse en varias sentencias, es importante saber identificar dónde ocurrió y entender cómo se propaga entre bloques. Estas técnicas ayudan a localizar fallos con más precisión y a decidir si el error debe tratarse localmente, propagarse o relanzarse.

macrovector. *Manejo de errores* (CC BY-NC-SA)

Si varias sentencias similares pueden provocar la misma excepción, existen dos enfoques habituales:

- **Encapsular cada sentencia en un bloque propio**, de forma que cada error pueda tratarse por separado y el programa pueda continuar después.
- Utilizar una **variable localizadora** para identificar qué sentencia produjo el error. Este método ayuda a saber dónde ocurrió, pero al entrar en la sección EXCEPTION del bloque ya no se continúa automáticamente por la siguiente sentencia.

Por ejemplo:

```
DECLARE
    sentencia NUMBER := 0;
BEGIN
    SELECT ... INTO ... FROM agentes;
    sentencia := 1;

    SELECT ... INTO ... FROM familias;
    sentencia := 2;

    SELECT ... INTO ... FROM oficinas;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
```

```
        IF sentencia = 0 THEN
            RAISE agente_no_encontrado;
        ELSIF sentencia = 1 THEN
            RAISE familia_no_encontrada;
        ELSIF sentencia = 2 THEN
            RAISE oficina_no_encontrada;
        END IF;
END;
/
```

La variable **sentencia** permite saber **en qué punto se produjo** **NO_DATA_FOUND** y lanzar una excepción más específica.

Propagación de excepciones

Si una excepción dispone de un manejador adecuado en el bloque donde se produce, se trata allí y después finaliza ese bloque.

Si no existe un manejador adecuado, la excepción **se propaga al bloque exterior**. Este proceso continúa hasta que algún bloque la capture.

Si ningún bloque la maneja, la excepción llega al **entorno desde el que se ejecutó PL/SQL**, que mostrará el error correspondiente.

También es posible **relanzar una excepción** desde un manejador mediante **RAISE**, por ejemplo cuando se quiere registrar el error localmente pero permitir que un bloque superior continúe gestionándolo.

En resumen, una excepción puede **tratarse, propagarse o relanzarse**, dependiendo de cómo esté estructurado el programa.

Para saber más

En la siguiente presentación puedes ver cómo se propagan diferentes excepciones entre diferentes bloques.

Propagación de excepciones en PL/SQL.

Autoevaluación

En un bloque PL/SQL se ejecutan tres consultas SELECT INTO consecutivas. Cualquiera de ellas puede lanzar NO_DATA_FOUND. El programador utiliza una variable llamada sentencia para saber cuál falló. Además, quiere que, si la excepción no se resuelve en el bloque actual, pueda ser tratada por un bloque exterior.

¿Cuál de las siguientes afirmaciones describe mejor el comportamiento correcto?

- ○ Al entrar en EXCEPTION, PL/SQL vuelve después automáticamente a la sentencia siguiente a la que produjo el error, por lo que la variable sentencia sirve para continuar desde ese punto.

- ○ La variable sentencia puede ayudar a identificar qué consulta provocó NO_DATA_FOUND, pero una vez transferido el control a EXCEPTION no se continúa automáticamente por la siguiente sentencia; además, si la excepción no se maneja, puede propagarse al bloque exterior.

- ○ Si el manejador ejecuta RAISE, la excepción desaparece porque ya ha sido capturada en el bloque actual.

- ○ Una excepción no puede atravesar más de un nivel de anidamiento; si no se captura en el bloque inmediatamente superior, siempre se pierde.

Incorrecta. Cuando se produce una excepción, la ejecución normal del bloque se interrumpe. Tras entrar en EXCEPTION, no se reanuda automáticamente en la sentencia posterior a la que falló.

Correcta. Una variable localizadora permite identificar dónde se produjo el error, pero no permite continuar desde ese punto. Si la excepción no se trata, puede propagarse a bloques exteriores hasta encontrar un manejador adecuado.

Incorrecta. RAISE dentro de un manejador permite relanzar la excepción, haciendo que pueda seguir propagándose hacia un bloque superior.

Incorrecta. Una excepción puede propagarse por varios niveles de bloques anidados hasta ser capturada. Si ningún bloque la maneja, termina llegando al entorno de ejecución.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

2.5.3.- Manejo de errores (IV).

PL/SQL permite personalizar el tratamiento de errores y devolver información útil a las aplicaciones. Conocer `RAISE_APPLICATION_ERROR`, `PRAGMA EXCEPTION_INIT`, `SQLCODE` y `SQLERRM` ayuda a identificar fallos con precisión, registrar incidencias y mostrar mensajes más claros.

Oracle permite generar **errores personalizados** mediante el procedimiento:

```
RAISE_APPLICATION_ERROR(error_number, messa
```

awesomecontent. *Manejo de errores* (CC BY-NC-SA)

El parámetro `error_number` debe estar entre **-20000 y -20999**, y `message` contiene el texto que recibirá la aplicación.

El tercer parámetro es opcional:

- `TRUE`: añade el error a la pila existente.
- `FALSE`: sustituye la pila por el nuevo error.

Por ejemplo:

```
1 | RAISE_APPLICATION_ERROR(-20001, 'Categoría no válida');
```

Este procedimiento se utiliza dentro de **subprogramas almacenados**, como procedimientos, funciones, métodos o triggers.

Asociar errores Oracle a excepciones

No todos los errores Oracle disponen de una excepción predefinida. Para asociar un código concreto a una excepción propia se utiliza:

```
1 | PRAGMA EXCEPTION_INIT(nombre_excepcion, codigo_error);
```

Por ejemplo:

```
1 | DECLARE
2 |     no_null EXCEPTION;
3 |     PRAGMA EXCEPTION_INIT(no_null, -1400);
4 |
```

```
BEGIN
    ...
EXCEPTION
    WHEN no_null THEN
        DBMS_OUTPUT.PUT_LINE('No se admite un valor NULL');
END;
```

Así, el error Oracle **ORA-01400** puede tratarse mediante la excepción **no_null**.

SQLCODE y SQLERRM

PL/SQL proporciona dos funciones útiles para conocer información sobre una excepción:

- **SQLCODE** devuelve el código del error.
- **SQLERRM** devuelve el mensaje asociado.

Por ejemplo:

```
DECLARE
    cod NUMBER;
    msg VARCHAR2(1000);
BEGIN
    ...
EXCEPTION
    WHEN OTHERS THEN
        cod := SQLCODE;
        msg := SQLERRM;

        INSERT INTO errores(codigo, mensaje)
        VALUES (cod, msg);
END;
```

Cuando no existe error, **SQLCODE** **devuelve 0**. La mayoría de errores devuelven códigos negativos, aunque **NO_DATA_FOUND** tiene asociado tradicionalmente el valor **+100**.

Estas herramientas permiten construir sistemas de **registro y diagnóstico de errores** más claros y útiles para aplicaciones reales.

Autoevaluación

¿Es verdadero o falso?

Un procedimiento almacenado debe impedir que se registre una categoría no válida y devolver a la aplicación el mensaje “Categoría no válida” con un código propio. El programador utiliza:

RAISE_APPLICATION_ERROR(-20001, 'Categoría no válida'); y afirma que el código elegido es correcto para un error definido por la aplicación.

○ Verdadero ○ Falso

Verdadero

RAISE_APPLICATION_ERROR utiliza códigos comprendidos entre -20000 y -20999. El valor -20001 pertenece a ese rango y permite devolver un mensaje específico a la aplicación.

Una inserción puede provocar ORA-01400 al intentar introducir NULL en una columna obligatoria. El desarrollador declara:

no_null EXCEPTION;PRAGMA EXCEPTION_INIT(no_null, -1400); y después utiliza WHEN no_null THEN para tratar ese error de forma específica.

○ Verdadero ○ Falso

Verdadero

PRAGMA EXCEPTION_INIT permite asociar una excepción definida por el programador con un código de error Oracle concreto. En este caso, -1400 corresponde a ORA-01400.

Un bloque captura cualquier error mediante WHEN OTHERS y guarda SQLCODE y SQLERRM en una tabla. El responsable afirma que SQLCODE contiene el texto descriptivo del error y SQLERRM devuelve únicamente su número.

○ Verdadero ○ Falso

Falso

Es justo al contrario: SQLCODE devuelve el código numérico del error y SQLERRM devuelve su mensaje descriptivo. Ambos pueden utilizarse conjuntamente para registrar y diagnosticar incidencias.

2.6.- Sentencias SQL en programas PL/SQL.

PL/SQL permite utilizar SQL directamente dentro de un programa para consultar y modificar los datos de la base de datos. Comprender cómo funciona **SELECT INTO**, las sentencias DML y el SQL dinámico es fundamental para integrar la lógica de programación con el acceso a los datos.

En PL/SQL se utilizan sentencias SQL para **consultar y modificar información** almacenada en las tablas.

A diferencia del uso interactivo de SQL, dentro de un programa los datos recuperados deben guardarse en **variables o estructuras PL/SQL** para poder utilizarlos posteriormente.

magnific. *Sentencias SQL* (CC BY-NC-SA)

PL/SQL admite directamente sentencias **DML**, como:

- **SELECT**
- **INSERT**
- **UPDATE**
- **DELETE**

También permite utilizar sentencias de **control de transacciones**, como **COMMIT** y **ROLLBACK**.

Las sentencias **DDL**, como **CREATE TABLE**, **ALTER TABLE** o **DROP TABLE**, y las sentencias **DCL**, como **GRANT** o **REVOKE**, no se ejecutan directamente como SQL estático dentro de un bloque PL/SQL. Para utilizarlas se recurre normalmente a **SQL dinámico**, por ejemplo mediante **EXECUTE IMMEDIATE**.

SELECT INTO

Cuando una consulta debe recuperar una única fila, se utiliza:

```
1 SELECT lista_campos
2 INTO variable1, variable2
3 FROM tabla
4 WHERE condicion;
```

Debe existir correspondencia entre los datos seleccionados y las variables:

- Mismo **número de elementos**.
- Orden compatible.
- **Tipos de datos compatibles**.

Por ejemplo:

```sql
SELECT nombre, salario
INTO v_nombre, v_salario
FROM empleados
WHERE id_empleado = 100;
```

Un **SELECT INTO** debe devolver **exactamente una fila**:

- Si no devuelve ninguna, se produce **NO_DATA_FOUND**.
- Si devuelve más de una, se produce **TOO_MANY_ROWS**.

Cuando es necesario procesar varias filas, se utilizan mecanismos como los **cursores**, que se estudiarán posteriormente.

Las instrucciones **INSERT**, **UPDATE** y **DELETE** se utilizan prácticamente con la misma sintaxis que en SQL convencional.

Autoevaluación

Una aplicación PL/SQL debe recuperar el nombre y el salario de un empleado a partir de su identificador. El programador escribe:

```sql
SELECT nombre, salario
INTO v_nombre, v_salario
FROM empleados
WHERE departamento = 10;
```

En ese departamento existen varios empleados. Además, en otra parte del mismo bloque necesita crear una tabla temporal mediante CREATE TABLE.

¿Cuál de las siguientes afirmaciones es la más correcta?

- ○ El SELECT INTO funcionará porque PL/SQL guardará automáticamente la primera fila obtenida, y CREATE TABLE puede ejecutarse directamente como SQL estático.
- ○ El SELECT INTO puede provocar TOO_MANY_ROWS porque devuelve más de una fila, y CREATE TABLE debe ejecutarse mediante SQL dinámico, por ejemplo con EXECUTE IMMEDIATE.
- ○ El SELECT INTO provocará NO_DATA_FOUND porque la consulta devuelve demasiadas filas, y CREATE TABLE debe sustituirse por INSERT.
- ○ El SELECT INTO admite cualquier número de filas si existen suficientes variables, y CREATE TABLE puede incluirse directamente dentro de BEGIN.

Incorrecta. SELECT INTO no selecciona automáticamente la primera fila. Debe devolver exactamente una fila. Además, CREATE TABLE es una sentencia DDL y no se ejecuta como SQL estático dentro del bloque.

Correcta. Si la consulta devuelve varias filas, PL/SQL genera TOO_MANY_ROWS. Para ejecutar una sentencia DDL como CREATE TABLE se utiliza normalmente SQL dinámico, por ejemplo EXECUTE IMMEDIATE.

Incorrecta. NO_DATA_FOUND se produce cuando la consulta no devuelve ninguna fila, no cuando devuelve varias. INSERT tampoco sustituye a una sentencia CREATE TABLE.

Incorrecta. Un SELECT INTO normal debe devolver una sola fila. Si se necesitan varias, deben utilizarse mecanismos como cursores u otras estructuras adecuadas.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

3.- Tipos de datos compuestos.

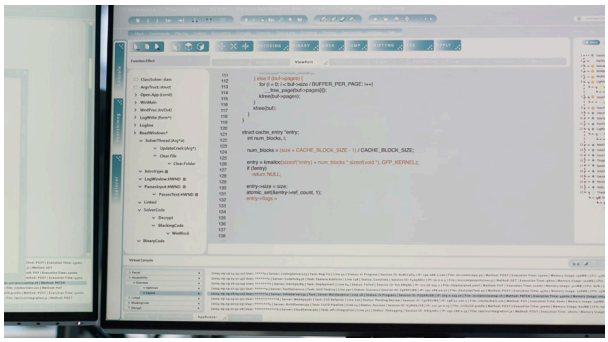
Caso práctico

Ada. (CC BY-NC-SA)

María, gracias a la ayuda de Juan, ha comprendido muy bien el manejo básico de PL/SQL. **Juan** le comenta que en la mayoría de los lenguajes de programación, además de los tipos de datos simples, existen tipos de datos compuestos que le dan mucha más versatibilidad a los mismos y una gran potencia. **Juan** no conoce bien si PL/SQL dispone de estos tipos de datos y si dispone de ellos ¿de cuáles?

María y **Juan** van a hablar con **Ada** para ver si les puede aclarar un poco la situación y dar unas pequeñas pautas para empezar. **Ada** les comenta que claro que PL/SQL cuenta con este tipo de datos, pero que hoy tiene una reunión importantísima y que tiene que terminar de preparársela, por lo que los remite al capítulo sobre tipos de datos compuestos del manual que les pasó. **Juan** y **María** le dicen que no se preocupe y que ya verá como a la vuelta de esa reunión son capaces de saber cuáles son e incluso de dominarlos. **Ada**, para motivarlos, les dice que si es así los invita a un aperitivo a su vuelta. Así que **Juan** y **María** se ponen con el capítulo de tipos de datos compuestos del manual para ver si pueden sorprender a **Ada** a su vuelta.

Los tipos de datos complejos permiten representar información más estructurada y resolver problemas que no se pueden modelar cómodamente con variables simples. Conocerlos facilita trabajar con conjuntos de datos relacionados y construir programas PL/SQL más organizados y potentes.



DC Studio. *Tipos de datos compuestos* (CC BY-NC-SA)

Hasta ahora se han estudiado los **tipos de datos simples** de PL/SQL, suficientes para almacenar valores individuales como números, textos o fechas.

Sin embargo, cuando los programas aumentan de complejidad es necesario utilizar estructuras capaces de **agrupar y organizar varios datos relacionados**.

En este apartado se introducirán los **tipos de datos complejos de PL/SQL**, aprendiendo cómo se definen, cómo se utilizan y en qué situaciones resultan más adecuados.

El objetivo es disponer de estructuras que permitan modelar mejor la información y aprovechar con mayor eficacia las posibilidades de **PL/SQL**.

Citas para pensar

"Todo lo complejo puede dividirse en partes simples".

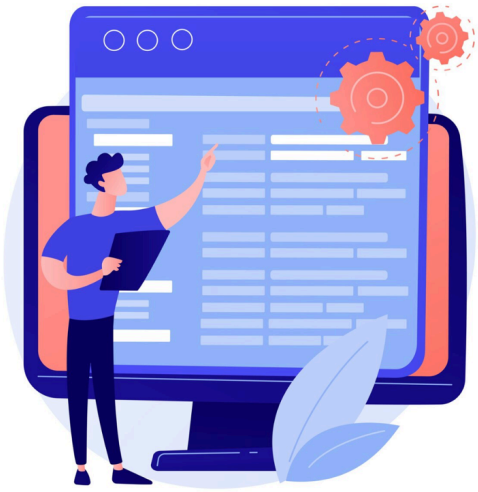
René Descartes.

3.1.- Registros.

Los registros permiten agrupar varios datos relacionados dentro de una sola variable. Son especialmente útiles para representar filas de una tabla o estructuras como direcciones, clientes o productos, haciendo el código PL/SQL más claro y organizado.

Un **registro (RECORD)** es un tipo de dato compuesto formado por varios **campos**, cada uno con su propio nombre y tipo de dato, pero agrupados bajo un único identificador.

Por ejemplo, una dirección puede contener calle, número, código postal, ciudad y país.



vectorjuice. *Registros* (CC BY-NC-SA)

Declaración de un registro

La sintaxis básica es:

```
TYPE nombre_tipo IS RECORD (
    campo1 tipo,
    campo2 tipo,
    ...
);
```

Por ejemplo:

```
TYPE direccion IS RECORD (
    calle          VARCHAR2(50),
    numero         INTEGER,
    piso           INTEGER,
    puerta         VARCHAR2(2),
    codigo_postal  VARCHAR2(5),
    ciudad         VARCHAR2(30),
    provincia      VARCHAR2(30),
    pais           VARCHAR2(30) DEFAULT 'España'
);

mi_direccion direccion;
```

Para acceder a cada campo se utiliza el **operador punto (.)**:

```
mi_direccion.calle := 'Ramírez de Arellano';
mi_direccion.numero := 15;
```

Los registros también pueden basarse directamente en la estructura de una tabla mediante **%ROWTYPE**:

```
mi_familia familias%ROWTYPE;
```

De esta forma, **mi_familia** tendrá los mismos campos y tipos de datos que una fila de la tabla **familias**.

También es posible recuperar directamente una fila mediante **SELECT INTO**:

```
SELECT *
INTO mi_familia
FROM familias
WHERE identificador = 10;
```

La asignación completa entre registros requiere que sean **tipos compatibles**; tener simplemente los mismos campos no garantiza que puedan asignarse directamente.

PL/SQL tampoco permite comparar dos registros completos mediante **=** ni comprobar un registro completo con **IS NULL**; estas operaciones deben realizarse sobre sus **campos individuales**.

En versiones actuales de Oracle, los registros basados en **%ROWTYPE** también pueden utilizarse en determinadas operaciones DML, como insertar una fila completa:

```
INSERT INTO familias VALUES mi_familia;
```

Los registros son, por tanto, una forma práctica de trabajar con **varios valores relacionados como una única estructura**.

Autoevaluación

¿Es verdadero o falso?

Un programador necesita recuperar una fila completa de la tabla familias y trabajar con sus columnas dentro de PL/SQL. Declara:

mi_familia familias%ROWTYPE;
y después ejecuta:
SELECT *INTO mi_familiaFROM familiasWHERE identificador = 10;
Afirma que mi_familia tendrá campos compatibles con las columnas de la tabla familias.

○ Verdadero ○ Falso

Verdadero

%ROWTYPE crea un registro con la misma estructura que una fila de la tabla indicada. Esto facilita recuperar todos sus campos con SELECT INTO y trabajar con ellos mediante el operador punto.

Un desarrollador define dos tipos RECORD distintos que contienen exactamente los mismos campos, en el mismo orden y con los mismos tipos de datos. Después afirma que siempre podrá asignar directamente una variable de un tipo a otra del segundo tipo.

○ Verdadero ○ Falso

Falso

Que dos registros tengan los mismos campos no garantiza que sean tipos compatibles. La asignación completa depende de cómo hayan sido definidos y de la compatibilidad entre ambos tipos, no solo de que su estructura visual sea idéntica.

Una variable mi_direccion es un registro con los campos calle, numero, ciudad y pais. El programador escribe:

IF mi_direccion IS NULL THEN...END IF;
y afirma que esta es la forma correcta de comprobar si todos los campos del registro están vacíos.

○ Verdadero ○ Falso

Falso

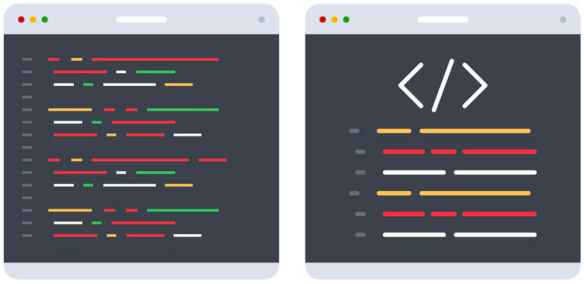
PL/SQL no permite comprobar un registro completo mediante IS NULL ni compararlo directamente con =. Si se necesita comprobar su contenido, deben evaluarse los campos individuales, por ejemplo mi_direccion.calle IS NULL.

3.2.- Colecciones. Arrays de longitud variable.

Las colecciones permiten almacenar y recorrer varios valores del mismo tipo dentro de una única estructura. En PL/SQL, conocer cómo funcionan los **VARRAY**, sus límites y sus métodos resulta útil para trabajar con conjuntos ordenados de datos de forma más clara y controlada.

Una **colección** es un conjunto de elementos del mismo tipo, cada uno identificado mediante un **subíndice**.

PL/SQL dispone de distintos tipos de colecciones. En este apartado nos centramos en los **VARRAY**, que tienen un tamaño máximo definido.



juicy_fish. *Colecciones* (CC BY-NC-SA)

VARRAY

Un VARRAY es una colección ordenada cuyo tamaño puede crecer dinámicamente hasta alcanzar un **límite máximo**.

Su sintaxis básica es:

```
TYPE nombre IS VARRAY(tamaño_maximo) OF tipo_elemento;
```

Por ejemplo:

```
TYPE numeros IS VARRAY(10) OF NUMBER;
```

En un VARRAY, los índices comienzan en **1** y los elementos se mantienen en posiciones consecutivas.

Antes de utilizarlo debe inicializarse mediante su **constructor**:

```
tabla_numeros numeros := numeros();
```

También puede inicializarse con valores:

```
tabla_numeros numeros := numeros(10, 20, 30);
```

Para acceder a un elemento se utiliza:

```
tabla_numeros(2)
```

Añadir elementos

El método **EXTEND** permite aumentar el número de elementos:

```
tabla_numeros.EXTEND;
tabla_numeros(1) := 10;
```

También existen:

- **EXTEND(n)**: añade **n** elementos.
- **EXTEND(n,i)**: añade **n** copias del elemento situado en la posición **i**.

Métodos habituales

Los **VARRAY** disponen de métodos útiles para consultar su contenido:

- **COUNT**: número actual de elementos.
- **LIMIT**: capacidad máxima.
- **FIRST**: primer índice, normalmente 1 si contiene elementos.
- **LAST**: último índice utilizado.
- **PRIOR** y **NEXT**: permiten obtener los índices anterior y siguiente.

Por ejemplo:

```
TYPE numeros IS VARRAY(20) OF NUMBER;
tabla numeros := numeros();

tabla.EXTEND(10);

DBMS_OUTPUT.PUT_LINE(tabla.COUNT); -- 10
DBMS_OUTPUT.PUT_LINE(tabla.LIMIT); -- 20
```

Dos colecciones pueden asignarse directamente si son del **mismo tipo declarado**, aunque tengan exactamente la misma estructura:

```
TYPE tabla1 IS VARRAY(10) OF NUMBER;
TYPE tabla2 IS VARRAY(10) OF NUMBER;

a tabla1 := tabla1();
```

```
b tabla2 := tabla2();

-- a := b;  -- No son del mismo tipo
```

Un uso incorrecto puede provocar excepciones como **COLLECTION_IS_NULL**, **SUBSCRIPT_BEYOND_COUNT**, **SUBSCRIPT_OUTSIDE_LIMIT** o **VALUE_ERROR**.

Por ejemplo, intentar acceder a una posición que todavía no existe puede producir un error:

```
tabla_numeros(5) := 20;
```

si la colección solo contiene cuatro elementos.

Los **VARRAY** son especialmente adecuados cuando se necesita una colección **ordenada, compacta y con un número máximo conocido de elementos**.

Autoevaluación

Un programador define el siguiente VARRAY:

```
TYPE numeros IS VARRAY(5) OF NUMBER;
tabla numeros := numeros(10, 20, 30);
```

Después ejecuta:

```
tabla.EXTEND;
tabla(4) := 40;
```

y quiere añadir posteriormente un valor en la posición 6.

¿Cuál de las siguientes afirmaciones describe correctamente la situación?

- ○ Puede escribir directamente tabla(6) := 60; porque un VARRAY aumenta automáticamente su tamaño cuando se accede a una posición nueva.
- ○ Puede usar EXTEND(2) y después asignar las posiciones 5 y 6, porque COUNT puede superar LIMIT si la colección ya estaba inicializada.
- ○ No puede llegar a utilizar la posición 6 porque el VARRAY fue declarado con límite máximo 5; EXTEND puede aumentar COUNT, pero nunca más allá de LIMIT.
- ○

Debe crear una segunda variable VARRAY(5) y asignarla a la primera para duplicar automáticamente su capacidad.

Incorrecta. Un VARRAY no crea automáticamente nuevas posiciones al acceder a ellas. Antes hay que utilizar EXTEND, y además no se puede superar el límite máximo definido.

Incorrecta. COUNT nunca puede superar LIMIT. En este caso, el límite máximo es 5, por lo que no puede existir una posición 6 válida.

Correcta. El VARRAY fue declarado como VARRAY(5), de modo que LIMIT = 5. Puede crecer mediante EXTEND hasta alcanzar cinco elementos, pero no superar esa capacidad.

Incorrecta. Crear otra colección no modifica la capacidad del tipo original. Además, las asignaciones entre colecciones requieren que sean del mismo tipo declarado y no permiten ampliar su límite máximo.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta
4. Incorrecto

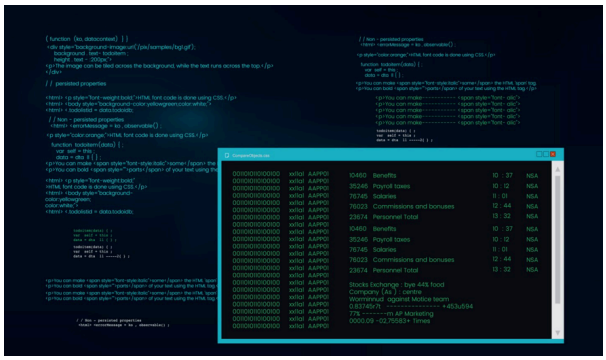
3.2.1.- Colecciones. Tablas anidadas.

Las tablas anidadas permiten trabajar con colecciones que pueden crecer dinámicamente y en las que es posible eliminar elementos concretos. Conocer sus métodos ayuda a gestionar conjuntos de datos de tamaño variable de forma más flexible que con un VARRAY.

Una **tabla anidada** es una colección cuyos elementos son del mismo tipo y cuyo tamaño **no tiene un límite máximo fijo**.

Su declaración básica es:

```
TYPE nombre IS TABLE OF tipo_elemento;
```



rawpixel.com. *Tablas anidadas* (CC BY-NC-SA)

Por ejemplo:

```
TYPE numeros IS TABLE OF NUMBER;
tabla_numeros numeros := numeros();
```

Al igual que ocurre con los VARRAY, una tabla anidada debe **inicializarse antes de utilizarse**.

Para acceder a sus elementos se utiliza:

```
tabla_numeros(1)
```

Métodos principales

- **EXTEND**: añade elementos al final.
- **COUNT**: devuelve el número de elementos existentes.
- **LIMIT**: devuelve **NULL**, ya que no existe un límite máximo fijo.
- **EXISTS(n)**: indica si existe el elemento **n**.
- **FIRST**: devuelve el primer índice existente.
- **LAST**: devuelve el último índice existente.
- **PRIOR** y **NEXT**: permiten recorrer índices existentes, ignorando los eliminados.
- **DELETE(n)**: elimina un elemento concreto.
- **DELETE(n,m)**: elimina un rango de elementos.
- **TRIM**: elimina elementos del final de la colección.

A diferencia de un VARRAY, una tabla anidada puede quedar **dispersa** después de utilizar DELETE. Por ello, COUNT no tiene por qué coincidir con LAST.

Por ejemplo:

```
TYPE numeros IS TABLE OF NUMBER;
tabla numeros := numeros(10, 20, 30, 40, 50);

tabla.DELETE(2);

DBMS_OUTPUT.PUT_LINE(tabla.COUNT); -- 4
DBMS_OUTPUT.PUT_LINE(tabla.LAST);  -- 5
```

El índice 2 ya no existe, por lo que:

```
tabla.EXISTS(2)
```

devolverá **FALSE**.

Un uso incorrecto puede producir excepciones como **COLLECTION_IS_NULL**, **NO_DATA_FOUND**, **SUBSCRIPT_BEYOND_COUNT** o **VALUE_ERROR**.

Las tablas anidadas son especialmente útiles cuando se necesita una colección **flexible, ampliable y con posibilidad de eliminar elementos individuales**.

Autoevaluación

¿Es verdadero o falso?

Una tabla anidada contiene inicialmente cinco valores:

tabla := numeros(10, 20, 30, 40, 50);
Después se ejecuta:
tabla.DELETE(2);
Un programador afirma que, tras borrar el segundo elemento, tanto COUNT como LAST devolverán 4.

○ Verdadero ○ Falso

Falso

COUNT devolverá 4 porque quedan cuatro elementos, pero LAST seguirá devolviendo 5, ya que el último índice existente continúa siendo el 5. Las tablas anidadas pueden quedar dispersas después de usar DELETE.

Una colección contiene los índices 1, 2, 3 y 4. El programador ejecuta:

tabla.DELETE(3);
y después comprueba:
tabla.EXISTS(3)

Afirma que el resultado será TRUE porque el índice 3 sigue perteneciendo al rango original de la colección.

○ Verdadero ○ Falso

Falso

DELETE(3) elimina ese elemento de la colección. Por tanto, EXISTS(3) devolverá FALSE, aunque existan índices posteriores como el 4.

Una aplicación necesita almacenar una cantidad de valores que no se conoce de antemano. El desarrollador decide utilizar una tabla anidada y afirma que puede ampliarla mediante EXTEND sin tener que definir previamente un límite máximo como sucede con un VARRAY.

○ Verdadero ○ Falso

Verdadero

Las tablas anidadas no tienen un límite máximo fijo definido en su declaración. Pueden crecer dinámicamente utilizando EXTEND, por lo que resultan adecuadas cuando el número de elementos puede variar durante la ejecución.

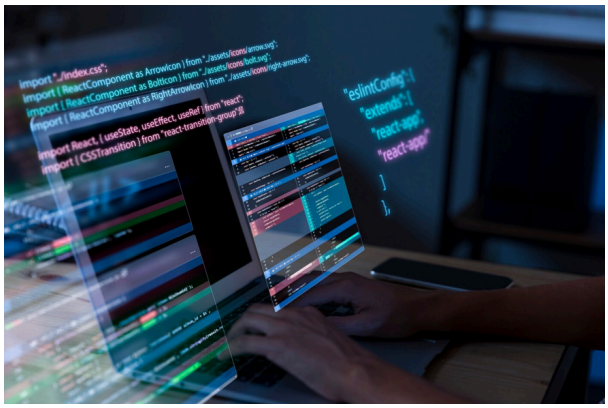
3.3.- Cursores.

Los cursores permiten trabajar con los resultados de una consulta fila a fila dentro de PL/SQL. Comprender la diferencia entre cursores implícitos y explícitos, junto con sus atributos, es fundamental para controlar correctamente consultas y operaciones DML.

Un **cursor** es una estructura que representa el conjunto de filas obtenido al ejecutar una sentencia SQL.

Oracle utiliza áreas de trabajo internas para procesar las sentencias SQL. En PL/SQL existen dos tipos principales de cursores:

- **Implícitos:** Oracle los crea automáticamente para sentencias SQL que no utilizan un cursor declarado.
- **Explícitos:** los define el programador, normalmente cuando una consulta devuelve **varias filas** y se quieren procesar individualmente.



magnific. *Cursores* (CC BY-NC-SA)

Cursores implícitos

Oracle crea automáticamente un cursor implícito para sentencias como **INSERT**, **UPDATE**, **DELETE** o consultas **SELECT INTO**.

Estos cursores se gestionan automáticamente, por lo que no se utilizan con ellos las instrucciones **OPEN**, **FETCH** ni **CLOSE**.

Sin embargo, podemos consultar información sobre la última sentencia ejecutada mediante los atributos del cursor implícito, utilizando el prefijo **SQL**.

Atributos de cursor

- **%FOUND**: indica si la última operación encontró o afectó alguna fila.
- **%NOTFOUND**: indica lo contrario a **%FOUND**.
- **%ISOPEN**: indica si el cursor está abierto. Para cursores implícitos, **SQL%ISOPEN** es siempre **FALSE**, porque Oracle los cierra automáticamente.
- **%ROWCOUNT**: devuelve el número de filas procesadas o afectadas.

Por ejemplo:

```
1 UPDATE empleados
2 SET salario = salario * 1.05
3 WHERE departamento = 20;
4
5 DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT);
```

SQL%ROWCOUNT indicará cuántos empleados fueron actualizados.

En un **cursor explícito**, estos mismos atributos se consultan utilizando el nombre del cursor, por ejemplo:

```
cursor_empleados%FOUND
cursor_empleados%NOTFOUND
cursor_empleados%ISOPEN
cursor_empleados%ROWCOUNT
```

Estos atributos permiten saber **si se han obtenido filas, cuántas se han procesado y cuál es el estado del cursor**, facilitando el control del flujo del programa.

Debes conocer

Aunque todavía no hemos visto las operaciones que se pueden realizar con un cursor explícito, es conveniente que te vayas familiarizando con la evaluación de sus atributos según las operaciones que hayamos realizado con el cursor y que tomarán pleno sentido cuando veamos el siguiente apartado.

Evaluación de los atributos de un cursor explícito.

Autoevaluación

En una aplicación PL/SQL se ejecuta la siguiente instrucción:

```
UPDATE empleados
SET salario = salario * 1.03
WHERE departamento = 50;
```

Después, el programador quiere saber si se actualizó alguna fila y cuántas fueron afectadas, sin declarar ningún cursor explícito.

¿Cuál es la opción más adecuada?

- Utilizar SQL%FOUND para comprobar si se afectó alguna fila y SQL%ROWCOUNT para conocer cuántas fueron modificadas.
- Utilizar SQL%ISOPEN para saber cuántas filas se actualizaron, ya que este atributo devuelve el número de registros procesados.
-

Declarar obligatoriamente un cursor explícito y utilizar OPEN, FETCH y CLOSE, porque los atributos SQL%... solo funcionan con consultas SELECT.

- ○ Utilizar SQL%NOTFOUND para obtener directamente el número de filas modificadas y SQL%ROWCOUNT para saber si el cursor sigue abierto.

Correcta. SQL%FOUND indica si la última sentencia DML afectó al menos una fila y SQL%ROWCOUNT devuelve el número de filas afectadas.

Incorrecta. SQL%ISOPEN no devuelve un número de filas. En los cursores implícitos es siempre FALSE, porque Oracle los cierra automáticamente.

Incorrecta. Para una sentencia UPDATE, Oracle crea automáticamente un cursor implícito. No es necesario utilizar OPEN, FETCH ni CLOSE.

Incorrecta. SQL%NOTFOUND devuelve un valor lógico que indica si no se afectó ninguna fila. No proporciona un recuento ni informa del estado de apertura del curso

Solución

1. Opción correcta
2. Incorrecto
3. Incorrecto
4. Incorrecto

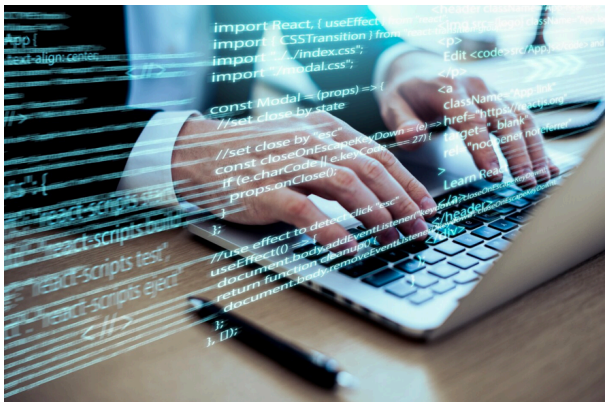
3.3.1.- Cursores explícitos.

Los cursores explícitos permiten procesar de forma controlada consultas que devuelven varias filas. Comprender cómo se declaran, abren, recorren y cierran es fundamental para trabajar con resultados múltiples dentro de PL/SQL sin perder el control sobre cada registro.

Cuando una consulta devuelve **varias filas**, podemos declarar un **cursor explícito** y recorrer su resultado fila a fila.

La declaración básica es:

```
CURSOR nombre_cursor [(parametros)] [R
IS sentencia_select;
```



magnific. *Cursores explícitos* (CC BY-NC-SA)

Por ejemplo:

```
CURSOR c_agentes IS
    SELECT * FROM agentes;
```

También puede indicarse el tipo de fila devuelto:

```
CURSOR c_familias RETURN familias%ROWTYPE IS
    SELECT * FROM familias;
```

Cursores con parámetros

Un cursor puede recibir **parámetros de entrada**, que se utilizan dentro de la consulta:

```
CURSOR c_agentes (cat INTEGER DEFAULT 0) IS
    SELECT *
    FROM agentes
    WHERE categoria = cat;
```

Los parámetros del cursor son de **entrada** y pueden tener un valor por defecto.

Abrir el cursor

Al ejecutar **OPEN**, Oracle evalúa la consulta asociada y prepara el conjunto de resultados:

```
OPEN c_agentes;
OPEN c_agentes(1);
```

Recuperar filas

La sentencia **FETCH** obtiene una fila y avanza el cursor hasta la siguiente:

```
FETCH c_familias
INTO mi_id, mi_nombre, mi_familia, mi_oficina;
```

Debe existir correspondencia entre las columnas seleccionadas y las variables del **INTO**.

Para recorrer todas las filas puede utilizarse:

```
OPEN c_familias;

LOOP
    FETCH c_familias
    INTO mi_id, mi_nombre, mi_familia, mi_oficina;

    EXIT WHEN c_familias%NOTFOUND;

    -- Procesar fila
END LOOP;

CLOSE c_familias;
```

Después de cerrar un cursor puede volver a abrirse. Intentar realizar determinadas operaciones sobre un cursor cerrado puede producir **INVALID_CURSOR**.

Bucle FOR para cursores

Una forma más sencilla es utilizar un **cursor FOR LOOP**:

```
FOR fila IN c_familias LOOP
    DBMS_OUTPUT.PUT_LINE(fila.nombre);
END LOOP;
```

En este caso, PL/SQL realiza automáticamente las operaciones de **OPEN**, **FETCH** y **CLOSE**, y crea una variable de registro para cada fila.

Por tanto, el bucle **FOR** suele ser la opción más cómoda cuando solo necesitamos **recorrer todas las filas**, mientras que **OPEN**, **FETCH** y **CLOSE** ofrecen un control más detallado sobre el

cursor.

Autoevaluación

¿Es verdadero o falso?

Se define el siguiente cursor:

```
CURSOR c_agentes (cat INTEGER DEFAULT 0) IS
SELECT *
FROM agentes
WHERE categoria = cat;
```

Un programador afirma que OPEN c_agentes; y OPEN c_agentes(0); pueden generar el mismo conjunto de resultados, ya que el parámetro tiene un valor por defecto.

○ Verdadero ○ Falso

Verdadero

Si no se proporciona un argumento, el cursor utiliza el valor definido con DEFAULT. En este caso, cat tomará el valor 0, por lo que ambas aperturas pueden ser equivalentes.

Un cursor devuelve varias filas y se recorre así:

```
OPEN c_familias;
LOOP
FETCH c_familias
INTO mi_id, mi_nombre, mi_familia, mi_oficina;
EXIT WHEN c_familias%NOTFOUND;
-- Procesar fila
END LOOP;
CLOSE c_familias;
```

Un alumno afirma que colocar EXIT WHEN c_familias%NOTFOUND después del FETCH evita procesar una fila inexistente cuando el cursor llega al final.

○ Verdadero ○ Falso

Verdadero

FETCH intenta recuperar la siguiente fila y después %NOTFOUND indica si esa operación no obtuvo datos. Al comprobarlo inmediatamente después, se evita procesar valores cuando ya no quedan filas.

Un programador utiliza:

```
FOR fila IN c_familias LOOP
DBMS_OUTPUT.PUT_LINE(fila.nombre);
END LOOP;
```

y decide añadir antes OPEN c_familias; y después CLOSE c_familias; porque piensa que el FOR LOOP no gestiona el cursor automáticamente.

○ Verdadero ○ Falso

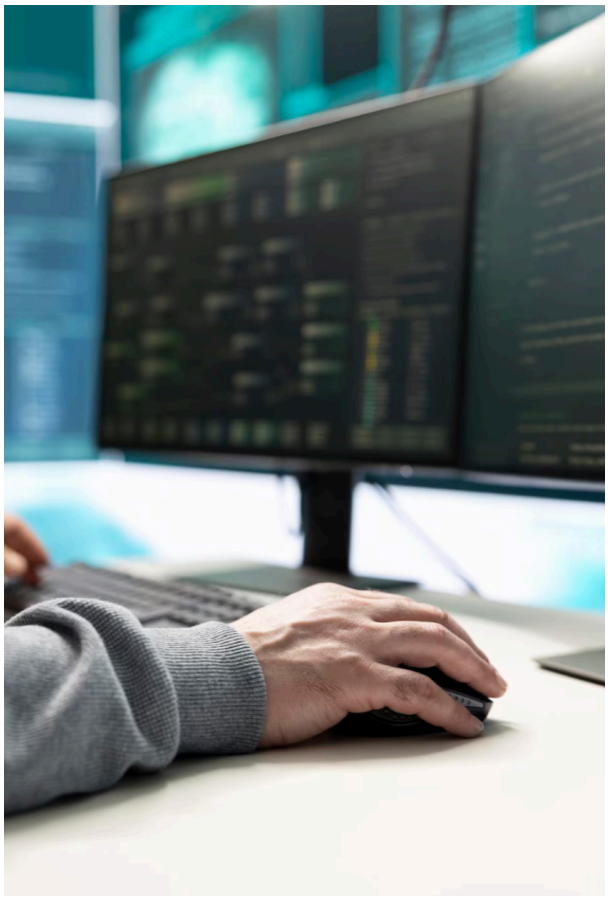
Falso

El cursor FOR LOOP abre, recupera y cierra el cursor automáticamente. No es necesario ejecutar manualmente OPEN, FETCH ni CLOSE, y hacerlo puede provocar errores o una gestión incorrecta del cursor.

3.3.2.- Cursores variables.

Los cursores variables permiten asociar una misma variable a distintas consultas durante la ejecución. Son útiles cuando la consulta que se va a recorrer no se conoce de forma fija al escribir el programa y se necesita mayor flexibilidad que con un cursor explícito tradicional.

Además de los cursores explícitos, PL/SQL permite utilizar **cursores variables**, basados en el tipo **REF CURSOR**. Funcionan como referencias a conjuntos de resultados y pueden asociarse dinámicamente a distintas consultas.



DC Studio. *Cursores variables* (CC BY-NC-SA)

Para utilizarlos se siguen dos pasos.

Primero, se define un tipo **REF CURSOR** y una variable de ese tipo:

```
TYPE tipo_cursor IS REF CURSOR RETURN
c_agentes tipo_cursor;
```

Después, durante la ejecución, se asocia el cursor variable a una consulta mediante **OPEN ... FOR**:

```
OPEN c_agentes FOR
    SELECT *
    FROM agentes
    WHERE oficina = 1;
```

A diferencia de un cursor explícito, la consulta no queda fijada en la declaración. El mismo cursor variable puede abrirse posteriormente para otra consulta compatible:

```
OPEN c_agentes FOR
    SELECT *
    FROM agentes
    WHERE categoria = 2;
```

Al volver a abrirlo, el conjunto de resultados anterior deja de estar asociado al cursor.

Una vez abierto, se utiliza de forma similar a un cursor explícito mediante **FETCH**, atributos como **%NOTFOUND** y finalmente **CLOSE**:

```plsql
DECLARE
    TYPE cursor_agentes IS REF CURSOR RETURN agentes%ROWTYPE;
    c_agentes cursor_agentes;
    agente agentes%ROWTYPE;
BEGIN
    OPEN c_agentes FOR
        SELECT *
        FROM agentes
        WHERE oficina = 1;

    LOOP
        FETCH c_agentes INTO agente;
        EXIT WHEN c_agentes%NOTFOUND;

        -- Procesar agente
    END LOOP;

    CLOSE c_agentes;
END;
/
```

Los **REF CURSOR** pueden ser **fuertes**, si especifican un tipo de retorno, o **débiles**, si se declaran sin **RETURN**, ofreciendo mayor flexibilidad.

También conviene distinguirlos del **cursor FOR LOOP**, como:

```plsql
FOR fila IN (SELECT * FROM agentes) LOOP
    ...
END LOOP;
```

Este último no utiliza una variable **REF CURSOR**, sino que PL/SQL gestiona automáticamente un cursor asociado a esa consulta.

Autoevaluación

Una aplicación PL/SQL necesita ejecutar, según una opción elegida por el usuario, una consulta sobre la tabla agentes. En un caso debe recuperar los agentes de una oficina y, en otro, los agentes de una categoría concreta. El desarrollador quiere reutilizar la misma variable de cursor.

¿Cuál de las siguientes opciones es la más adecuada?

- ○ Declarar un cursor explícito fijo con la primera consulta y después cambiar su SELECT durante la ejecución mediante FETCH.
- ○ Declarar un REF CURSOR, abrirlo con OPEN ... FOR para la consulta necesaria y reutilizarlo después con otra consulta compatible si cambia la opción del usuario.

- ○ Utilizar un cursor FOR LOOP, porque permite cambiar dinámicamente la consulta de un cursor ya abierto sin necesidad de cerrarlo ni volver a abrirlo.

- ○ Declarar un REF CURSOR fuerte y asociarlo posteriormente a cualquier consulta, aunque devuelva columnas incompatibles con el tipo indicado en RETURN.

Incorrecta. Un cursor explícito tiene su consulta asociada en la declaración. FETCH sirve para recuperar filas, no para modificar la consulta.

Correcta. Un REF CURSOR puede asociarse dinámicamente a distintas consultas mediante OPEN ... FOR, lo que permite reutilizar la misma variable de cursor según las necesidades del programa.

Incorrecta. El cursor FOR LOOP gestiona automáticamente una consulta concreta, pero no funciona como una variable REF CURSOR reutilizable para diferentes consultas.

Incorrecta. Un REF CURSOR fuerte especifica un tipo de retorno, por lo que las consultas asociadas deben ser compatibles con esa estructura. Para una mayor flexibilidad puede utilizarse un REF CURSOR débil.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

4.- Abstracción en PL/SQL.

Caso práctico

María, gracias a la ayuda de **Juan**, tiene bastante claro cómo programar en PL/SQL pero no entiende muy bien cómo integrar todo esto con la base de datos de juegos on-line. Sabe que puede utilizar unos tipos de datos, que hay unas estructuras de control y que se pueden manejar los errores que surjan, pero lo que no sabe es cómo y donde utilizar todo eso.

INTEF. *Abstracto* (CC BY-NC-SA)

Juan le explica que lo que hasta ahora ha aprendido es el comienzo, pero que ahora viene lo bueno y que será donde le va a encontrar pleno sentido a lo aprendido anteriormente. Le explica que PL/SQL permite crear funciones y procedimientos y además agruparlos en paquetes y que eso será lo que realmente van a hacer con la base de datos de juegos on-line. Deberán ver qué es lo que utilizan más comúnmente e implementarlo en PL/SQL utilizando funciones y procedimientos según convenga. **Juan** la tranquiliza y le dice que lo primero que va a hacer es explicarle cómo se escriben dichas funciones y procedimientos y luego pasarán a implementar alguno y que así verá la potencia real de PL/SQL. **María** se queda más tranquila y está deseando implementar esa primera función o procedimiento que le resolverá la gran duda que tiene.

La modularidad permite dividir una aplicación PL/SQL en unidades más pequeñas, reutilizables y fáciles de mantener. Comprender cómo funcionan los **procedimientos, funciones, paquetes y objetos** ayuda a organizar mejor el código y a centrarse en qué realiza cada módulo sin tener que conocer todos sus detalles internos.

En programación, la **abstracción** permite trabajar con componentes que realizan una tarea concreta sin necesidad de conocer constantemente cómo están implementados internamente.

PL/SQL ofrece distintos mecanismos para conseguirlo:

- **Procedimientos:** agrupan instrucciones para realizar una tarea específica.
- **Funciones:** ejecutan una tarea y **devuelven un valor**.

magnific. *Abstracción* (CC BY-NC-SA)

- **Paquetes:** permiten agrupar procedimientos, funciones, variables y otros elementos relacionados.
- **Objetos:** permiten representar información y comportamiento mediante tipos definidos por el usuario.

El uso de estas estructuras aporta varias ventajas:

- Mejora la **modularidad** del programa.
- Favorece la **reutilización del código**.
- Facilita el **mantenimiento y las modificaciones**.
- Permite ocultar detalles internos mediante **abstracción**.

A medida que una aplicación crece, organizar la lógica en módulos bien definidos permite crear programas PL/SQL más claros, escalables y fáciles de mantener.

Para saber más

En los siguientes enlaces puedes ampliar información sobre la abstracción en programación.

Abstracción en los lenguajes de programación.

Programación orientada a objetos.

4.1.- Procedimientos y funciones.

Los subprogramas permiten dividir una aplicación PL/SQL en funciones y procedimientos reutilizables. Entender cómo se declaran, cómo reciben parámetros y cómo se invocan ayuda a reducir código repetido, mejorar el mantenimiento y organizar mejor la lógica de una aplicación.

Los **subprogramas** son bloques PL/SQL con nombre que realizan una tarea concreta. Pueden recibir parámetros y estar:

- **Almacenados en la base de datos**, para poder reutilizarlos desde distintas aplicaciones o sesiones.
- **Declarados dentro de otros bloques**, con un ámbito limitado a ese contexto.

Existen dos tipos principales:

- **Funciones:** realizan una tarea y **devuelven un valor** mediante RETURN.
- **Procedimientos:** realizan una acción y no necesitan devolver un valor directamente.

vectorjuice. *Procedimientos y funciones* (CC BY-NC-SA)

Funciones

La estructura básica es:

```
FUNCTION nombre [(parametros)]
    RETURN tipo_dato IS
    declaraciones_locales
BEGIN
    sentencias;
    RETURN valor;
EXCEPTION
    manejadores;
END nombre;
```

Por ejemplo:

```
FUNCTION hijos_familia(id_familia NUMBER)
    RETURN NUMBER IS
    hijos NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO hijos
    FROM agentes
    WHERE familia = id_familia;
```

```
9
10      RETURN hijos;
11  END;
```

Procedimientos

Su estructura es similar, pero no requieren un **RETURN** de valor:

```
1  PROCEDURE nombre [(parametros)] IS
2      declaraciones_locales
3  BEGIN
4      sentencias;
5  EXCEPTION
6      manejadores;
7  END nombre;
```

Los parámetros pueden ser:

- **IN**: reciben información.
- **OUT**: devuelven información.
- **IN OUT**: reciben y pueden devolver un valor modificado.

También pueden tener valores por defecto mediante **DEFAULT** o **:=**.

En los parámetros no se indican normalmente restricciones como **NUMBER(10,2)** ni **NOT NULL**; se especifica el **tipo base**.

Los subprogramas declarados dentro de un bloque deben estar definidos antes de utilizarse. Cuando existen dependencias entre varios subprogramas puede utilizarse una **declaración adelantada**:

```
1  DECLARE
2      PROCEDURE calculo(...);
3
4      PROCEDURE inicio(...) IS
5      BEGIN
6          calculo(...);
7      END;
8  BEGIN
9      ...
10 END;
```

Esta técnica es especialmente útil cuando dos subprogramas se llaman entre sí o cuando se quiere mantener una organización lógica del código.

Autoevaluación

¿Es verdadero o falso?

Una empresa necesita crear un subprograma que reciba el identificador de una familia, calcule cuántos agentes pertenecen a ella y permita utilizar directamente ese resultado en otras expresiones PL/SQL. El programador decide implementarlo como un procedimiento sin parámetros OUT, afirmando que funcionará igual que una función.

○ Verdadero ○ Falso

Falso

Si se necesita obtener directamente un valor como resultado del subprograma, lo más adecuado es una función, ya que devuelve un valor mediante RETURN. Un procedimiento podría devolver información mediante parámetros OUT, pero no se comporta igual que una función dentro de una expresión.

Un procedimiento recibe una variable con valor 10 mediante un parámetro IN OUT, la incrementa en 5 y finaliza. El programador afirma que el subprograma puede utilizar el valor inicial y devolver al llamador el valor modificado.

○ Verdadero ○ Falso

Verdadero

Un parámetro IN OUT permite tanto recibir un valor de entrada como devolver ese mismo parámetro modificado. En este caso, puede entrar con 10 y regresar con 15.

Dentro de un bloque PL/SQL, el procedimiento inicio llama a calculo, pero calculo se define más adelante. El programador añade antes una declaración:

PROCEDURE calculo(...);
y afirma que así puede evitar el error de compilación y mantener los subprogramas en el orden lógico que desee.

○ Verdadero ○ Falso

Verdadero

La declaración adelantada informa previamente a PL/SQL de la existencia del subprograma. Esto permite utilizarlo antes de escribir su definición completa y es especialmente útil cuando existen dependencias o llamadas mutuas entre subprogramas.

4.1.1.- Almacenar subprogramas en la base de datos.

Almacenar funciones y procedimientos en Oracle permite reutilizar lógica directamente desde la base de datos. Saber crearlos, reemplazarlos, ejecutarlos y revisar sus errores de compilación facilita el mantenimiento y evita repetir código en distintas aplicaciones.

Para almacenar un subprograma en la base de datos se utiliza **CREATE** o **CREATE OR REPLACE** delante de **PROCEDURE** o **FUNCTION**.

```
1  CREATE OR REPLACE FUNCTION hijos_famil
2  RETURN NUMBER IS
3      hijos NUMBER;
4  BEGIN
5      SELECT COUNT(*)
6      INTO hijos
7      FROM agentes
8      WHERE familia = id_familia;
9
10     RETURN hijos;
11 END;
12 /
```

djvstock. *Almacenar subprogramas* (CC BY-NC-SA)

La cláusula **OR REPLACE** permite sustituir una función o procedimiento ya existente sin tener que eliminarlo previamente.

El carácter / no forma parte de PL/SQL: en herramientas como **SQL*Plus o SQLcl** indica que debe enviarse al servidor el bloque almacenado en el búfer.

Invocación de subprogramas

Desde PL/SQL, un procedimiento puede invocarse directamente:

```
1  nombre_procedimiento(parametros);
```

Una función puede utilizarse asignando su resultado:

```
1  hijos := hijos_familia(10);
```

En SQL*Plus o SQLcl también puede utilizarse **EXECUTE** para procedimientos:

```
EXECUTE nombre_procedimiento(parametros);
```

Compilación y errores

Los subprogramas almacenados se **compilan en la base de datos**. Si contienen errores, el objeto puede quedar en estado **INVALID** hasta que se corrija y compile correctamente.

Para consultar los errores puede utilizarse:

```
SHOW ERRORS
```

También existen vistas del diccionario de datos útiles:

Información	Vista
Código fuente	USER_SOURCE
Errores de compilación	USER_ERRORS
Objetos almacenados y estado	USER_OBJECTS
Tamaño de objetos	USER_OBJECT_SIZE

Por ejemplo, **USER_OBJECTS** permite comprobar si una función o procedimiento está **VALID** o **INVALID**.

En subprogramas almacenados independientes no se utilizan declaraciones adelantadas entre objetos separados; cada procedimiento o función debe poder resolver correctamente las referencias a los subprogramas que utiliza.

👀 Debes conocer

En la siguiente presentación te mostramos las vistas relacionadas con los subprogramas, la compilación de un subprograma con errores y cómo mostrar los mismos, la compilación de un subprograma correctamente y cómo mostrar su código fuente y la ejecución de un subprograma desde SQL*Plus y desde SQLDeveloper

Presentación Subprogramas.

Autoevaluación

Un administrador crea una función almacenada con:

```
CREATE OR REPLACE FUNCTION calcular_total(...)
RETURN NUMBER IS
BEGIN
    ...
END;
/
```

La compilación termina con errores y la función queda registrada en la base de datos. El administrador quiere saber qué errores contiene y comprobar después si el objeto ha quedado VALID o INVALID.

¿Cuál es la opción más adecuada?

- ○ Consultar USER_SOURCE para ver los errores y USER_ERRORS para comprobar si el objeto está VALID o INVALID.
- ○ Ejecutar SHOW ERRORS o consultar USER_ERRORS para revisar los fallos de compilación, y utilizar USER_OBJECTS para comprobar el estado del objeto.
- ○ Ejecutar de nuevo la función sin corregirla, porque CREATE OR REPLACE garantiza que el objeto siempre quede en estado VALID.
- ○ Eliminar obligatoriamente la función con DROP FUNCTION y volver a crearla, ya que OR REPLACE no permite corregir objetos existentes.

Incorrecta. USER_SOURCE permite consultar el código fuente, mientras que los errores de compilación se encuentran en USER_ERRORS. El estado del objeto se consulta en USER_OBJECTS.

Correcta. SHOW ERRORS y USER_ERRORS permiten localizar los errores de compilación, mientras que USER_OBJECTS muestra información del objeto, incluido su estado VALID o INVALID.

Incorrecta. CREATE OR REPLACE permite sustituir una definición existente, pero no garantiza que compile correctamente. Si existen errores, el objeto puede quedar INVALID.

Incorrecta. No es necesario eliminar previamente el objeto. Con CREATE OR REPLACE puede corregirse y recompilarse directamente la función o procedimiento existente.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

4.1.2.- Parámetros de los subprogramas.

Los parámetros permiten intercambiar información entre un subprograma y el código que lo llama. Comprender la diferencia entre parámetros **formales y actuales**, los modos IN, OUT e IN OUT y las distintas formas de asociación ayuda a escribir llamadas más claras y evitar errores.

En un subprograma PL/SQL distinguimos entre:

- **Parámetros formales:** aparecen en la declaración del procedimiento o función.
- **Parámetros actuales:** son los valores o variables que se pasan al invocar el subprograma.

magnific. *Parámetros de los subprogramas* (CC BY-NC-SA)

La asociación puede hacerse de varias formas.

Notación posicional

Los argumentos se relacionan según su posición:

```
1 | mi_procedimiento(valor1, valor2);
```

El primer argumento corresponde al primer parámetro formal, el segundo al segundo, etc.

Notación nombrada

Se utiliza el operador **=>**:

```
1 | mi_procedimiento(p_nombre => valor1,
2 |                  p_edad   => valor2);
```

Esta forma mejora la legibilidad y permite omitir parámetros intermedios que tengan valor por defecto.

También puede utilizarse **notación mixta**, respetando las reglas de asociación de PL/SQL.

Modos de los parámetros

- **IN**: recibe información. Es el modo **predeterminado**.
- **OUT**: devuelve información al código que llama.
- **IN OUT**: recibe un valor y puede devolverlo modificado.

Por ejemplo:

```
PROCEDURE actualizar(
    p_id      IN NUMBER,
    p_total   OUT NUMBER,
    p_contador IN OUT NUMBER
)
```

Los parámetros **OUT** e **IN OUT** necesitan recibir una **variable** como argumento, ya que el subprograma debe poder devolver un valor a través de ella.

Valores por defecto

Los parámetros **IN** pueden declararse con un **valor por defecto**:

```
PROCEDURE mostrar(
    p_texto IN VARCHAR2,
    p_lineas IN NUMBER DEFAULT 1
)
```

Puede invocarse:

```
mostrar('Hola');
```

o sobrescribir el valor:

```
mostrar('Hola', 3);
```

Si se quiere omitir un parámetro intermedio con valor por defecto, resulta especialmente útil la **notación nombrada**.

```
mi_procedimiento(
    p_primero => 10,
    p_tercero => 30
);
```

Así, los parámetros permiten crear subprogramas **más flexibles, reutilizables y fáciles de entender**.

Debes conocer

En el siguiente enlace puedes encontrar ejemplos sobre el paso de parámetros a nuestros subprogramas.

Paso de parámetros a subprogramas.

Autoevaluación

¿Es verdadero o falso?

Un procedimiento tiene la siguiente cabecera:

```
PROCEDURE calcular_total(
p_id IN NUMBER,
p_total OUT NUMBER
)
```

Un programador intenta llamarlo así:

```
calcular_total(10, 0);
```

y afirma que es válido porque 0 es compatible con el tipo NUMBER.

○ Verdadero ○ Falso

Falso

Un parámetro OUT debe recibir una variable como argumento, ya que el procedimiento necesita poder almacenar en ella el valor de salida. Un literal como 0 no puede modificarse.

Se define el procedimiento:

```
PROCEDURE mostrar(
p_texto IN VARCHAR2,
p_lineas IN NUMBER DEFAULT 1
)
```

El programador ejecuta:

```
mostrar('Aviso');
```

y afirma que p_lineas tomará automáticamente el valor 1.

○ Verdadero ○ Falso

Verdadero

Los parámetros IN pueden tener valores por defecto. Si no se proporciona un argumento para p_lineas, PL/SQL utiliza el valor definido con DEFAULT.

Un procedimiento tiene tres parámetros IN, y el segundo dispone de valor por defecto. El programador quiere proporcionar únicamente el primero y el tercero, por lo que utiliza:

```
mi_procedimiento(
p_primero => 10,
p_tercero => 30
```

);

Afirma que esta llamada es válida aunque no indique el segundo parámetro.

○ Verdadero ○ Falso

Verdadero

La notación nombrada permite asociar argumentos por nombre y omitir parámetros que tengan valor por defecto. Esto resulta especialmente útil cuando el parámetro que se desea omitir está situado entre otros dos.

4.1.3.- Sobrecarga de subprogramas y recursividad.

La sobrecarga permite utilizar el mismo nombre para varias funciones o procedimientos que realizan una tarea similar, pero reciben parámetros diferentes. Esto hace que el código resulte más claro y cómodo de usar, siempre que Oracle pueda distinguir sin ambigüedad qué versión debe ejecutar.

PL/SQL permite **sobrecargar subprogramas**, es decir, definir varias funciones o procedimientos con el mismo nombre cuando se diferencian por su lista de parámetros.

La sobrecarga puede hacerse cuando cambia:

- El **número de parámetros**.
- El **tipo de los parámetros**, siempre que Oracle pueda distinguirlos claramente.

Por ejemplo:

```
1 | PROCEDURE mostrar(valor NUMBER);
2 | PROCEDURE mostrar(valor VARCHAR2);
```

macrovector. *Sobrecarga* (CC BY-NC-SA)

Ambos procedimientos pueden coexistir porque reciben tipos diferentes.

Sin embargo, no se pueden sobrecargar subprogramas que tengan el mismo número y tipo de parámetros y solo cambien el **modo**:

```
1 | PROCEDURE calcular(valor IN NUMBER);
2 | PROCEDURE calcular(valor OUT NUMBER); -- No válido
```

Tampoco es posible diferenciarlos únicamente por tipos pertenecientes a la **misma familia** o por subtipos compatibles que puedan provocar ambigüedad.

Por tanto, la sobrecarga funciona correctamente cuando las distintas versiones tienen firmas lo bastante diferentes como para que PL/SQL pueda decidir **qué subprograma debe ejecutar en cada llamada**.

👀 Debes conocer

En el siguiente enlace podrás ver un ejemplo de una función que es sobrecargada tres veces dependiendo del tipo de parámetros que acepta.

Sobrecarga de subprogramas.

Debes conocer

En el siguiente enlace podrás ampliar información sobre la recursividad.

Recursividad.

En el siguiente enlace podrás ver un ejemplo del uso de la recursividad en nuestros subprogramas.

Ejemplo del uso de la recursividad.

Autoevaluación

En un paquete PL/SQL se quieren definir varias versiones del procedimiento mostrar para reutilizar el mismo nombre en distintas situaciones.

¿Cuál de las siguientes combinaciones representa una sobrecarga válida y sin ambigüedad?

- ○

```
PROCEDURE mostrar(valor IN NUMBER);
PROCEDURE mostrar(valor OUT NUMBER);
```

- ○

```
PROCEDURE mostrar(valor NUMBER);
PROCEDURE mostrar(valor INTEGER);
```

- ○

```
PROCEDURE mostrar(valor NUMBER);
PROCEDURE mostrar(valor VARCHAR2);
```

```
SUBTYPE entero IS NUMBER;
PROCEDURE mostrar(valor NUMBER);
PROCEDURE mostrar(valor entero);
```

Incorrecta. No se puede sobrecargar un subprograma si la única diferencia está en el modo del parámetro (IN frente a OUT).

Incorrecta. NUMBER e INTEGER pertenecen a la misma familia numérica, por lo que pueden producir ambigüedad y no sirven para distinguir de forma segura dos versiones.

Correcta. NUMBER y VARCHAR2 son tipos claramente diferenciables, por lo que Oracle puede determinar qué versión debe ejecutar según el argumento recibido.

Incorrecta. El subtipo entero deriva de NUMBER, por lo que ambas firmas siguen siendo esencialmente de la misma familia y no permiten una sobrecarga válida.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta
4. Incorrecto

4.2.- Paquetes.

Los paquetes permiten organizar en una sola unidad los elementos relacionados de una aplicación PL/SQL. Separar la interfaz pública de la implementación ayuda a estructurar mejor el código, reutilizarlo y ocultar detalles internos que no necesitan conocer otros programas.

Un **paquete** es un objeto de Oracle que agrupa **tipos, variables, constantes, cursores, procedimientos y funciones** relacionados.

Normalmente consta de dos partes:

- **Especificación del paquete:** define los elementos **públicos** que podrán utilizar otros programas.
- **Cuerpo del paquete:** contiene la implementación de los subprogramas y los elementos **privados**.

DC Studio. *Paquetes* (CC BY-NC-SA)

El cuerpo no siempre es necesario. Si la especificación solo contiene declaraciones que no requieren implementación, puede existir sin **PACKAGE BODY**.

La estructura básica es:

```
CREATE OR REPLACE PACKAGE nombre AS
    -- Declaraciones públicas
    -- Especificación de procedimientos y funciones
END nombre;
/
```

```
CREATE OR REPLACE PACKAGE BODY nombre AS
    -- Declaraciones privadas

    -- Implementación de procedimientos y funciones

BEGIN
    -- Inicialización opcional
END nombre;
/
```

La sección de **inicialización** del cuerpo es opcional y se ejecuta la primera vez que el paquete se utiliza en una sesión.

Para acceder a los elementos públicos se emplea la **notación de punto**:

```
BEGIN
    call_center.borra_agente(10);
END;
/
```

En este ejemplo, **borra_agente** es un procedimiento público perteneciente al paquete **call_center**.

Esta separación entre **interfaz pública** e **implementación privada** permite crear aplicaciones PL/SQL más organizadas y fáciles de mantener.

Debes conocer

En el siguiente enlace te mostramos un ejemplo de un paquete que agrupa las principales tareas que realizamos con nuestra base de datos de ejemplo.

Ejemplo de paquete.

Para saber más

Oracle nos suministra varios paquetes para simplificarnos algunas tareas. En el siguiente enlace puedes encontrar más información sobre los mismos.

Paquetes suministrados por Oracle.

Autoevaluación

¿Es verdadero o falso?

Un desarrollador declara el procedimiento borra_agente únicamente dentro del PACKAGE BODY del paquete call_center, pero no aparece en la especificación. Después intenta ejecutarlo desde otro bloque:

BEGIN
call_center.borra_agente(10);
END;
/
Afirma que funcionará porque el procedimiento existe dentro del paquete.

○ Verdadero ○ Falso

Falso

Para poder invocar un elemento desde fuera del paquete, debe estar declarado en la especificación, que actúa como interfaz pública. Si solo aparece en el cuerpo, se considera privado.

Un paquete contiene una sección BEGIN ... END al final de su PACKAGE BODY para inicializar varias variables internas. Un programador afirma que ese código se ejecutará cada vez que se llame a uno de sus procedimientos.

○ Verdadero ○ Falso

Falso

La sección de inicialización del paquete se ejecuta la primera vez que el paquete se utiliza en una sesión, no en cada llamada a sus procedimientos o funciones.

Un paquete contiene únicamente constantes y tipos declarados en su especificación, sin procedimientos ni funciones que necesiten implementación. El desarrollador decide no crear un PACKAGE BODY.

Afirma que el paquete puede ser válido en estas condiciones.

○ Verdadero ○ Falso

Verdadero

El cuerpo del paquete no siempre es obligatorio. Si la especificación solo contiene elementos que no requieren implementación adicional, el paquete puede existir sin PACKAGE BODY.

4.2.1.- Ejemplos de utilización del paquete DBMS_OUTPUT.

DBMS_OUTPUT permite mostrar mensajes generados desde PL/SQL durante la ejecución. Es especialmente útil para comprobar valores, seguir el flujo de un programa y localizar errores mientras se desarrollan procedimientos, funciones, paquetes o disparadores.

Oracle incluye el paquete público **DBMS_OUTPUT**, que permite escribir mensajes en un **búfer** y recuperarlos posteriormente desde herramientas cliente.

magnific. *SQL* (CC BY-NC-SA)

En **SQL*Plus** o **SQLcl**, estos mensajes pueden mostrarse activando:

```
SET SERVEROUTPUT ON
```

Su uso más habitual es la **depuración** y comprobación del funcionamiento de programas PL/SQL.

Los principales subprogramas de **DBMS_OUTPUT** son:

- **ENABLE**: habilita la salida y permite indicar el tamaño del búfer.

```
DBMS_OUTPUT.ENABLE;
DBMS_OUTPUT.ENABLE(buffer_size);
```

Si **SERVEROUTPUT** ya está activado, normalmente no es necesario llamarlo de forma explícita.

- **DISABLE**: deshabilita la salida y limpia el búfer.

```
DBMS_OUTPUT.DISABLE;
```

- **PUT**: añade información al búfer **sin salto de línea**.

```
DBMS_OUTPUT.PUT('Procesando agente ');
DBMS_OUTPUT.PUT(id_agente);
```

- **PUT_LINE**: añade información y termina la línea con un **salto de línea**.

```
DBMS_OUTPUT.PUT_LINE('Proceso completado');
```

- **NEW_LINE**: añade únicamente un salto de línea. Resulta útil después de utilizar varios **PUT**.

```
DBMS_OUTPUT.NEW_LINE;
```

- **GET_LINE**: recupera una línea del búfer. El parámetro **status** vale **0** si se obtiene una línea y **1** si no hay ninguna disponible.

```
DBMS_OUTPUT.GET_LINE(line, status);
```

- **GET_LINES**: recupera varias líneas del búfer de una sola vez. El parámetro **numlines** indica cuántas líneas se desean leer y, después de la llamada, cuántas se han recuperado realmente.

```
DBMS_OUTPUT.GET_LINES(lines, numlines);
```

En la práctica, durante el desarrollo suele utilizarse principalmente **PUT_LINE junto con SET SERVEROUTPUT ON**, ya que permite comprobar de forma sencilla qué está ocurriendo dentro de un programa PL/SQL.

Ejercicio resuelto

Debes crear un procedimiento que visualice todos los agentes, su nombre, nombre de la familia y/o nombre de la oficina a la que pertenece.

Mostrar retroalimentación

```
CREATE OR  REPLACE PROCEDURE lista_agentes IS
     CURSOR cAgentes IS SELECT identificador, nombre,familia, oficin
     fam familias.nombre%TYPE;
     ofi oficinas.nombre%TYPE;
     num_ag INTEGER := 0;
BEGIN
     DBMS_OUTPUT.ENABLE( 1000000 );
```

```
     DBMS_OUTPUT.PUT_LINE('Agente              |Familia
     DBMS_OUTPUT.PUT_LINE('----------------------------------------
     FOR ag_rec IN cAgentes LOOP
          IF (ag_rec.familia IS NOT NULL) THEN
               SELECT nombre INTO fam FROM familias WHERE identifica
               ofi := NULL;
               DBMS_OUTPUT.PUT_LINE(rpad(ag_rec.nombre,20) || '|' ||
               num_ag := num_ag + 1;
          ELSIF (ag_rec.oficina IS NOT NULL) THEN
               SELECT nombre INTO ofi FROM oficinas WHERE identifica
               fam := NULL;
               DBMS_OUTPUT.PUT_LINE(rpad(ag_rec.nombre,20) || '|' ||
               num_ag := num_ag + 1;
          END IF;
     END LOOP;
DBMS_OUTPUT.PUT_LINE('----------------------------------------
     DBMS_OUTPUT.PUT_LINE('Número de agentes: ' || num_ag);
END lista_agentes;
```

Recuerda que para ejecutarlo desde SQL*Plus debes ejecutar las siguientes sentencias:

```
SQL>SET SERVEROUTPUT ON;
SQL>EXEC lista_agentes;
```

Autoevaluación

Durante la depuración de un procedimiento PL/SQL se ejecuta el siguiente código en SQL*Plus:

```
1  SET SERVEROUTPUT ON
2
3  BEGIN
4      DBMS_OUTPUT.PUT('Agente: ');
5      DBMS_OUTPUT.PUT(25);
6      DBMS_OUTPUT.NEW_LINE;
7      DBMS_OUTPUT.PUT_LINE('Proceso finalizado');
8  END;
9  /
```

El desarrollador quiere saber qué ocurrirá con la salida mostrada.
¿Cuál de las siguientes opciones es correcta?

- ○ Se mostrarán tres líneas distintas: una con Agente:, otra con 25 y otra con Proceso finalizado.
- ○ Se mostrará una primera línea con Agente: 25 y una segunda línea con Proceso finalizado.
- ○ No se mostrará ningún mensaje porque DBMS_OUTPUT.ENABLE es obligatorio incluso cuando SET SERVEROUTPUT ON está activado.
- ○ NEW_LINE eliminará el contenido acumulado por los dos PUT anteriores y solo se mostrará Proceso finalizado.

Incorrecta. PUT añade contenido al búfer sin salto de línea, por lo que los dos primeros mensajes aparecen en la misma línea.

Correcta. Los dos PUT forman la línea Agente: 25. Después, NEW_LINE la finaliza y PUT_LINE escribe Proceso finalizado en una nueva línea.

Incorrecta. Con SET SERVEROUTPUT ON activado, normalmente no es necesario ejecutar DBMS_OUTPUT.ENABLE de forma explícita.

Incorrecta. NEW_LINE no elimina el contenido anterior. Únicamente añade un salto de línea al búfer.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

4.3.- Objetos.

Los tipos de objeto permiten aplicar conceptos de programación orientada a objetos dentro de Oracle. Entender cómo se combinan **atributos, métodos, constructores y referencias a** SELF ayuda a modelar entidades más complejas y a preparar el trabajo con objetos persistentes en la base de datos.

Un **tipo de objeto** es un tipo de dato compuesto que agrupa **atributos** y **métodos**. Los atributos almacenan el estado del objeto y los métodos definen su comportamiento.

Podemos distinguir entre:

- **Tipo de objeto:** define de forma abstracta qué atributos y métodos tendrán sus instancias.
- **Instancia:** representa un objeto concreto con valores determinados.

Los tipos de objeto suelen tener dos partes:

- **Especificación:** declara primero los atributos y después los métodos.
- **Cuerpo:** implementa los métodos declarados en la especificación.

magnific. *Objetos* (CC BY-NC-SA)

Los atributos definidos en la especificación son visibles públicamente. No se declaran atributos en el cuerpo, aunque sí pueden existir subprogramas auxiliares privados para apoyar la implementación de los métodos.

No todos los tipos PL/SQL pueden utilizarse como atributos. Entre otros, no se admiten tipos específicos de PL/SQL como BOOLEAN, PLS_INTEGER, RECORD, REF CURSOR, %TYPE o %ROWTYPE. Tampoco pueden inicializarse los atributos directamente en su declaración ni declararse con NOT NULL dentro de la definición del tipo.

Métodos

Los métodos se declaran mediante MEMBER:

```
1  CREATE TYPE empleado_t AS OBJECT (
2      id      NUMBER,
3      nombre  VARCHAR2(50),
4
5      MEMBER FUNCTION descripcion RETURN VARCHAR2
6  );
7  /
```

Cada método declarado en la especificación debe tener su correspondiente implementación en el cuerpo:

```
CREATE TYPE BODY empleado_t AS
    MEMBER FUNCTION descripcion RETURN VARCHAR2 IS
    BEGIN
        RETURN nombre;
    END;
END;
/
```

Los métodos disponen implícitamente de SELF, que representa la instancia sobre la que se está trabajando. Si se declara expresamente, debe ser el primer parámetro.

Los métodos también pueden **sobrecargarse**, siempre que Oracle pueda distinguirlos claramente por sus parámetros. No pueden diferenciarse únicamente por el modo IN, OUT o IN OUT, ni una función miembro solo por su tipo de retorno.

Creación de instancias

Una variable de tipo objeto comienza con valor NULL hasta que se inicializa mediante su constructor o se le asigna otra instancia.

```
DECLARE
    empleado empleado_t;
BEGIN
    empleado := empleado_t(10, 'Ana');
END;
/
```

El constructor por defecto tiene el **mismo nombre que el tipo** y recibe los valores de sus atributos.

Si se intenta acceder a un atributo de un objeto que todavía es NULL, Oracle puede generar la excepción ACCESS_INTO_NULL.

Un tipo de objeto también puede contener como atributo otro tipo de objeto. En ese caso, el tipo utilizado debe estar previamente definido o resolverse mediante una **declaración adelantada** cuando exista dependencia entre tipos.

Ejercicio resuelto

Cómo declararías los objetos para nuestra base de datos de ejemplo.

Mostrar retroalimentación

```
CREATE OBJECT Oficina;     --Definición hacia delante
CREATE OBJECT Familia AS OBJECT (
     identificador     NUMBER,
     nombre          VARCHAR2(20),
     familia_     Familia,
     oficina_     Oficina,
     ...
);
CREATE OBJECT Agente AS OBJECT (
     identificador     NUMBER,
     nombre          VARCHAR2(20),
     familia_     Familia,
     oficina_     Oficina,
     ...
);
CREATE OBJECT Oficina AS OBJECT (
     identificador     NUMBER,
     nombre          VARCHAR2(20),
     jefe          Agente,
     ...
);
```

Autoevaluación

¿Es verdadero o falso?

Un desarrollador declara una variable de tipo empleado_t y, antes de llamar a su constructor, intenta mostrar su nombre:

DECLARE
empleado empleado_t;
BEGIN
DBMS_OUTPUT.PUT_LINE(empleado.nombre);
END;
/
Afirma que Oracle mostrará NULL, porque el atributo nombre todavía no tiene ningún valor.

○ Verdadero ○ Falso

Falso

La variable empleado representa un objeto nulo completo, no simplemente un objeto cuyos atributos contienen NULL. Al intentar acceder a empleado.nombre antes de inicializar la instancia, puede producirse la excepción ACCESS_INTO_NULL.

Se define el siguiente método:

MEMBER FUNCTION descripcion RETURN VARCHAR2 IS
BEGIN
RETURN nombre;
END;
Un programador afirma que es válido acceder directamente a nombre sin escribir SELF.nombre, porque el método trabaja implícitamente sobre la instancia que lo invoca.

○ Verdadero ○ Falso

Verdadero

Los métodos MEMBER disponen implícitamente de SELF, que representa el objeto actual. Por ello, dentro del método puede accederse a sus atributos directamente, aunque también podría hacerse explícitamente mediante SELF.nombre.

En un tipo de objeto se intentan declarar estas dos funciones:

MEMBER FUNCTION calcular(p_valor NUMBER) RETURN NUMBER,
MEMBER FUNCTION calcular(p_valor NUMBER) RETURN VARCHAR2
El desarrollador afirma que la sobrecarga es válida porque ambas funciones devuelven tipos distintos.

○ Verdadero ○ Falso

Falso

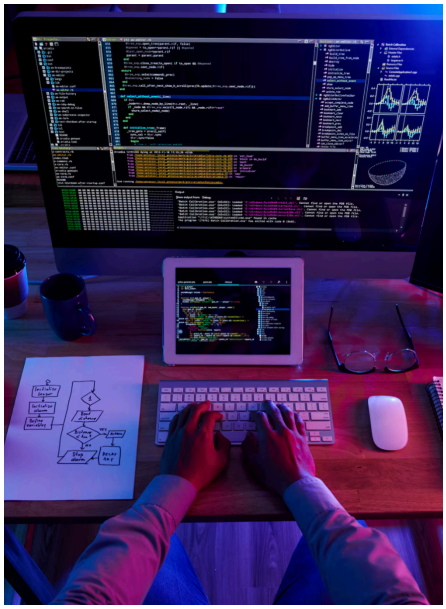
Una función miembro no puede sobrecargarse únicamente por el tipo de retorno. Como ambas reciben exactamente el mismo parámetro NUMBER, Oracle no puede decidir qué versión debe ejecutar basándose solo en el valor que devolverán.

4.3.1.- Objetos. Funciones mapa y funciones de orden.

Los objetos de Oracle no tienen un criterio de ordenación implícito. Si queremos compararlos, ordenarlos o utilizarlos de forma coherente en operaciones que necesitan establecer un orden, debemos definir cómo se comparan mediante funciones **MAP** u **ORDER**. Comprender esta diferencia evita resultados ambiguos y permite trabajar mejor con tipos de objeto.

Por defecto, los **tipos de objeto** no tienen un orden definido. Por ello, no pueden compararse directamente con operadores como `>`, `<` ni utilizarse de forma natural en operaciones como **ORDER BY** o **DISTINCT** sin establecer antes un criterio de comparación.

Oracle permite definir ese criterio mediante dos mecanismos: **funciones MAP** y **funciones ORDER**.



pressfoto. *Funciones mapa* (CC BY-NC-SA)

Funciones MAP

Una función **MAP** es una **función miembro sin parámetros** que transforma el objeto en un valor escalar que Oracle puede comparar.

Puede devolver tipos como **NUMBER**, **VARCHAR2** o **DATE**. Solo puede existir una función **MAP** por tipo de objeto.

```
CREATE TYPE Familia AS OBJECT (
    identificador NUMBER,
    nombre        VARCHAR2(20),

    MAP MEMBER FUNCTION orden RETURN NUMBER
);
/
```

```
CREATE TYPE BODY Familia AS
    MAP MEMBER FUNCTION orden RETURN NUMBER IS
    BEGIN
        RETURN identificador;
    END;
END;
/
```

En este caso, dos objetos **Familia** se compararán utilizando su **identificador**.

Funciones ORDER

Una función ORDER compara directamente el objeto actual con **otro objeto del mismo tipo**.

Debe devolver:

- Un valor **negativo** si el objeto actual es menor.
- **0** si ambos son iguales.
- Un valor **positivo** si el objeto actual es mayor.

```
CREATE TYPE Oficina AS OBJECT (
    identificador NUMBER,
    nombre        VARCHAR2(20),

    ORDER MEMBER FUNCTION comparar(ofi Oficina)
        RETURN INTEGER
);
/
```

```
CREATE TYPE BODY Oficina AS
    ORDER MEMBER FUNCTION comparar(ofi Oficina)
        RETURN INTEGER IS
    BEGIN
        IF identificador < ofi.identificador THEN
            RETURN -1;
        ELSIF identificador = ofi.identificador THEN
            RETURN 0;
        ELSE
            RETURN 1;
        END IF;
    END;
END;
/
```

La diferencia principal es que MAP **convierte cada objeto en un valor comparable**, mientras que ORDER **compara directamente dos objetos entre sí**. Para un mismo tipo de objeto se utiliza uno de estos mecanismos de comparación, no ambos.

Autoevaluación

Una aplicación almacena objetos del tipo Producto_t y necesita ordenarlos por su precio. El desarrollador quiere que Oracle pueda comparar los objetos directamente y decide definir el criterio dentro del propio tipo.

¿Cuál de las siguientes soluciones es la más adecuada?

- ○ Crear una función MAP que reciba como parámetro otro objeto Producto_t y devuelva -1, 0 o 1 según el precio.

- ○ Crear una función ORDER sin parámetros que devuelva directamente el precio del producto como NUMBER.
- ○ Crear una función MAP sin parámetros que devuelva el precio como NUMBER, de forma que Oracle utilice ese valor para comparar y ordenar los objetos.
- ○ Definir simultáneamente una función MAP y una función ORDER, para que Oracle utilice automáticamente la más apropiada en cada comparación.

Incorrecta. Ese comportamiento corresponde a una función ORDER, no a una MAP. Una función MAP no recibe otro objeto como parámetro.

Incorrecta. Una función ORDER debe comparar el objeto actual con otro objeto del mismo tipo, por lo que necesita recibir ese objeto como parámetro.

Correcta. Una función MAP puede devolver un valor escalar, como un NUMBER, que Oracle utilizará como criterio de comparación. Si devuelve el precio, los objetos podrán ordenarse según ese valor.

Incorrecta. Para un mismo tipo de objeto se utiliza un único mecanismo de comparación, MAP u ORDER, no ambos simultáneamente.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta
4. Incorrecto

5.- Disparadores y eventos.

Caso práctico

Juan y **María** ya han hecho un paquete en el que tienen agrupadas las funciones más usuales que realizan sobre la base de datos de juegos on-line. Sin embargo, **María** ve que hay cosas que aún no pueden controlar. Por ejemplo, **María** quiere que la clave y el usuario de un jugador o jugadora no puedan ser la misma y eso no sabe cómo hacerlo. **Juan** le dice que para ese cometido están los disparadores y sin más dilaciones se pone a explicarle a **María** para qué sirven y cómo utilizarlos.

Loren (INTEF). *Clave* (CC BY-NC-SA)

Los disparadores o TRIGGER permiten ejecutar automáticamente acciones cuando ocurre un evento en la base de datos. Bien utilizados, ayudan a reforzar la **integridad, la auditoría y la automatización**, aunque conviene evitar reglas innecesariamente complejas o duplicar funciones que Oracle ya ofrece de forma nativa.

Un **disparador** es un bloque PL/SQL que se ejecuta automáticamente cuando se produce un determinado **evento** en la base de datos y se cumplen las condiciones definidas.

magnific. *Disparadores y eventos* (CC BY-NC-SA)

Los principales tipos son:

- **Disparadores de tabla**.
- **Disparadores INSTEAD OF**.
- **Disparadores de sistema**.

Disparadores de tabla

Se asocian a operaciones de manipulación de datos sobre una tabla, como:

- **INSERT**
- **UPDATE**
- **DELETE**

Pueden ejecutarse:

- **BEFORE**: antes de realizar la operación.
- **AFTER**: después de realizarla.

También pueden actuar:

- **Una vez por sentencia**, cuando no se especifica **FOR EACH ROW**.
- **Una vez por cada fila afectada**, utilizando **FOR EACH ROW**.

Por ejemplo, un trigger de fila puede utilizarse para registrar cambios realizados sobre datos sensibles.

Disparadores INSTEAD OF

Los disparadores **INSTEAD OF** se ejecutan **en lugar de** la operación que los activa.

Se utilizan sobre **vistas** y permiten controlar cómo se realizan operaciones **INSERT**, **UPDATE** o **DELETE** sobre ellas.

Disparadores de sistema

Se activan ante eventos relacionados con la base de datos o con determinados objetos. Algunos ejemplos son:

- **LOGON**
- **LOGOFF**
- **CREATE**
- **DROP**

Dependiendo del evento, pueden definirse como **BEFORE** o **AFTER**.

Los disparadores pueden utilizarse para:

- Registrar **auditorías** o históricos.
- Aplicar reglas de **integridad** complejas.
- Automatizar determinadas acciones asociadas a cambios en los datos.
- Reforzar controles de **seguridad** específicos.

Al diseñarlos conviene evitar que sustituyan funcionalidades que Oracle ya proporciona mediante **restricciones, valores por defecto u otros mecanismos nativos**. También es recomendable mantenerlos pequeños y claros; si la lógica crece demasiado, es preferible trasladarla a procedimientos o funciones que el trigger pueda invocar.

Además, debe prestarse especial atención a los **disparadores recursivos** y a las dependencias entre triggers, ya que pueden provocar comportamientos difíciles de mantener o depurar.

Autoevaluación

¿Es verdadero o falso?

Una empresa quiere registrar en una tabla de auditoría cada modificación individual realizada sobre los salarios de los empleados. Una sentencia

UPDATE puede afectar a 200 filas.

El desarrollador afirma que basta con crear un trigger AFTER UPDATE sin FOR EACH ROW, porque Oracle ejecutará automáticamente el trigger una vez por cada fila modificada.

○ Verdadero ○ Falso

Falso

Sin FOR EACH ROW, el disparador se ejecuta una sola vez por sentencia, aunque el UPDATE afecte a muchas filas. Para registrar cada modificación individual debe utilizarse un trigger de fila con FOR EACH ROW.

Una aplicación necesita permitir actualizaciones sobre una vista compleja. El desarrollador propone crear un trigger INSTEAD OF UPDATE sobre la vista para definir manualmente qué operaciones deben realizarse en las tablas implicadas.

Afirma que este enfoque es correcto porque el trigger sustituye la operación original.

○ Verdadero ○ Falso

Verdadero

Los triggers INSTEAD OF se utilizan sobre vistas y se ejecutan en lugar de la operación INSERT, UPDATE o DELETE que los activa. Esto permite controlar cómo se aplican realmente los cambios.

En una tabla, la columna estado debe tomar siempre el valor 'ACTIVO' cuando no se indique otro. Un programador decide crear un trigger BEFORE INSERT exclusivamente para asignar ese valor.

Afirma que esta es siempre la opción más recomendable porque los triggers deben utilizarse para cualquier automatización de datos.

○ Verdadero ○ Falso

Falso

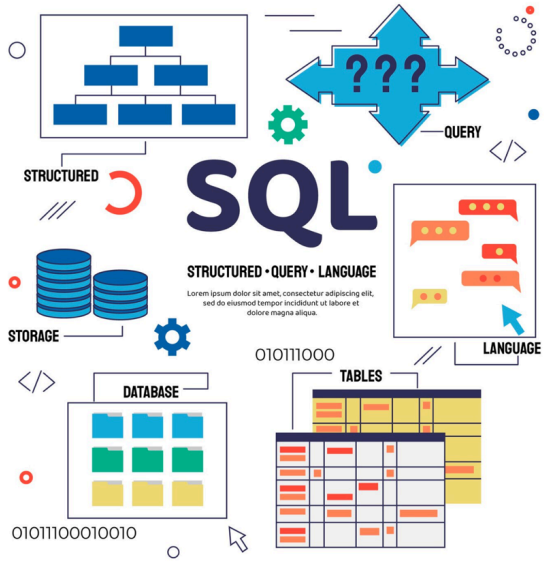
Los triggers no deberían duplicar funcionalidades que Oracle puede resolver con mecanismos más sencillos y claros. En este caso, un valor DEFAULT en la columna suele ser más apropiado que crear un trigger únicamente para esa tarea.

5.1.- Definición de disparadores de Tablas

Los disparadores de tabla permiten reaccionar automáticamente ante operaciones INSERT, UPDATE o DELETE y controlar qué ocurre antes o después de cada cambio. Comprender cuándo se ejecutan, cómo usar :OLD y :NEW y qué limitaciones tienen ayuda a evitar errores difíciles de detectar y a diseñar triggers más seguros y mantenibles.

Para definir un disparador sobre una tabla debemos indicar:

- El **momento** de ejecución: BEFORE o AFTER.
- El **evento** que lo activa: INSERT, UPDATE o DELETE.
- Si se ejecutará **una vez por sentencia** o **una vez por cada fila** mediante FOR EACH ROW.



magnific. *Disparadores de tablas* (CC BY-NC-SA)

La estructura básica es:

```
CREATE OR REPLACE TRIGGER nombre
    momento evento ON tabla
    [REFERENCING OLD AS alias_old NEW
    [FOR EACH ROW]
    [WHEN (condicion)]
BEGIN
    -- Sentencias PL/SQL
END;
/
```

Las cláusulas REFERENCING y WHEN se utilizan en disparadores de fila. REFERENCING permite asignar alias a los valores antiguos y nuevos, mientras que WHEN limita la ejecución a las filas que cumplan una condición.

Valores :OLD y :NEW

En un trigger de fila podemos consultar los valores anteriores y nuevos de cada registro:

- INSERT: tiene sentido :NEW.
- UPDATE: están disponibles :OLD y :NEW.
- DELETE: tiene sentido :OLD.

Por ejemplo, en una actualización:

```
IF :NEW.salario <> :OLD.salario THEN
    ...
END IF;
```

También pueden utilizarse los predicados:

- **INSERTING**
- **UPDATING**
- **DELETING**

Estos permiten saber qué operación ha provocado la ejecución del trigger.

Tabla mutante

Un trigger **de fila** no debe consultar ni modificar directamente la misma tabla que está provocando su ejecución. Si lo intenta, Oracle puede generar el error **ORA-04091: table is mutating**.

```
CREATE OR REPLACE TRIGGER prueba
BEFORE UPDATE ON agentes
FOR EACH ROW
BEGIN
    SELECT identificador
    INTO ...
    FROM agentes
    WHERE ...;  -- Puede provocar ORA-04091
END;
/
```

Por ello, conviene diseñar con cuidado los triggers de fila y evitar dependencias recursivas entre disparadores.

Orden general de ejecución

Cuando existen triggers de distintos niveles sobre una misma operación, el orden general es:

1. **BEFORE de sentencia**
2. **BEFORE de fila**
3. **AFTER de fila**
4. **AFTER de sentencia**

Si existen varios triggers del mismo tipo y es necesario controlar su orden relativo, Oracle permite utilizar cláusulas como **FOLLOWS**.

Para consultar los disparadores definidos por el usuario puede utilizarse la vista:

```
USER_TRIGGERS
```

En ella se puede revisar información como el **nombre del trigger, evento, tabla asociada, condición, estado y cuerpo** del disparador.

Autoevaluación

Una empresa tiene un trigger sobre la tabla empleados que debe auditar cambios de salario. El trigger está definido como BEFORE UPDATE FOR EACH ROW. Dentro de él, el programador compara :OLD.salario y :NEW.salario, pero además ejecuta una consulta SELECT sobre la propia tabla empleados para calcular el salario medio del departamento.

¿Cuál es la afirmación correcta?

- ○ El trigger es correcto porque, al ser BEFORE, Oracle todavía no ha modificado la tabla y puede consultarse sin restricciones.
- ○ El trigger puede provocar el error ORA-04091 porque un trigger de fila no debe consultar directamente la misma tabla que está siendo modificada.
- ○ El problema se evita sustituyendo :OLD y :NEW por los predicados UPDATING y INSERTING.
- ○ El trigger funcionará correctamente siempre que la consulta sobre empleados aparezca después de comparar :OLD.salario y :NEW.salario.

Incorrecta. El hecho de que sea un trigger BEFORE no elimina la restricción de tabla mutante. El problema se debe a que es un trigger de fila sobre la misma tabla que intenta consultar.

Correcta. Un trigger FOR EACH ROW que intenta consultar directamente la tabla que está siendo modificada puede producir el error ORA-04091: table is mutating.

Incorrecta. UPDATING, INSERTING y DELETING sirven para identificar qué operación activó el trigger, pero no solucionan el problema de la tabla mutante.

Incorrecta. El orden de las instrucciones dentro del trigger no elimina la restricción. Aunque primero se utilicen :OLD y :NEW, una consulta posterior sobre la misma tabla puede seguir provocando ORA-04091.

Solución

1. Incorrecto

2. Opción correcta
3. Incorrecto
4. Incorrecto

5.2.- Ejemplos de disparadores.

Los disparadores permiten automatizar tareas y aplicar reglas directamente cuando se producen cambios en la base de datos. Ver ejemplos prácticos ayuda a entender cuándo resulta útil utilizarlos y cómo pueden contribuir a la **integridad, auditoría y automatización** de una aplicación.

magnific. *SQL* (CC BY-NC-SA)

Una vez conocidos los **tipos de disparadores**, su sintaxis y las principales consideraciones de diseño, es importante ver cómo se aplican en situaciones reales.

Los ejemplos permiten comprobar cómo un trigger puede responder automáticamente ante operaciones como **INSERT**, **UPDATE** o **DELETE**, utilizando elementos como **:OLD**, **:NEW**, condiciones y distintos momentos de ejecución.

A partir de estos casos prácticos se puede entender mejor la utilidad de los disparadores y, sobre todo, cuándo conviene utilizarlos frente a otros mecanismos disponibles en Oracle.

Debes conocer

En la siguiente presentación podrás ver la descripción de la tabla **USER_TRIGGERS**, cómo se almacena un disparador en la base de datos, cómo se consulta su código y cómo se lanza su ejecución. Pruébalo en tu máquina para comprobar su funcionamiento.

Presentación sobre disparadores. Creación, consulta de su código y lanzamiento con una sentencia insert.

Ejercicio resuelto

Como un agente debe pertenecer a una familia o una oficina pero no puede pertenecer a una familia y a una oficina a la vez, deberemos implementar un disparador para llevar a cabo esta restricción que Oracle no nos permite definir desde el DDL.

Mostrar retroalimentación

Para este cometido definiremos un disparador de fila que saltará antes de que insertemos o actualicemos una fila en la tabla agentes, cuyo código podría ser el siguiente:

```
CREATE OR REPLACE TRIGGER integridad_agentes
BEFORE INSERT OR UPDATE ON agentes
FOR EACH ROW
BEGIN
     IF (:new.familia IS NULL and :new.oficina IS NULL) THEN -- Si l
          RAISE_APPLICATION_ERROR(-20201, 'Un agente no puede ser hu
     ELSIF (:new.familia IS NOT NULL and :new.oficina IS NOT NULL) T
          RAISE_APPLICATION_ERROR(-20202, 'Un agente no puede tener
     END IF;
END;
/
```

Ejercicio resuelto

Supongamos que tenemos una tabla de históricos (histagentes) para agentes que nos permita auditar las familias y oficinas por la que ha ido pasando un agente. La tabla tiene la fecha de inicio y la fecha de finalización del agente en esa familia u oficina, el identificador del agente, el nombre del agente, el nombre de la familia y el nombre de la oficina. Queremos hacer un disparador que registre (inserte o actualice en la tabla histagentes) los cambios que se producen en la tabla agentes (al insertar, actualizar o borrar).

La tabla histagentes necesita ser creada antes de la creación del trigger y es la siguiente:

```
create table histagentes (
       fecha_inicio date,
       fecha_hasta date,
       identificador number(6),
       nombre varchar2(40),
       familia varchar2(40),
       oficina varchar2(40)
 );
```

Mostrar retroalimentación

Para llevar a cabo esta tarea definiremos un disparador de fila que saltará después de insertar, actualizar o borrar una fila en la tabla agentes, cuyo código podría ser el siguiente:

```
CREATE OR REPLACE TRIGGER historico_agentes
AFTER INSERT OR UPDATE OR DELETE ON agentes
FOR EACH ROW
DECLARE
     oficina VARCHAR2(40);
     familia VARCHAR2(40);
     ahora DATE := sysdate;
BEGIN
     IF INSERTING THEN
          IF (:new.familia IS NOT NULL) THEN
               SELECT nombre INTO familia FROM familias WHERE identi
               oficina := NULL;
          ELSE
               SELECT nombre INTO oficina FROM oficinas WHERE identi
               familia := NULL;
          END IF;
          INSERT INTO histagentes VALUES (ahora, NULL, :new.identifi
          COMMIT;
     ELSIF UPDATING THEN
          UPDATE histagentes SET fecha_hasta = ahora WHERE identific
          IF (:new.familia IS NOT NULL) THEN
               SELECT nombre INTO familia FROM familias WHERE identi
               oficina := NULL;
          ELSE
               SELECT nombre INTO oficina FROM oficinas WHERE identi
               familia := NULL;
          END IF;
          INSERT INTO histagentes VALUES (ahora, NULL, :new.identifi
          COMMIT;
     ELSE
          UPDATE histagentes SET fecha_hasta = ahora WHERE identific
          COMMIT;
     END IF;
```

```
END;
/
```

Ejercicio resuelto

Queremos realizar un disparador que no nos permita llevar a cabo operaciones con familias si no estamos en la jornada laboral.

Mostrar retroalimentación

```
CREATE OR REPLACE TRIGGER jornada_familias
BEFORE INSERT OR DELETE OR UPDATE ON familias
DECLARE
     ahora DATE := sysdate;
BEGIN
     IF (TO_CHAR(ahora, 'DY') = 'SAT' OR TO_CHAR(ahora, 'DY') = 'SUN
          RAISE_APPLICATION_ERROR(-20301, 'No podemos manipular fami
     END IF;
     IF (TO_CHAR(ahora, 'HH24') < 8 OR TO_CHAR(ahora, 'HH24') > 18)
          RAISE_APPLICATION_ERROR(-20302, 'No podemos manipular fami
     END IF;
END;
/
```

6.- Interfaces de programación de aplicaciones para lenguajes externos.

Caso práctico

Juan y **María** tienen la base de datos de juegos on-line lista, pero no tienen claro si todo lo que han hecho lo tienen que utilizar desde dentro de la base de datos o si pueden reutilizar todo ese trabajo desde otro lenguaje de programación desde el que están creando la interfaz web desde la que los jugadores y jugadoras se conectarán para jugar y desde la que también se conectará la administradora para gestionar la base de datos de una forma más amigable.

En ese momento pasa **Ada** al lado y ambos la asaltan con su gran duda. **Ada**, muy segura, les responde que para eso existen las interfaces de programación de aplicaciones o APIs, que les permiten acceder desde otros lenguajes de programación a la base de datos.

Abraham Pérez Pérez (INTEF). *Duda* (CC BY-NC-SA)

La programación de una base de datos no queda aislada dentro de Oracle: puede integrarse con aplicaciones desarrolladas en otros lenguajes. Conocer las principales **APIs y controladores de acceso a datos** permite conectar programas externos con Oracle y aprovechar desde ellos consultas, procedimientos, funciones y otras operaciones de la base de datos.

Una vez que sabemos programar la base de datos, es importante comprender que esa lógica puede utilizarse desde aplicaciones desarrolladas en otros lenguajes.

Para ello se emplean **APIs, controladores y librerías de acceso a bases de datos**, que actúan como puente entre la aplicación y Oracle.

Históricamente, Oracle proporcionó tecnologías como **Pro*C**, **Pro*Fortran** y **Pro*COBOL**. Estas permitían incluir instrucciones de acceso a la base de datos dentro del código fuente y utilizar un **precompilador** antes de la compilación normal del programa.

Elaboración propia. *Interfaces de programación* (CC BY-NC-SA)

Entre ellas, **Pro*C** tuvo una gran difusión por el uso extendido de C y C++.

Actualmente existen alternativas mucho más habituales según el lenguaje y la plataforma, por ejemplo:

- **JDBC** para aplicaciones Java.
- **ODBC** para aplicaciones que utilizan esta interfaz estándar.
- **python-oracledb** para Python.
- **Oracle JDBC** y **Oracle Database drivers** para distintos entornos de desarrollo.
- Tecnologías de acceso desde **.NET** mediante proveedores específicos de Oracle.

La evolución de las aplicaciones web, los servicios distribuidos y la computación en la nube ha hecho que las formas de acceso a las bases de datos sean mucho más variadas.

Lo importante es entender que una aplicación externa puede **conectarse a Oracle, ejecutar SQL e invocar procedimientos y funciones PL/SQL**, eligiendo el controlador o API que mejor se adapte al lenguaje y al entorno utilizado.

Para saber más

En los siguientes enlaces podrás obtener más información sobre Pro*C y cómo crear un programa para acceder a nuestra base de datos.

Introducción a Pro*C.

Para saber más

En los siguientes enlaces podrás ampliar información sobre algunas APIs muy comunes para acceder a bases de datos.

Tecnologías Java para acceder a bases de datos.

Conectividad abierta de bases de datos. (ODBC)

Autoevaluación

¿Es verdadero o falso?

Una empresa desarrolla una aplicación en Python que debe consultar datos de Oracle y ejecutar un procedimiento PL/SQL ya almacenado en la base de datos. Un programador afirma que es necesario reescribir ese procedimiento completamente en Python porque el código PL/SQL no puede invocarse desde una aplicación externa.

○ Verdadero ○ Falso

Falso

Una aplicación externa puede conectarse a Oracle mediante un controlador como python-oracledb y desde ella ejecutar SQL e invocar procedimientos y funciones PL/SQL almacenados en la base de datos.

Un desarrollador sostiene que Pro*C y JDBC funcionan exactamente de la misma forma, ya que ambos obligan a utilizar un precompilador que transforma las llamadas a Oracle antes de compilar el programa.

○ Verdadero ○ Falso

Falso

Tecnologías como Pro*C utilizaban un precompilador para transformar instrucciones embebidas en el código fuente. En cambio, JDBC funciona mediante una API y un controlador de acceso a la base de datos, sin seguir ese mismo modelo de precompilación.

Una organización dispone de una aplicación Java y otra desarrollada en Python. El equipo decide usar JDBC para Java y python-oracledb para Python, manteniendo en Oracle los mismos procedimientos PL/SQL utilizados por ambas aplicaciones.

El responsable afirma que esta arquitectura es válida porque distintas aplicaciones pueden acceder a la misma lógica almacenada utilizando el controlador adecuado para cada lenguaje.

○ Verdadero ○ Falso

Verdadero

La lógica PL/SQL almacenada en Oracle puede ser reutilizada desde diferentes aplicaciones. Cada una puede utilizar la API o controlador más adecuado a su entorno, como JDBC en Java o python-oracledb en Python.

Anexo I.- Caso de estudio.

Una empresa de telefonía tiene sus **centros de llamadas** distribuidos por la geografía española en diferentes **oficinas**. Estas oficinas están jerarquizadas en **familias de agentes telefónicos**. Cada familia, por tanto, podrá contener **agentes** u **otras familias**.

Los agentes telefónicos, según su **categoría**, además se encargarán de **supervisar** el trabajo de todos los agentes de una oficina o de **coordinar** el trabajo de los agentes de una familia dada.

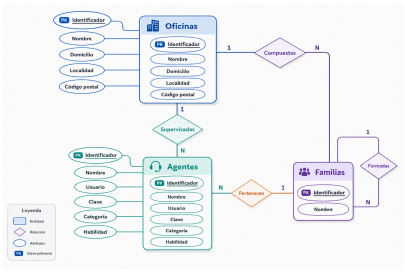
El único agente que pertenecerá directamente a una oficina y que no formará parte de ninguna familia será el **supervisor de dicha oficina**, cuya **categoría es la 2**.

Los **coordinadores de las familias** deben pertenecer a dicha familia y su **categoría será 1**. No todas las familias tienen por qué tener un coordinador, ya que esta función también puede asumirla el supervisor de la oficina.

Los demás agentes deberán pertenecer a una familia, su **categoría será 0** y serán los que principalmente se ocupen de **atender las llamadas**.

- De los **agentes** queremos conocer su **nombre**, su **clave y contraseña** para entrar al sistema, su **categoría** y su **habilidad**, que será un número entre **0 y 9** indicando su capacidad para atender llamadas.
- Para las **familias** sólo nos interesa conocer su **nombre**.
- Finalmente, para las **oficinas** queremos saber su **nombre, domicilio, localidad y código postal**.

Un posible modelo entidad-relación para el problema expuesto podría ser el siguiente:



Elaboración propia. *Modelo Entidad-relación del caso de estudio.* (CC BY-NC-SA)

El **Modelo Relacional** resultante sería:

OFICINAS (**identificador**, nombre, domicilio, localidad, codigo_postal)

FAMILIAS (**identificador**, nombre, **familia (FK)**, **oficina (FK)**)

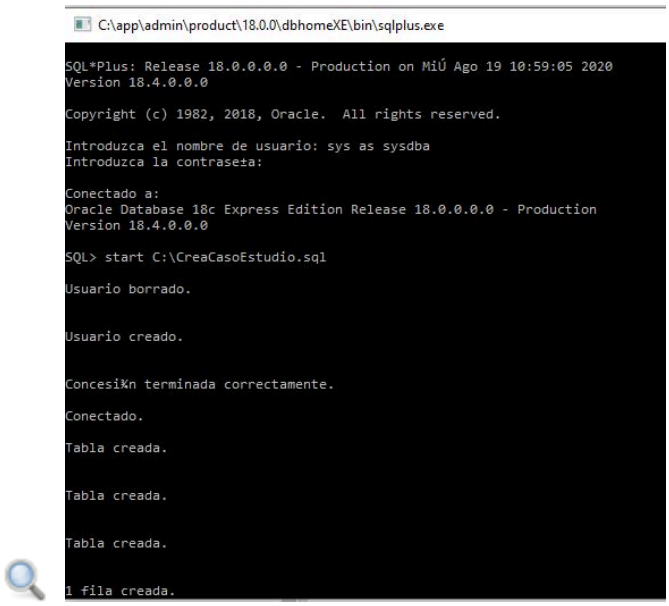
AGENTES (**identificador**, nombre, usuario, clave, habilidad, categoría, **familia (FK)**, **oficina (FK)**)

De este modelo de datos surgen **tres tablas**, que puedes crear en **Oracle** con el script del siguiente enlace:

Script CreaCasoEstudio.zip (zip - 1,62 KB)

El script crea un usuario llamado **c##agencia** con la clave **agencia**. Para poder ejecutarlo tantas veces como sea necesario y comenzar desde cero, el propio script **elimina previamente el usuario c##agencia y sus tablas** antes de volver a crearlos.

Conecta como administrador mediante **SYS AS SYSDBA** y ejecuta el script anteponiendo el símbolo **@** o la palabra **START** al nombre del archivo.



Ejecución script en SQLPlus (CC BY-NC-SA)

Habilitando la Salida/OUTPUT en Bloques PL/SQL.

PL/SQL no proporciona funcionalidad de **entrada o salida directamente**, siendo necesario utilizar **paquetes predefinidos de Oracle** para tales fines. Para generar una salida debes realizar lo siguiente:

- Ejecutar el siguiente comando tanto en **SQL*Plus** como en cualquier otro entorno:

```
SET SERVEROUTPUT ON
```

- En el bloque **PL/SQL**, hay que utilizar el procedimiento **PUT_LINE** del paquete **DBMS_OUTPUT** para **mostrar la salida**. El valor a mostrar en pantalla se pasará como **argumento del procedimiento**.

Ejecuta el siguiente código desde **SQLPlus** y desde **SQLDeveloper** para comprobar su funcionamiento.

```
SET SERVEROUTPUT ON
BEGIN

```

```
    DBMS_OUTPUT.PUT_LINE('Hola mundo');
END;
```

Autoevaluación

Una empresa incorpora un nuevo agente al sistema. El empleado atenderá llamadas, pertenece a una familia concreta de una oficina de Madrid, tiene una habilidad de 8 y no realiza tareas de supervisión ni coordinación.

¿Cuál de las siguientes configuraciones es la más coherente con el modelo planteado?

- ○ Registrar al agente con categoría 2, asignarlo directamente a la oficina y dejar su familia vacía, porque todos los agentes pertenecen primero a una oficina.
- ○ Registrar al agente con categoría 1, asignarlo a una familia y a una oficina, porque cualquier agente que pertenezca a una familia debe actuar como coordinador.
- ○ Registrar al agente con categoría 0, asociarlo a una familia y a la oficina correspondiente, manteniendo su habilidad dentro del intervalo permitido de 0 a 9.
- ○ Registrar al agente con categoría 0, asociarlo únicamente a la oficina y dejar la familia vacía, porque la pertenencia a una familia es opcional para los agentes de atención.

Incorrecta. La categoría 2 corresponde al supervisor de la oficina, que es precisamente el único agente que pertenece directamente a la oficina sin formar parte de una familia.

Incorrecta. La categoría 1 corresponde a los coordinadores de familia. Un agente que simplemente atiende llamadas debe tener categoría 0.

Correcta. Los agentes encargados principalmente de atender llamadas tienen categoría 0, deben pertenecer a una familia y estar vinculados a su oficina. Además, una habilidad de 8 es válida porque se encuentra entre 0 y 9.

Incorrecta. Los agentes de categoría 0 deben pertenecer a una familia. Dejar la familia vacía rompería la regla organizativa definida para este tipo de agente.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta
4. Incorrecto

Anexo II.- Excepciones predefinidas en Oracle.

Las excepciones predefinidas son:

Excepciones predefinidas en Oracle.

Excepción.	SQLCODE	Lanzada cuando ...
ACCES_INTO_NULL	-6530	Intentamos asignar valor a atributos de objetos no inicializados.
COLECTION_IS_NULL	-6531	Intentamos asignar valor a elementos de colecciones no inicializadas, o acceder a métodos distintos de EXISTS.
CURSOR_ALREADY_OPEN	-6511	Intentamos abrir un cursor ya abierto.
DUP_VAL_ON_INDEX	-1	Índice único violado.
INVALID_CURSOR	-1001	Intentamos hacer una operación con un cursor que no está abierto.
INVALID_NUMBER	-1722	Conversión de cadena a número falla.
LOGIN_DENIED	-1403	El usuario y/o contraseña para conectarnos a Oracle no es válido.
NO_DATA_FOUND	+100	Una sentencia SELECT no devuelve valores, o intentamos acceder a un elemento borrado de una tabla anidada.
NOT_LOGGED_ON	-1012	No estamos conectados a Oracle.
PROGRAM_ERROR	-6501	Ha ocurrido un error interno en PL/SQL.
ROWTYPE_MISMATCH	-6504	Diferentes tipos en la asignación de 2 cursores.
STORAGE_ERROR	-6500	Memoria corrupta.
SUBSCRIPT_BEYOND_COUNT	-6533	El índice al que intentamos acceder en una colección sobrepasa su límite superior.
SUBSCRIPT_OUTSIDE_LIMIT	-6532	Intentamos acceder a un rango no válido dentro de una colección (-1 por ejemplo).
TIMEOUT_ON_RESOURCE	-51	Un timeout ocurre mientras Oracle espera por un recurso.

Excepción.	SQLCODE	Lanzada cuando ...
TOO_MANY_ROWS	-1422	Una sentencia SELECT...INTO... devuelve más de una fila.
VALUE_ERROR	-6502	Ocurre un error de conversión, aritmético, de truncado o de restricción de tamaño.
ZERO_DIVIDE	-1476	Intentamos dividir un número por 0.

Autoevaluación

¿Es verdadero o falso?

Un procedimiento recibe el valor '12A' en una variable VARCHAR2 e intenta convertirlo directamente a NUMBER para realizar una operación aritmética. El programador afirma que, si la conversión falla, Oracle puede lanzar la excepción INVALID_NUMBER.

○ Verdadero ○ Falso

Verdadero

La excepción INVALID_NUMBER se produce cuando Oracle no puede convertir correctamente una cadena de caracteres a un valor numérico. Un texto como '12A' no representa un número válido.

Un bloque PL/SQL ejecuta:

```
SELECT nombre
INTO v_nombre
FROM agentes
WHERE categoria = 0;
```

La consulta devuelve cinco filas. El desarrollador afirma que Oracle lanzará NO_DATA_FOUND porque una variable no puede almacenar varias filas.

○ Verdadero ○ Falso

Falso

NO_DATA_FOUND se produce cuando un SELECT ... INTO no devuelve ninguna fila. Si devuelve más de una fila, la excepción correcta es TOO_MANY_ROWS.

Se declara una colección, pero no se ejecuta su constructor. Después se intenta consultar coleccion.COUNT. El programador afirma que esa operación puede provocar COLLECTION_IS_NULL, mientras que EXISTS es una excepción porque puede utilizarse incluso cuando la colección no está inicializada.

○ Verdadero ○ Falso

Verdadero

Una colección no inicializada es una colección nula. Intentar utilizar métodos como COUNT, FIRST o acceder a sus elementos puede producir COLLECTION_IS_NULL. El método EXISTS es la excepción y puede utilizarse sin provocar ese error.

Anexo III.- Evaluación de los atributos de un cursor explícito.

Evaluación de los atributos de un cursor explícito según las operaciones realizadas con él.

Operación realizada.	%FOUND	%NOTFOUND	%ISOPEN	%ROWCOUNT
Antes del OPEN	Excepción.	Excepción.	FALSE	Excepción.
Después del OPEN	NULL	NULL	TRUE	0
Antes del primer FETCH	NULL	NULL	TRUE	0
Después del primer FETCH	TRUE	FALSE	TRUE	1
Antes de los siguientes FETCH	TRUE	FALSE	TRUE	1
Después de los siguientes FETCH	TRUE	FALSE	TRUE	Depende datos.
Antes del último FETCH	TRUE	FALSE	TRUE	Depende datos.
Después del último FETCH	FALSE	TRUE	TRUE	Depende datos.
Antes del CLOSE	FALSE	TRUE	TRUE	Depende datos.
Después del CLOSE	Excepción.	Excepción.	FALSE	Excepción.

Autoevaluación

Un programa PL/SQL utiliza un cursor explícito c_agentes. El cursor se abre correctamente y contiene 3 filas. Después se ejecutan tres FETCH, uno por cada fila, y a continuación se realiza un cuarto FETCH para comprobar si quedan más registros.

Justo después de ese cuarto FETCH, ¿qué combinación de atributos es la correcta?

- ○ %FOUND = TRUE, %NOTFOUND = FALSE, %ISOPEN = TRUE, %ROWCOUNT = 4
- ○

%FOUND = FALSE, %NOTFOUND = TRUE, %ISOPEN = TRUE, %ROWCOUNT = 3

- ○ %FOUND = FALSE, %NOTFOUND = TRUE, %ISOPEN = FALSE, %ROWCOUNT = 3
- ○ %FOUND = NULL, %NOTFOUND = NULL, %ISOPEN = TRUE, %ROWCOUNT = 0

Incorrecta. El cuarto FETCH no recupera ninguna fila, por lo que %FOUND pasa a FALSE y %NOTFOUND a TRUE. Además, %ROWCOUNT no aumenta porque no se ha recuperado una nueva fila.

Correcta. Tras intentar leer más allá de la última fila, %FOUND = FALSE, %NOTFOUND = TRUE y el cursor sigue abierto, por lo que %ISOPEN = TRUE. Como solo se han recuperado 3 filas, %ROWCOUNT = 3.

Incorrecta. Un FETCH que no encuentra más filas no cierra automáticamente el cursor. %ISOPEN seguirá siendo TRUE hasta ejecutar CLOSE.

Incorrecta. Los valores NULL para %FOUND y %NOTFOUND corresponden al estado posterior a OPEN y anterior al primer FETCH, no después de recorrer todas las filas.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

Anexo IV.- Paso de parámetros a subprogramas.

Veamos algunos ejemplos del paso de parámetros a subprogramas:

- Notación mixta.

```
DECLARE
     PROCEDURE prueba( formal1 NUMBER, formal2 VARCHAR2) IS
     BEGIN
          ...
     END;
     actual1 NUMBER;
     actual2 VARCHAR2;
BEGIN
     ...
     prueba(actual1, actual2);               --posicional
     prueba(formal2=>actual2,formal1=>actual1);     --nombrada
     prueba(actual1, formal2=>actual2);          --mixta
END;
```

- Parámetros de entrada.

```
FUNCTION categoria( id_agente IN NUMBER )
RETURN NUMBER IS
     cat NUMBER;
BEGIN
     ...
     SELECT categoria INTO cat FROM agentes
WHERE identificador = id_agente;
RETURN cat;
EXCEPTION
     WHEN NO_DATA_FOUND THEN
          id_agente := -1; --ilegal, parámetro de entrada
END;
```

- Parámetros de salida.

```
PROCEDURE nombre( id_agente NUMBER, nombre OUT VARCHAR2) IS
BEGIN
     IF (nombre = 'LUIS') THEN     --error de sintaxis
     END IF;
     ...
END;
```

- Parámetros con valor por defecto de los que podemos prescindir.

```
DECLARE
     SUBTYPE familia IS familias%ROWTYPE;
     SUBTYPE agente IS agentes%ROWTYPE;
     SUBTYPE tabla_agentes IS TABLE OF agente;
     familia1 familia;
     familia2 familia;
     hijos_fam tabla_agentes;
     FUNCTION inserta_familia( mi_familia familia,
          mis_agentes tabla_agentes := tabla_agentes() )
RETURN NUMBER IS
     BEGIN
          INSERT INTO familias VALUES (mi_familia);
          FOR i IN 1..mis_agentes.COUNT LOOP
               IF (mis_agentes(i).oficina IS NOT NULL) or (mis_agentes(i).familia
                    ROLLBACK;
                    RETURN -1;
               END IF;
               INSERT INTO agentes VALUES (mis_agentes(i));
          END LOOP;
          COMMIT;
          RETURN 0;
     EXCEPTION
          WHEN DUP_VAL_ON_INDEX THEN
               ROLLABACK;
               RETURN -1;
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN SQLCODE;
     END inserta_familia;
BEGIN
     ...
     resultado := inserta_familia( familia1 );
     ...
     resultado := inserta_familia( familia2, hijos_fam2 );
     ...
END;
```

Autoevaluación

¿Es verdadero o falso?

Un procedimiento está declarado así:

PROCEDURE prueba(formal1 NUMBER, formal2 VARCHAR2);
Un programador realiza esta llamada:
prueba(formal2 => actual2, actual1);

y afirma que es una notación mixta válida porque uno de los argumentos está asociado por nombre y el otro por posición.

○ Verdadero ○ Falso

Falso

En la notación mixta, los argumentos posicionales deben aparecer antes que los argumentos nombrados. Una llamada válida sería:

prueba(actual1, formal2 => actual2);

Una vez utilizada la asociación por nombre, no se debe volver a la asociación posicional.

Una función recibe id_agente como parámetro IN. En el manejador de excepciones intenta ejecutar:

id_agente := -1;
El desarrollador afirma que es válido porque la asignación solo se realiza cuando ocurre NO_DATA_FOUND.

○ Verdadero ○ Falso

Falso

Un parámetro IN es de entrada y no puede modificarse dentro del subprograma. El hecho de realizar la asignación dentro de una sección EXCEPTION no cambia esta restricción.

Una función se declara así:

FUNCTION inserta_familia(
mi_familia familia,
mis_agentes tabla_agentes := tabla_agentes()
)
RETURN NUMBER;
Un programador invoca:
resultado := inserta_familia(familia1);
y afirma que la llamada es válida porque mis_agentes tiene un valor por defecto y, por tanto, puede omitirse.

○ Verdadero ○ Falso

Verdadero

Cuando un parámetro dispone de un valor por defecto, puede omitirse en la llamada. En este caso, mis_agentes tomará automáticamente el valor tabla_agentes(), mientras que mi_familia seguirá siendo obligatorio.

Anexo V.- Sobrecarga de subprogramas.

Veamos un ejemplo de sobrecarga de subprogramas en el que una misma función es sobrecargada tres veces para diferentes tipos de parámetros.

```
DECLARE
     TYPE agente IS agentes%ROWTYPE;
     TYPE familia IS familias%ROWTYPE;
     TYPE tAgentes IS TABLE OF agente;
     TYPE tFamilias IS TABLE OF familia;

     FUNCTION inserta_familia( mi_familia familia )
     RETURN NUMBER IS
     BEGIN
          INSERT INTO familias VALUES (mi_familia.identificador, mi_familia.nombre, mi_
          COMMIT;
          RETURN 0;
     EXCEPTION
          WHEN DUP_VAL_ON_INDEX THEN
               ROLLBACK;
               RETURN -1;
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN SQLCODE;
     END inserta_familia;

     FUNCTION inserta_familia( mi_familia familia, hijas tFamilias )
     RETURN NUMBER IS
     BEGIN
          INSERT INTO familias VALUES (mi_famila.identificador, mi_familia.nombre, mi_f
          IF (hijas IS NOT NULL) THEN
               FOR i IN 1..hijas.COUNT LOOP
                    IF (hijas(i).oficina IS NOT NULL) or (hijas(i).familia ¡= mi_famili
                         ROLLBACK;
                         RETURN -1;
                    END IF;
                    INSERT INTO familias VALUES (hijas(i).identificador, hijas(i).nombr
               END LOOP;
          END IF;
          COMMIT;
          RETURN 0;
     EXCEPTION
          WHEN DUP_VAL_ON_INDEX THEN
               ROLLBACK;
               RETURN -1;
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN -1;
     END inserta_familia;

     FUNCTION inserta_familia( mi_familia familia, hijos tAgentes )
     RETURN NUMBER IS
     BEGIN
```

```
          INSERT INTO familias VALUES (mi_familia.identificador, mi_familia.nombre, mi_
          IF (hijos IS NOT NULL) THEN
               FOR i IN 1..hijos.COUNT LOOP
                    IF (hijos(i).oficina IS NOT NULL) or (hijos(i).familia ¡= mi_famili
                         ROLLBACK;
                         RETURN -1;
                    END IF;
                    INSERT INTO agentes VALUES (hijos(i).identificador, hijos(i).nombre
               END LOOP;
          END IF;
          COMMIT;
          RETURN 0;
     EXCEPTION
          WHEN DUP_VAL_ON_INDEX THEN
               ROLLBACK;
               RETURN -1;
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN -1;
     END inserta_familias;

     mi_familia familia;
     mi_familia1 familia;
     familias_hijas tFamilias;
     mi_familia2 familia;
     hijos tAgentes;
BEGIN
     ...
     resultado := inserta_familia(mi_familia);
     ...
     resultado := inserta_familia(mi_familia1, familias_hijas);
     ...
     resultado := inserta_familia(mi_familia2, hijos);
     ...
```

Autoevaluación

En un bloque PL/SQL existen tres versiones sobrecargadas de la función **inserta_familia**:

- Una recibe únicamente una variable de tipo **familia**.
- Otra recibe una **familia** y una colección **tFamilias**.
- Otra recibe una **familia** y una colección **tAgentes**.

Durante la ejecución se realizan estas llamadas:

```
1 resultado := inserta_familia(mi_familia);
2 resultado := inserta_familia(mi_familia1, familias_hijas);
3 resultado := inserta_familia(mi_familia2, hijos);
```

¿Cuál de las siguientes afirmaciones explica correctamente cómo PL/SQL decide qué versión ejecutar?

- ○ PL/SQL ejecuta siempre la primera función declarada con ese nombre y después adapta automáticamente los parámetros sobrantes.
- ○ PL/SQL distingue las versiones por el número y tipo de los parámetros, por lo que cada llamada puede resolverse hacia una función diferente sin cambiar el nombre.
- ○ PL/SQL solo puede sobrecargar procedimientos, pero no funciones, por lo que las tres definiciones producirían un error de compilación.
- ○ PL/SQL diferencia únicamente por el número de parámetros, por lo que las dos funciones que reciben dos argumentos serían ambiguas y no podrían coexistir.

Incorrecta. PL/SQL no selecciona una función por el simple orden en que fue declarada ni adapta automáticamente argumentos incompatibles.

Correcta. La sobrecarga permite utilizar el mismo nombre siempre que Oracle pueda distinguir las versiones por su firma, en este caso por el número de parámetros y por tipos diferentes como tFamilias y tAgentes.

Incorrecta. PL/SQL permite sobrecargar tanto funciones como procedimientos, siempre que las distintas versiones puedan diferenciarse de forma inequívoca.

Incorrecta. Aunque dos funciones reciben dos parámetros, el segundo parámetro tiene tipos distintos: una recibe tFamilias y otra tAgentes. Por ello, PL/SQL puede determinar qué versión corresponde a cada llamada.

Solución

1. Incorrecto
2. Opción correcta
3. Incorrecto
4. Incorrecto

Anexo VI.- Ejemplo de recursividad.

Aquí tienes un ejemplo del uso de la recursividad en nuestros subprogramas.

```
DECLARE
     TYPE agente IS agentes%ROWTYPE;
     TYPE tAgentes IS TABLE OF agente;
     hijos10 tAgentes;
     PROCEDURE dame_hijos( id_familia NUMBER,
                    hijos IN OUT tAgentes ) IS
          CURSOR hijas IS SELECT identificador FROM familias WHERE familia = id_familia
hija NUMBER;
     CURSOR cHijos IS SELECT * FROM agentes WHERE familia = id_familia;
hijo agente;
     BEGIN
          --Si la tabla no está inicializada -> la inicializamos
          IF hijos IS NULL THEN
               hijos = tAgentes();
          END IF;
          --Metemos en la tabla los hijos directos de esta familia
          OPEN cHijos;
          LOOP
               FETCH cHijos INTO hijo;
               EXIT WHEN cHijos%NOTFOUND;
               hijos.EXTEND;
               hijos(hijos.LAST) := hijo;
          END LOOP;
          CLOSE cHijos;
          --Hacemos lo mismo para todas las familias hijas de la actual
          OPEN hijas;
          LOOP
               FETCH hijas INTO hija;
               EXIT WHEN hijas%NOTFOUND;
               dame_hijos( hija, hijos );
          END LOOP;
          CLOSE hijas;
     END dame_hijos;
BEGIN
     ...
     dame_hijos( 10, hijos10 );
     ...
END;
```

Autoevaluación

¿Es verdadero o falso?

El procedimiento dame_hijos recibe la colección hijos como parámetro IN OUT. En cada llamada añade los agentes de la familia actual y después se llama recursivamente para cada familia hija.

Un programador afirma que los agentes encontrados en las llamadas recursivas se acumulan en la misma colección, en lugar de perderse al volver de cada llamada.

○ Verdadero ○ Falso

Verdadero

Al pasar hijos como parámetro IN OUT, cada llamada recursiva trabaja sobre la misma colección recibida y puede ampliarla con EXTEND. Así, los agentes encontrados en distintos niveles de la jerarquía quedan acumulados.

El procedimiento no contiene una condición explícita del tipo IF nivel = 0 THEN RETURN. Un desarrollador afirma que, por ese motivo, la recursividad será necesariamente infinita.

○ Verdadero ○ Falso

Falso

La recursividad termina de forma natural cuando una familia no tiene familias hijas. En ese caso, el cursor hijas no devuelve filas y no se realizan nuevas llamadas a dame_hijos.

Se ejecuta:

dame_hijos(10, hijos10);
La familia 10 no tiene agentes directos, pero posee una familia hija 20, y esta sí contiene varios agentes. Un programador afirma que hijos10 quedará vacío porque el primer cursor cHijos de la familia 10 no recupera ninguna fila.

○ Verdadero ○ Falso

Falso

Aunque la familia 10 no tenga agentes directos, el procedimiento recorre sus familias hijas de forma recursiva. Al llamar a dame_hijos(20, hijos10), los agentes de la familia 20 se añadirán a la colección.

Anexo VII.- Ejemplo de paquete.

Aquí puedes ver un ejemplo de un paquete que agrupa las principales tareas que llevamos a cabo con la base de datos de ejemplo.

```
CREATE OR REPLACE PACKAGE call_center AS     --inicialización
     --Definimos los tipos que utilizaremos
     SUBTYPE agente IS agentes%ROWTYPE;
     SUBTYPE familia IS familias%ROWTYPE;
     SUBTYPE oficina IS oficinas%ROWTYPE;
     TYPE tAgentes IS TABLE OF agente;
     TYPE tFamilias IS TABLE OF familia;
     TYPE tOficinas IS TABLE OF oficina;

     --Definimos las excepciones propias
     referencia_no_encontrada exception;
     referencia_encontrada exception;
     no_null exception;
     PRAGMA EXCEPTION_INIT(referencia_no_encontrada, -2291);
     PRAGMA EXCEPTION_INIT(referencia_encontrada, -2292);
     PRAGMA EXCEPTION_INIT(no_null, -1400);

     --Definimos los errores que vamos a tratar
     todo_bien               CONSTANT NUMBER := 0;
     elemento_existente           CONSTANT NUMBER:= -1;
     elemento_inexistente           CONSTANT NUMBER:= -2;
     padre_existente          CONSTANT NUMBER:= -3;
     padre_inexistente          CONSTANT NUMBER:= -4;
     no_null_violado          CONSTANT NUMBER:= -5;
     operacion_no_permitida          CONSTANT NUMBER:= -6;

     --Definimos los subprogramas públicos
     --Nos devuelve la oficina padre de un agente
     PROCEDURE oficina_padre( mi_agente agente, padre OUT oficina );

     --Nos devuelve la oficina padre de una familia
     PROCEDURE oficina_padre( mi_familia familia, padre OUT oficina );

     --Nos da los hijos de una familia
     PROCEDURE dame_hijos( mi_familia familia, hijos IN OUT tAgentes );

     --Nos da los hijos de una oficina
     PROCEDURE dame_hijos( mi_oficina oficina, hijos IN OUT tAgentes );

     --Inserta un agente
     FUNCTION inserta_agente ( mi_agente agente )
     RETURN NUMBER;

     --Inserta una familia
     FUNCTION inserta_familia( mi_familia familia )
     RETURN NUMBER;

     --Inserta una oficina
```

```
     FUNCTION inserta_oficina ( mi_oficina oficina )
     RETURN NUMBER;

     --Borramos una oficina
     FUNCTION borra_oficina( id_oficina NUMBER )
     RETURN NUMBER;

     --Borramos una familia
     FUNCTION borra_familia( id_familia NUMBER )
     RETURN NUMBER;

     --Borramos un agente
     FUNCTION borra_agente( id_agente NUMBER )
     RETURN NUMBER;
END call_center;
/
CREATE OR REPLACE PACKAGE BODY call_center AS          --cuerpo
     --Implemento las funciones definidas en la especificación

     --Nos devuelve la oficina padre de un agente
     PROCEDURE oficina_padre( mi_agente agente, padre OUT oficina ) IS
          mi_familia familia;
     BEGIN
          IF (mi_agente.oficina IS NOT NULL) THEN
               SELECT * INTO padre FROM oficinas
               WHERE identificador = mi_agente.oficina;
          ELSE
               SELECT * INTO mi_familia FROM familias
               WHERE identificador = mi_agente.familia;
               oficina_padre( mi_familia, padre );
          END IF;
     EXCEPTION
          WHEN OTHERS THEN
               padre := NULL;
     END oficina_padre;

     --Nos devuelve la oficina padre de una familia
     PROCEDURE oficina_padre( mi_familia familia, padre OUT oficina ) IS
          madre familia;
     BEGIN
          IF (mi_familia.oficina IS NOT NULL) THEN
               SELECT * INTO padre FROM oficinas
               WHERE identificador = mi_familia.oficina;
          ELSE
               SELECT * INTO madre FROM familias
               WHERE identificador = mi_familia.familia;
               oficina_padre( madre, padre );
          END IF;
     EXCEPTION
          WHEN OTHERS THEN
               padre := NULL;
     END oficina_padre;

     --Nos da los hijos de una familia
     PROCEDURE dame_hijos( mi_familia familia, hijos IN OUT tAgentes ) IS
          CURSOR cHijos IS SELECT * FROM agentes
          WHERE familia = mi_familia.identificador;
          CURSOR cHijas IS SELECT * FROM familias
```

```
          WHERE familia = mi_familia.identificador;
          hijo agente;
          hija familia;
     BEGIN
          --inicializamos la tabla si no lo está
          if (hijos IS NULL) THEN
               hijos := tAgentes();
          END IF;
          --metemos en la tabla los hijos directos
          OPEN cHijos;
          LOOP
               FETCH cHijos INTO hijo;
               EXIT WHEN cHijos%NOTFOUND;
               hijos.EXTEND;
               hijos(hijos.LAST) := hijo;
          END LOOP;
          CLOSE cHijos;
          --hacemos lo mismo para las familias hijas
          OPEN cHijas;
          LOOP
               FETCH cHijas INTO hija;
               EXIT WHEN cHijas%NOTFOUND;
               dame_hijos( hija, hijos );
          END LOOP;
          CLOSE cHijas;
     EXCEPTION
          WHEN OTHERS THEN
               hijos := tAgentes();
     END dame_hijos;

     --Nos da los hijos de una oficina
     PROCEDURE dame_hijos( mi_oficina oficina, hijos IN OUT tAgentes ) IS
          CURSOR cHijos IS SELECT * FROM agentes
          WHERE oficina = mi_oficina.identificador;
          CURSOR cHijas IS SELECT * FROM familias
          WHERE oficina = mi_oficina.identificador;
          hijo agente;
          hija familia;
     BEGIN
          --inicializamos la tabla si no lo está
          if (hijos IS NULL) THEN
               hijos := tAgentes();
          END IF;
          --metemos en la tabla los hijos directos
          OPEN cHijos;
          LOOP
               FETCH cHijos INTO hijo;
               EXIT WHEN cHijos%NOTFOUND;
               hijos.EXTEND;
               hijos(hijos.LAST) := hijo;
          END LOOP;
          CLOSE cHijos;
          --hacemos lo mismo para las familias hijas
          OPEN cHijas;
          LOOP
               FETCH cHijas INTO hija;
               EXIT WHEN cHijas%NOTFOUND;
               dame_hijos( hija, hijos );
```

```
          END LOOP;
          CLOSE cHijas;
     EXCEPTION
          WHEN OTHERS THEN
               hijos := tAgentes();
     END dame_hijos;

     --Inserta un agente
     FUNCTION inserta_agente ( mi_agente agente )
     RETURN NUMBER IS
     BEGIN
          IF (mi_agente.familia IS NULL and mi_agente.oficina IS NULL) THEN
               RETURN operacion_no_permitida;
          END IF;
          IF (mi_agente.familia IS NOT NULL and mi_agente.oficina IS NOT NULL) THEN
               RETURN operacion_no_permitida;
          END IF;
          INSERT INTO agentes VALUES (mi_agente.identificador, mi_agente.nombre, mi_age
          COMMIT;
          RETURN todo_bien;
     EXCEPTION
          WHEN referencia_no_encontrada THEN
               ROLLBACK;
               RETURN padre_inexistente;
          WHEN no_null THEN
               ROLLBACK;
               RETURN no_null_violado;
          WHEN DUP_VAL_ON_INDEX THEN
               ROLLBACK;
               RETURN elemento_existente;
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN SQLCODE;
     END inserta_agente;

     --Inserta una familia
     FUNCTION inserta_familia( mi_familia familia )
     RETURN NUMBER IS
     BEGIN
          IF (mi_familia.familia IS NULL and mi_familia.oficina IS NULL) THEN
               RETURN operacion_no_permitida;
          END IF;
          IF (mi_familia.familia IS NOT NULL and mi_familia.oficina IS NOT NULL) THEN
               RETURN operacion_no_permitida;
          END IF;
          INSERT INTO familias VALUES ( mi_familia.identificador, mi_familia.nombre, mi
          COMMIT;
          RETURN todo_bien;
     EXCEPTION
          WHEN referencia_no_encontrada THEN
               ROLLBACK;
               RETURN padre_inexistente;
          WHEN no_null THEN
               ROLLBACK;
               RETURN no_null_violado;
           WHEN DUP_VAL_ON_INDEX THEN
               ROLLBACK;
               RETURN elemento_existente;
```

```
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN SQLCODE;
     END inserta_familia;

     --Inserta una oficina
     FUNCTION inserta_oficina ( mi_oficina oficina )
     RETURN NUMBER IS
     BEGIN
          INSERT INTO oficinas VALUES (mi_oficina.identificador, mi_oficina.nombre, mi_
          COMMIT;
          RETURN todo_bien;
     EXCEPTION
          WHEN no_null THEN
               ROLLBACK;
               RETURN no_null_violado;
          WHEN DUP_VAL_ON_INDEX THEN
               ROLLBACK;
               RETURN elemento_existente;
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN SQLCODE;
     END inserta_oficina;

     --Borramos una oficina
     FUNCTION borra_oficina( id_oficina NUMBER )
     RETURN NUMBER IS
          num_ofi NUMBER;
     BEGIN
          SELECT COUNT(*) INTO num_ofi FROM oficinas
          WHERE identificador = id_oficina;
          IF (num_ofi = 0) THEN
               RETURN elemento_inexistente;
          END IF;
          DELETE oficinas WHERE identificador = id_oficina;
          COMMIT;
          RETURN todo_bien;
     EXCEPTION
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN SQLCODE;
     END borra_oficina;

     --Borramos una familia
     FUNCTION borra_familia( id_familia NUMBER )
     RETURN NUMBER IS
          num_fam NUMBER;
     BEGIN
          SELECT COUNT(*) INTO num_fam FROM familias
          WHERE identificador = id_familia;
          IF (num_fam = 0) THEN
               RETURN elemento_inexistente;
          END IF;
          DELETE familias WHERE identificador = id_familia;
          COMMIT;
          RETURN todo_bien;
     EXCEPTION
          WHEN OTHERS THEN
```

```
                ROLLBACK;
                RETURN SQLCODE;
     END borra_familia;

     --Borramos un agente
     FUNCTION borra_agente( id_agente NUMBER )
     RETURN NUMBER IS
          num_ag NUMBER;
     BEGIN
          SELECT COUNT(*) INTO num_ag FROM agentes
          WHERE identificador = id_agente;
          IF (num_ag = 0) THEN
               RETURN elemento_inexistente;
          END IF;
          DELETE agentes WHERE identificador = id_agente;
          COMMIT;
          RETURN todo_bien;
     EXCEPTION
          WHEN OTHERS THEN
               ROLLBACK;
               RETURN SQLCODE;
     END borra_agente;
END call_center;
```

Autoevaluación

En el paquete **call_center**, la función **inserta_agente** recibe un registro de tipo **agente** y comprueba que el agente pertenezca **o bien a una familia o bien a una oficina**, pero no a ambas ni a ninguna. Después intenta insertarlo y devuelve distintos códigos según el resultado.

Supongamos que se llama a la función con un agente que tiene:

```
1 | mi_agente.familia := 15;
2 | mi_agente.oficina := 3;
```

y ambos valores existen realmente en la base de datos.

¿Cuál será el comportamiento más probable de la función?

- ○ Insertará el agente correctamente porque ambas referencias existen y devolverá todo_bien.
- ○ Intentará insertar el agente y, al detectar dos claves foráneas válidas, devolverá padre_existente.
- ○ No llegará a ejecutar el INSERT, devolverá operacion_no_permitida y no será necesario hacer ROLLBACK.

- [ ] Ejecutará el INSERT, pero después lanzará referencia_encontrada y devolverá padre_existente.

Incorrecta. La función no permite que familia y oficina tengan valor al mismo tiempo. Antes del INSERT comprueba esa situación y la considera una operación no válida.

Incorrecta. Aunque ambas referencias existan, el código devuelve directamente operacion_no_permitida. La excepción referencia_encontrada ni siquiera se utiliza en esta función.

Correcta. Al cumplirse:

```
mi_agente.familia IS NOT NULL
AND mi_agente.oficina IS NOT NULL
```

la función ejecuta inmediatamente:

```
RETURN operacion_no_permitida;
```

Por tanto, no se ejecuta el INSERT y tampoco es necesario hacer ROLLBACK, ya que todavía no se ha realizado ninguna modificación.

Incorrecta. La función no lanza la excepción referencia_encontrada en ese caso. La validación se realiza antes de insertar y termina mediante un RETURN.

Solución

1. Incorrecto
2. Incorrecto
3. Opción correcta
4. Incorrecto

Licencia: licencia propietaria